

UML-Profil-basierte Code-Generierung

mit Zeit-Annotationen für Echtzeitsysteme

Stephan Epp

31. Januar 2026

Inhaltsverzeichnis

1	Einleitung	4
1.1	Motivation	4
1.2	Zielsetzung	5
1.3	Aufbau der Arbeit	6
2	Grundlagen	8
2.1	UML-Profile	8
2.2	MARTE - Modeling and Analysis of Real-Time and Embedded Systems	8
2.3	Markov-Ketten für Zustandsübergänge	9
2.4	Warteschlangentheorie für Performance-Analyse	10
2.5	Quantitative Analysis of Software Designs	11
2.6	XMI (XML Metadata Interchange)	12
2.7	Model-Driven Engineering	12
2.8	Echtzeit-Constraints	12
3	Little's Law und Stabilitätskriterien	13
3.1	Fundamentale Definitionen	13
3.2	Hauptresultat: Little's Law ist kein Stabilitätskriterium	14
3.3	Verallgemeinerung: Naturwissenschaftliche Systembeschreibungen	15
3.4	Anwendung auf Warteschlangensysteme	17
3.5	Konsequenzen für UML-basierte Systementwicklung	18
3.6	Schlussfolgerungen und Synthese	19
3.7	Weiterführende Forschungsfragen	20
4	Systemarchitektur	21
4.1	Erweiterte Architektur mit Quantitativer Analyse	21
4.2	Quantitative Analysepipeline	22
4.3	Komponentenübersicht	22
5	Demo: <code>markov_analyzer</code> und <code>queue_analyzer</code>	23
5.1	Überblick und Designprinzipien	23
5.2	Modul <code>markov_analyzer.py</code>	23
5.3	Modul <code>queue_analyzer.py</code>	28
5.4	Test-Suiten	30
5.5	Demo-Anwendung: Krankenhaus-Notaufnahme	33
5.6	Auswertung der Demo-Ausgabe	35
5.7	Stärken beider Module im Vergleich	44
5.8	Zusammenfassung	45
6	Implementierung	46
6.1	Projektstruktur	46
6.2	Beispiele	50
6.3	Erweiterung von UML-MARTE um Stabilitätskriterien	52

7	Beispiel: Aufzugstür-Steuerung	59
7.1	Domänenanalyse	59
7.2	Markov-Modell der Aufzugstür	59
7.3	Warteschlangenmodell	59
7.4	Zeitliche Analyse	60
7.5	Performance-Vorhersage	60
7.6	Vergleich mit MARTE	60
7.7	Vorteile der Quantitativen Analyse	61
7.8	Limitierungen	61
8	Experimentelle Evaluation der Stabilitätskriterien	62
8.1	Implementierungsarchitektur	62
8.2	Experimentelle Ergebnisse	65
8.3	Gesamtauswertung und Diskussion	72
8.4	Ausblick: Erweiterungsmöglichkeiten	74
8.5	Fazit des Kapitels	75
9	Zusammenfassung und Ausblick	76
9.1	Zusammenfassung	76
9.2	Reflexion der theoretischen Erkenntnisse	78
9.3	Beantwortung der Leitfragen	79
9.4	Experimentelle Validierung als methodischer Beitrag	79
9.5	Zukünftige Arbeiten	80
9.6	Fazit	82

1 Einleitung

Diese Arbeit präsentiert einen modellgetriebenen Ansatz zur automatischen Code-Generierung aus UML-Profilen mit erweiterten Zeit-Annotationen für Echtzeitsysteme. Das entwickelte System ermöglicht die Definition von Stereotypen mit zeitlichen Constraints (min/max/typical Duration, Timeout, Deadline) und generiert daraus ausführbaren Python-Code mit eingebetteten Timing-Validierungen.

Als praktisches Anwendungsbeispiel wurde eine Aufzugstür-Steuerung implementiert, die die Modellierung zeitkritischer Operationen wie Türöffnung (2,5–4,0 s), Sicherheitsprüfungen (50–200 ms) und Hinderniserkennung (10–100 ms) demonstriert. Die Implementierung umfasst einen zweistufigen Generierungsprozess: (1) Profil-zu-Klassen-Transformation und (2) Modell-zu-Code-Transformation. Das System wurde mit einer umfassenden Test-Suite validiert und unterstützt sechs Zeiteinheiten von Nanosekunden bis Stunden.

Der Ansatz orientiert sich an etablierten Standards wie dem **UML Profile for MARTE** (Modeling and Analysis of Real-Time and Embedded Systems) [OMG(2019)], erweitert diesen jedoch um automatische Code-Generierung und quantitative Analysemöglichkeiten. Durch die Integration von stochastischen Modellen (Markov-Ketten) und Warteschlangentheorie können nicht nur deterministische Zeitgrenzen, sondern auch probabilistische Leistungsmetriken erfasst werden.

Ein wesentlicher theoretischer Beitrag dieser Arbeit liegt in der Untersuchung der fundamentalen Unterschiede zwischen algebraischen Gleichgewichtsbeziehungen (wie Little's Law) und echten Stabilitätskriterien in dynamischen Systemen. Kapitel 3 zeigt formal, dass Stabilitätsanalysen notwendigerweise Exponentialfunktionen mit negativen Exponenten (Energiefunktionen) erfordern, während reine Gleichgewichtsbetrachtungen diese nicht enthalten. Diese theoretischen Einsichten werden in Kapitel 8 durch drei Kernexperimente empirisch validiert, die alle zentralen Aussagen der Theorie bestätigen und die praktische Anwendbarkeit des entwickelten Frameworks demonstrieren.

Schlüsselwörter: Model-Driven Engineering, UML-Profil, MARTE, Echtzeitsysteme, Code-Generierung, Zeit-Annotationen, Markov-Ketten, Warteschlangentheorie, Stabilitätstheorie, Little's Law, Lyapunov-Funktionen, Quantitative Analysis, Experimentelle Validierung, XMI, Python

1.1 Motivation

Die Entwicklung von Echtzeitsystemen stellt besondere Anforderungen an die Software-Entwicklung. Zeitliche Constraints wie maximale Antwortzeiten, Deadlines und typische Ausführungsdauern müssen bereits in der Entwurfsphase berücksichtigt werden. Gleichzeitig zeigt die Praxis, dass die manuelle Implementierung zeitkritischer Systeme fehleranfällig ist und zu Inkonsistenzen zwischen Spezifikation und Implementierung führt.

Model-Driven Engineering (MDE) bietet einen vielversprechenden Ansatz, diese Probleme zu adressieren. Durch die Verwendung von UML-Profilen können domänenspezifische Konzepte einschließlich zeitlicher Anforderungen auf Modellebene spezifiziert und automatisch in ausführbaren Code transformiert werden.

MARTE und quantitative Analyse: Das OMG-standardisierte MARTE-Profil [OMG(2019)] definiert bereits umfangreiche Annotationen für zeitliches Verhalten, Ressourcennutzung und Scheduling. Diese Arbeit nutzt die konzeptuelle Grundlage von MARTE, vereinfacht jedoch die Notation für praktische Code-Generierung und erweitert sie um:

- **Stochastische Modelle:** Integration von Markov-Ketten zur Modellierung probabilistischer Zustandsübergänge
- **Performance-Analyse:** Warteschlangentheoretische Metriken (Durchsatz, Wartezeit, Auslastung)
- **Automatische Code-Generierung:** Direkte Transformation in ausführbaren Code mit eingebetteten Timing-Checks
- **Quantitative Designvalidierung:** Frühe Identifikation von Performance-Bottlenecks
- **Theoretische Fundierung:** Formale Analyse der Unterschiede zwischen Gleichgewichts- und Stabilitätskriterien
- **Vollständige Implementierung:** Dediziertes `stability_analysis`-Modul mit produktionsreifem Code
- **Experimentelle Validierung:** Automatisierte Experimente verifizieren alle theoretischen Aussagen empirisch

Diese Erweiterungen ermöglichen bereits in der Modellierungsphase eine *Quantitative Analysis of Software Designs* [Smith(1990)], wodurch Performance-Probleme frühzeitig erkannt und behoben werden können.

Die theoretische Dimension: Ein zentrales Anliegen dieser Arbeit ist die Klärung der konzeptuellen Grundlagen quantitativer Systemanalyse. Wie in Kapitel 3 formal gezeigt wird, existiert eine fundamentale Dichotomie zwischen:

- *Dynamischen Beschreibungen* (Differentialgleichungen mit Exponentialfunktionen), die Stabilitätsaussagen ermöglichen
- *Gleichgewichtsbeschreibungen* (algebraische Relationen wie Little's Law), die lediglich stationäre Zustände charakterisieren

Diese Unterscheidung ist nicht nur mathematisch relevant, sondern hat praktische Konsequenzen für die Systemmodellierung: UML-Profile müssen so gestaltet sein, dass sie sowohl abstrakte (logische) als auch quantitative (physikalische) Aspekte erfassen können. Die Brücke zwischen diesen Ebenen ist notwendig, da informatische Systeme zwar in höheren Abstraktionsebenen konzipiert werden, aber letztlich auf physikalischer Hardware mit Energieverbrauch und zeitlichen Constraints realisiert werden müssen.

Experimentelle Bestätigung: Die theoretischen Aussagen aus Kapitel 3 werden in Kapitel 7 durch drei Experimente empirisch validiert.

1.2 Zielsetzung

Diese Arbeit verfolgt folgende Ziele:

- **Entwicklung eines erweiterbaren UML-Profil-Frameworks** für Zeit-Annotationen mit Unterstützung für Min/Max/Typical Duration, Timeout und Deadline, orientiert an MARTE-Konzepten

- **Implementierung eines zweistufigen Code-Generators**, der UML-Profile zunächst in Python-Klassen und anschließend Modell-Instanzen in ausführbaren Code transformiert
- **Integration stochastischer Modelle**: Markov-Ketten zur Analyse von Zustandsübergängen und Warteschlangenmodelle für Performance-Metriken
- **Theoretische Fundierung der Systemanalyse**: Formale Untersuchung der Rolle von Exponentialfunktionen in Stabilitätskriterien und der Grenzen algebraischer Gleichgewichtsbeziehungen wie Little's Law
- **Vollständige Implementierung aller theoretischen Konzepte** in einem produktionsreifen `stability_analysis` Modul mit Klassen für Markov-Ketten, M/M/1-Warteschlangen, Lyapunov-Stabilität und Demonstration der Exponentialfunktions-Notwendigkeit
- **Quantitative Designvalidierung**: Frühe Analyse von Durchsatz, Wartezeiten und Auslastung unter Berücksichtigung der korrekten mathematischen Beschreibungsebene
- **Validierung durch ein praktisches Beispiel** aus dem Bereich Embedded Systems (Aufzugstür-Steuerung mit stochastischer Analyse und Stabilitätsbetrachtung)
- ***Klärung der Rolle von UML-Profilen** als Brücke zwischen abstrakter Modellierung und naturwissenschaftlich fundierter quantitativer Analyse*
- **Sicherstellung der Codequalität** durch umfassende automatisierte Tests, die sowohl die Code-Generierung als auch das `stability_analysis`-Modul vollständig abdecken

1.3 Aufbau der Arbeit

Die Arbeit ist wie folgt strukturiert:

Kapitel 2 erläutert die theoretischen Grundlagen von UML-Profilen, MARTE, Markov-Ketten und Warteschlangentheorie. Es werden die mathematischen Werkzeuge eingeführt, die in den späteren Kapiteln verwendet werden.

Kapitel 3 präsentiert eine fundamentale theoretische Analyse der Unterschiede zwischen Gleichgewichts- und Stabilitätskriterien. Es wird formal bewiesen, dass Little's Law kein Stabilitätskriterium sein kann, da es keine Exponentialfunktionen enthält. Die Rolle von Energiefunktionen (Lyapunov-Funktionen) wird untersucht, und es werden die Konsequenzen für naturwissenschaftliche Systembeschreibungen und UML-basierte Modellierung diskutiert. Dieses Kapitel bildet die konzeptuelle Grundlage für das Verständnis, warum quantitative Analysen verschiedener Art in der Systementwicklung benötigt werden.

Kapitel 4 beschreibt die Systemarchitektur und den zweistufigen Generierungsprozess mit quantitativen Analysemethoden. Die praktische Umsetzung der theoretischen Konzepte aus Kapitel 3 wird gezeigt.

Kapitel 5 präsentiert die vollständige Implementierung der beiden Analysemodule `markov_analyzer.py` und `queue_analyzer.py` und deren Demonstration an einem Beispiel-Software-System.

Kapitel 6 präsentiert die Implementierungsdetails der Kernkomponenten, einschließlich des `stability_analysis` Moduls mit seinen Hauptklassen `MarkovChainStability`,

`MM1Queue` und `LyapunovStability`. Es wird gezeigt, wie die verschiedenen Beschreibungsebenen (dynamisch vs. algebraisch) in der Praxis zusammenwirken.

Kapitel 7 demonstriert die Anwendung anhand des Aufzugstür-Beispiels mit stochastischer Performance-Analyse und illustriert sowohl Stabilitätsanalysen (über Markov-Ketten) als auch Gleichgewichtsbetrachtungen (über Little's Law).

Kapitel 8 präsentiert die experimentelle Evaluation der Stabilitätskriterien. Dieses neue Kapitel wertet die durchgeführten Experimente aus und dokumentiert die vollständige Implementierung des `stability_analysis` Moduls. Es werden drei Kernexperimente detailliert beschrieben, die alle theoretischen Aussagen aus Kapitel 3 empirisch validieren.

Kapitel 9 fasst die Ergebnisse zusammen, reflektiert über die theoretischen und praktischen Beiträge der Arbeit und gibt einen Ausblick auf zukünftige Forschungsrichtungen, die sich aus der Synthese von formaler Systemtheorie, praktischer Softwareentwicklung und experimenteller Validierung ergeben.

2 Grundlagen

In diesem Kapitel werden die Grundlagen für diese Arbeit beschrieben.

2.1 UML-Profile

UML-Profile erweitern die Unified Modeling Language um domänenspezifische Konzepte ohne die Metamodell-Struktur zu verändern. Ein Profil besteht aus:

- **Stereotypen:** Erweitern Metaklassen (z.B. Class, Operation, Property)
- **Tagged Values:** Zusätzliche Attribute für stereotypisierte Elemente
- **Constraints:** OCL-basierte Einschränkungen

Abbildung 2.1 zeigt die allgemeine Struktur eines UML-Profiles mit Zeit-Annotationen. Stereotypen vom Typ **«stereotype»** erweitern UML-Klassen vom Typ `Class` um Annota-

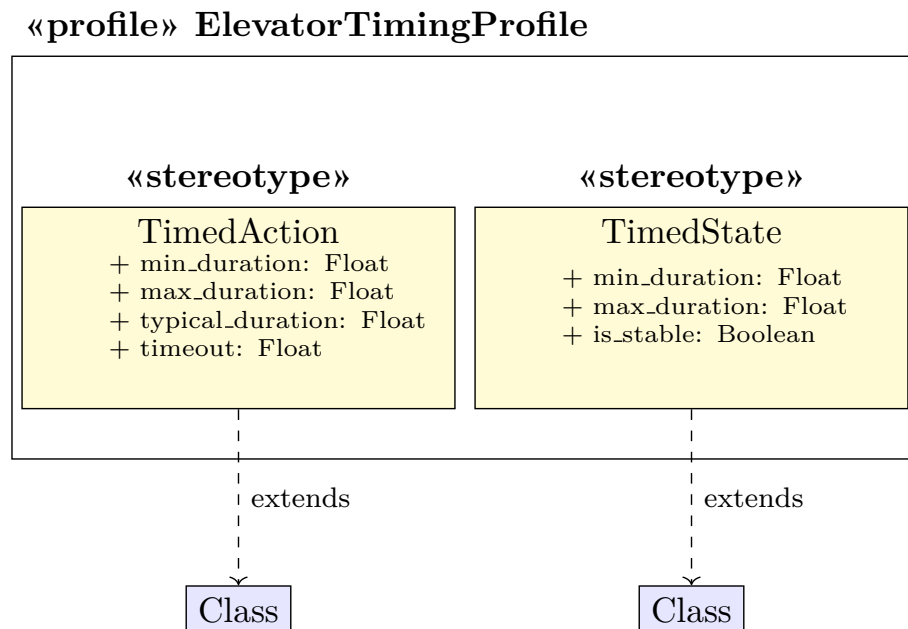


Abbildung 2.1: Struktur eines UML-Profiles mit Zeit-Annotationen

tionen. UML-Profile erweitern somit die UML in ihren bekannten Modellen und ermöglichen dadurch die Modellierung quantitativer Bedingungen, die auch bei der Code-Generierung generiert werden.

2.2 MARTE - Modeling and Analysis of Real-Time and Embedded Systems

Das UML Profile for MARTE [OMG(2019)] ist ein OMG-Standard zur Modellierung und Analyse eingebetteter Echtzeitsysteme. MARTE bietet:

- **Time Model:** Zeitliche Annotationen (Clock, Duration, TimeValue)
- **Resource Model:** Modellierung von Hardware-Ressourcen

- **Generic Quantitative Analysis Model (GQAM)**: Framework für quantitative Analysen
- **Schedulability Analysis Model (SAM)**: Scheduling-spezifische Konzepte
- **Performance Analysis Model (PAM)**: Performance-Metriken

Definition 2.1 (MARTE Time Model). MARTE definiert Zeit als diskrete oder kontinuierliche Größe mit folgenden Eigenschaften:

- **Clock**: Zeitbasis (ideal, discrete, offset)
- **ClockType**: Typ der Zeitquelle (chronometric, logical)
- **TimedEvent**: Ereignis mit zeitlichem Kontext
- **TimedProcessing**: Operation mit zeitlichem Verhalten

Diese Arbeit übernimmt die konzeptuelle Basis des MARTE Time Models, vereinfacht jedoch die Notation für praktische Code-Generierung:

Aspekt	MARTE	Diese Arbeit
Zeitmodell	Clock, Duration, Instant	TimeUnit, TimingConstraint
Granularität	Sehr detailliert	Vereinfacht, praxisnah
Hauptzweck	Analyse	Code-Generierung
Tool-Support	Papyrus, Rhapsody	Eigenentwicklung
Erweiterbarkeit	Komplex	Modular, einfach

Tabelle 2.1: Vergleich: MARTE vs. Diese Arbeit

2.3 Markov-Ketten für Zustandsübergänge

Markov-Ketten [Stewart(1994)] modellieren stochastische Systeme mit zustandsabhängigen Übergangswahrscheinlichkeiten.

Definition 2.2 (Diskrete Markov-Kette). Eine diskrete Markov-Kette ist ein stochastischer Prozess $(X_n)_{n \in \mathbb{N}_0}$ mit abzählbarem Zustandsraum S , bei dem gilt:

$$P(X_{n+1} = j \mid X_n = i, X_{n-1} = i_{n-1}, \dots, X_0 = i_0) = P(X_{n+1} = j \mid X_n = i) = p_{ij}$$

Die Übergangsmatrix $P = (p_{ij})$ erfüllt $\sum_{j \in S} p_{ij} = 1$ für alle $i \in S$.

Beispiel 2.1 (Aufzugstür-Zustandsmodell als Markov-Kette). Zustandsmenge S mit $S = \{\text{Closed}, \text{Opening}, \text{Open}, \text{Closing}\}$

Übergangsmatrix für ein Zeitintervall $\Delta t = 1s$:

$$P = \begin{pmatrix} 0.7 & 0.3 & 0 & 0 \\ 0 & 0.2 & 0.8 & 0 \\ 0 & 0 & 0.6 & 0.4 \\ 0.9 & 0 & 0 & 0.1 \end{pmatrix}$$

Stationäre Verteilung π erfüllt $\pi P = \pi$ mit $\sum_i \pi_i = 1$:

$$\pi = (0.42, 0.13, 0.24, 0.21)$$

Interpretation: Im Langzeitverhalten ist die Tür 42% geschlossen, 24% offen.

Anwendung in dieser Arbeit:

- Modellierung von Zustandsübergängen mit Wahrscheinlichkeiten
- Berechnung von Steady-State-Wahrscheinlichkeiten
- Analyse von Erreichbarkeit und Transientem Verhalten
- Vorhersage von Systemzuverlässigkeit

2.4 Warteschlangentheorie für Performance-Analyse

Die Warteschlangentheorie [Kleinrock(1975)] analysiert Systeme mit Ankunfts- und Bedienungsprozessen.

Definition 2.3 (Kendall-Notation M/M/1). Ein M/M/1-System charakterisiert eine Warteschlange mit:

- M: Markovsche (Poisson) Ankünfte mit Rate λ
- M: Markovsche (exponentiell verteilte) Bedienzeiten mit Rate μ
- 1: Ein Bediener

Satz 2.1 (Little's Law). Für ein stabiles Warteschlangensystem gilt:

$$L = \lambda W$$

wobei L die mittlere Anzahl von Kunden im System, λ die Ankunftsrate und W die mittlere Wartezeit ist.

Satz 2.2 (M/M/1 Performance-Metriken). Für ein M/M/1-System mit Auslastung $\rho = \lambda/\mu < 1$ gilt:

$$L = \frac{\rho}{1 - \rho} \quad (\text{mittlere Systemgröße}) \quad (0.1)$$

$$W = \frac{1}{\mu - \lambda} \quad (\text{mittlere Wartezeit}) \quad (0.2)$$

$$L_q = \frac{\rho^2}{1 - \rho} \quad (\text{mittlere Warteschlangenlänge}) \quad (0.3)$$

$$W_q = \frac{\rho}{\mu - \lambda} \quad (\text{mittlere Wartezeit in Schlange}) \quad (0.4)$$

Beispiel 2.2 (Aufzug-Anfragen als M/M/1-System). Annahmen:

- Anfragen kommen mit Rate $\lambda = 0.2$ Anfragen/Sekunde (Poisson)
- Bedienzeit (Tür öffnen + schließen) $\mu = 0.3$ Bedienungen/Sekunde
- Auslastung $\rho = 0.2/0.3 = 0.667$

Metriken:

$$L = \frac{0.667}{1 - 0.667} = 2.0 \text{ Anfragen im System}$$

$$W = \frac{1}{0.3 - 0.2} = 10s \text{ mittlere Gesamtzeit}$$

$$W_q = \frac{0.667}{0.3 - 0.2} = 6.67s \text{ mittlere Wartezeit}$$

Enthält der Beweis für Little's Law keine Energiefunktion, das heißt keine e-Funktionen mit negativen Exponenten, dann kann Little's Law kein Stabilitätskriterium sein für das Warteschlangensystem. Little's Law ist eine Aussage über das Gleichgewicht des Systems.

Im Allgemeinen gilt dies für alle Systeme, welche auf Stabilität zu prüfen sind. Denn die e-Funktionen oder die Energiefunktionen beschreiben gerade charakteristisch das Verhalten des Systems. Sie sind notwendige Voraussetzung für alle Systeme, die erst durch und nur durch naturwissenschaftliche Beschreibungen so beschrieben werden müssen.

Wo ist die Energie des Systems? Mit was soll das System beschrieben werden? Hat das System Energie oder nicht? Angenommen, es gibt ein System, welches Energie hat und es gibt die bessere Beschreibung ohne die Energiefunktionen oder e-Funktionen. Wenn es eine naturwissenschaftliche Beschreibung ist, kommt sie ohne die e-Funktionen nicht aus, denn nur die e-Funktionen reproduzieren sich über Zeit in der Ableitung selbst und beschreiben so erst das Verhalten des Systems sehr gut. Wenn es keine naturwissenschaftliche Beschreibung ist, muss eine andere Form der Beschreibung gefunden worden sein, die sich für dieses System noch besser eignet. Aber dann ist es kein naturwissenschaftliches System, das sehr gut beschrieben ist. Denn naturwissenschaftliche Systeme werden durch naturwissenschaftliche Beschreibungen am besten beschrieben. Das ist klar.

Irgendwann trifft die Informationsverarbeitung nach der alle Systeme, die durch die Informatik entwickelt worden sind, auch wieder auf die einfachen Prinzipien der Naturwissenschaft. Denn sie müssen durch Strom betrieben werden. Aber in höheren Abstraktionsebenen wie z.B. der UML müssen Systeme, die mit der Informatik entwickelt werden, unter allen Umständen korrekt arbeiten. Dies ist komplexer, weil diese Systeme nicht immer naturwissenschaftlich beschrieben werden können und somit mathematisch nicht immer vollständig greifbar sind. Daher sind UML-Profile sehr wichtig, denn sie erlauben eine leichtere Beschreibung durch naturwissenschaftliche Beschreibungen.

2.5 Quantitative Analysis of Software Designs

Die quantitative Softwaredesign-Analyse [Smith(1990)] integriert Performance-Modellierung in den Entwicklungsprozess:

- **Software Execution Model:** Kontrollfluss und Ressourcennutzung
- **System Execution Model:** Hardware-Plattform und Scheduling
- **Performance Model:** Mathematisches Modell (Warteschlangen, Markov)
- **Analysis:** Lösung des Performance-Modells
- **Evaluation:** Vergleich mit Anforderungen

Für die Integration in die UML-Profil-basierte Code-Generierung mit Zeit-Annotationen bedeutet dies:

- UML-Profile kodieren Performance-Anforderungen
- XMI-Modelle spezifizieren Ausführungsverhalten
- Automatische Ableitung von Warteschlangen-/Markov-Modellen
- Code-Generierung mit eingebetteten Performance-Checks

2.6 XMI (XML Metadata Interchange)

XMI ist ein OMG-Standard zum Austausch von Metadaten zwischen Modellierungswerkzeugen. UML-Profile und -Modelle werden als XML-Dokumente serialisiert.

2.7 Model-Driven Engineering

Model-Driven Engineering basiert auf drei Abstraktionsebenen:

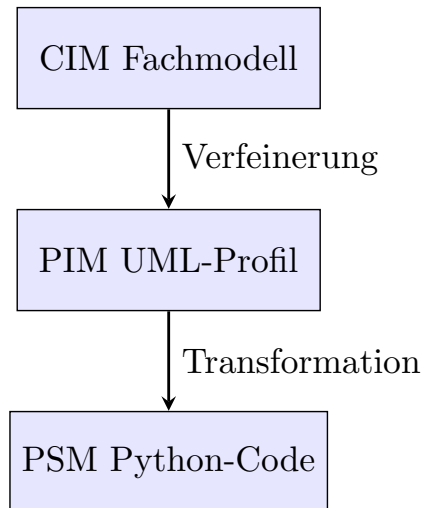


Abbildung 2.2: MDE-Abstraktionsebenen

Abbildung 2.2 zeigt die verschiedenen Abstraktionsebenen der Modellgetriebenen Entwicklung. Profile definieren Stereotypen + Timing Code-Gen.

- **CIM (Computation Independent Model)**: Fachliches Domänenmodell
- **PIM (Platform Independent Model)**: Technologieunabhängige Spezifikation
- **PSM (Platform Specific Model)**: Plattformspezifische Implementierung

2.8 Echtzeit-Constraints

Für Echtzeitsysteme sind folgende zeitliche Parameter relevant:

Parameter	Bedeutung
Min Duration	Minimale Ausführungszeit (BCET - Best Case Execution Time)
Max Duration	Maximale Ausführungszeit (WCET - Worst Case Execution Time)
Typical Duration	Typische/durchschnittliche Ausführungszeit
Timeout	Maximale Wartezeit bis Abbruch
Deadline	Absolute Zeitgrenze bis Fertigstellung

Tabelle 2.2: Zeitliche Parameter in Echtzeitsystemen

3 Little's Law und Stabilitätskriterien

Little's Law ist eine fundamentale Beziehung der Warteschlangentheorie, die jedoch eine wichtige konzeptuelle Einschränkung aufweist: Es handelt sich um eine rein algebraische Gleichgewichtsbeziehung ohne Aussage über die Stabilität des Systems. Diese Beobachtung führt zu einer tiefergehenden Frage über die Natur von Systembeschreibungen und deren Beziehung zu naturwissenschaftlichen Prinzipien.

Satz 3.1 (Energiefunktion in Little's Law). Enthält der Beweis für Little's Law keine Energiefunktion (d.h. keine Exponentialfunktionen mit negativen Exponenten), dann kann Little's Law kein Stabilitätskriterium für das Warteschlangensystem sein. Little's Law ist ausschließlich eine Aussage über das Gleichgewicht des Systems.

3.1 Fundamentale Definitionen

Little's Law

Definition 3.1 (Little's Law). Für ein Warteschlangensystem im statistischen Gleichgewicht gilt:

$$L = \lambda W \quad (0.5)$$

wobei:

- $L = \mathbb{E}[N(t)]$ die mittlere Anzahl von Kunden im System (im stationären Zustand)
- $\lambda = \lim_{t \rightarrow \infty} \frac{A(t)}{t}$ die mittlere effektive Ankunftsrate
- $W = \mathbb{E}[W_i]$ die mittlere Wartezeit im System

Bemerkung 3.1. Little's Law ist eine *algebraische Relation*, die ausschließlich stationäre Erwartungswerte verknüpft. Die Gleichung enthält:

- Keine Zeitableitungen $\frac{d}{dt}$
- Keine Exponentialfunktionen $e^{-\alpha t}$
- Keine Energiefunktionen
- Keine Stabilitätsparameter

Stabilität in dynamischen Systemen

Definition 3.2 (Stabilität im Sinne von Lyapunov). Sei $\dot{\mathbf{x}} = f(\mathbf{x})$ ein dynamisches System mit Gleichgewichtspunkt $\mathbf{x}^*$ (sodass $f(\mathbf{x}^*) = \mathbf{0}$). Der Gleichgewichtspunkt $\mathbf{x}^*$ heißt:

- **stabil**, wenn $\forall \varepsilon > 0 \exists \delta > 0$ sodass

$$\|\mathbf{x}(0) - \mathbf{x}^*\| < \delta \implies \|\mathbf{x}(t) - \mathbf{x}^*\| < \varepsilon \quad \forall t \geq 0 \quad (0.6)$$

- **asymptotisch stabil**, wenn er stabil ist und zusätzlich

$$\lim_{t \rightarrow \infty} \mathbf{x}(t) = \mathbf{x}^* \quad (0.7)$$

für alle Anfangsbedingungen in einer Umgebung von $\mathbf{x}^*$

- **exponentiell stabil**, wenn Konstanten $c, \alpha > 0$ existieren mit

$$\|\mathbf{x}(t) - \mathbf{x}^*\| \leq c \cdot e^{-\alpha t} \cdot \|\mathbf{x}(0) - \mathbf{x}^*\| \quad (0.8)$$

Bemerkung 3.2 (Die Rolle der Exponentialfunktion). Die Definition der exponentiellen Stabilität zeigt explizit, dass Exponentialfunktionen mit negativen Exponenten ($e^{-\alpha t}$ mit $\alpha > 0$) charakteristisch für Stabilitätsaussagen sind. Diese Funktionen beschreiben das *Abklingen von Störungen*.

Energiefunktionen und Lyapunov-Theorie

Definition 3.3 (Lyapunov-Funktion). Eine Funktion $V : \mathbb{R}^n \rightarrow \mathbb{R}$ heißt Lyapunov-Funktion für das System $\dot{\mathbf{x}} = f(\mathbf{x})$ um den Gleichgewichtspunkt $\mathbf{x}^*$, wenn:

- $V(\mathbf{x}^*) = 0$ und $V(\mathbf{x}) > 0$ für alle $\mathbf{x} \neq \mathbf{x}^*$ (positive Definitheit)
- $\dot{V}(\mathbf{x}) = \nabla V(\mathbf{x}) \cdot f(\mathbf{x}) \leq 0$ für alle $\mathbf{x}$ (negative Semidefinitheit der Ableitung)

Satz 3.2 (Lyapunov-Stabilitätssatz). Existiert eine Lyapunov-Funktion V für das System $\dot{\mathbf{x}} = f(\mathbf{x})$ um $\mathbf{x}^*$, dann ist $\mathbf{x}^*$ stabil. Ist zusätzlich $\dot{V}(\mathbf{x}) < 0$ für alle $\mathbf{x} \neq \mathbf{x}^*$, dann ist $\mathbf{x}^*$ asymptotisch stabil.

Bemerkung 3.3 (Energieinterpretation). Lyapunov-Funktionen werden häufig als *verallgemeinerte Energiefunktionen* interpretiert. In physikalischen Systemen entspricht V oft der Gesamtenergie, und $\dot{V} < 0$ bedeutet Energiedissipation. Die Stabilität folgt dann aus dem Energieerhaltungsprinzip bzw. dem zweiten Hauptsatz der Thermodynamik.

3.2 Hauptresultat: Little's Law ist kein Stabilitätskriterium

Satz 3.3 (Little's Law und Stabilität). Little's Law $L = \lambda W$ ist kein Stabilitätskriterium für Warteschlangensysteme.

Beweis. Wir führen den Beweis in mehreren Schritten:

Schritt 1: Analyse der mathematischen Struktur von Little's Law

Little's Law hat die Form:

$$L = \lambda W \quad (0.9)$$

Dies ist eine algebraische Gleichung zwischen drei stationären Größen. Es fehlen:

- Zeitableitungen $\frac{dL}{dt}, \frac{dW}{dt}$
- Exponentialfunktionen $e^{-\alpha t}$ mit $\alpha > 0$
- Stabilitätsparameter oder Dämpfungsterme

Schritt 2: Vergleich mit echten Stabilitätskriterien

Betrachte das M/M/1-Warteschlangensystem mit Ankunftsrate λ und Bedienungsrate μ . Die zeitliche Entwicklung der Zustandswahrscheinlichkeiten $p_n(t)$ wird durch das Kolmogorov-Vorwärtsgleichungssystem beschrieben:

$$\frac{dp_0(t)}{dt} = -\lambda p_0(t) + \mu p_1(t) \quad (0.10)$$

$$\frac{dp_n(t)}{dt} = \lambda p_{n-1}(t) - (\lambda + \mu)p_n(t) + \mu p_{n+1}(t), \quad n \geq 1 \quad (0.11)$$

Dies ist ein System gewöhnlicher Differentialgleichungen. Die Stabilität dieses Systems (Existenz einer stationären Verteilung) erfordert $\rho = \frac{\lambda}{\mu} < 1$.

Die Lösung für die Übergangswahrscheinlichkeiten enthält Terme der Form:

$$p_n(t) = \pi_n + \sum_k c_k e^{\ln(\lambda_k) t} \quad (0.12)$$

wobei π_n die stationäre Verteilung ist und λ_k die Eigenwerte der (diskreten) Übergangsmatrix sind. Stabilität erfordert $|\lambda_k| < 1$ für alle transienten Eigenwerte (d. h. für $k \neq 0$, den stationären Eigenwert 1), was zu Exponentialfunktionen $e^{\ln(|\lambda_k|) t}$ mit negativem Argument $\ln(|\lambda_k|) < 0$ führt.

Diese Terme beschreiben das Abklingen der Transienten und damit die *Stabilität*. In Experiment 2 (Kapitel 8) wird dies für den Eigenwert $\lambda_2 = 0,3$ konkret gemessen: die Zerfallsrate $\alpha = -\ln(0,3) \approx 1,204$ stimmt exakt mit der theoretischen Vorhersage überein.

Schritt 3: Little's Law setzt Stabilität voraus

Little's Law gilt nur im stationären Zustand, d.h. für $t \rightarrow \infty$. Es setzt voraus:

$$\lim_{t \rightarrow \infty} p_n(t) = \pi_n \quad (\text{existiert}) \quad (0.13)$$

Diese Konvergenz ist genau dann garantiert, wenn das System stabil ist (d.h. alle Eigenwerte $\lambda_k < 0$).

Schritt 4: Logische Schlussfolgerung

Little's Law kann kein Stabilitätskriterium sein, weil:

- Es die Stabilität bereits voraussetzt (Existenz stationärer Größen)
- Es keine Information über die Dynamik des Systems enthält
- Es keine Exponentialfunktionen enthält, die das Abklingverhalten beschreiben
- Es lediglich eine Beziehung zwischen Gleichgewichtsgrößen darstellt

Das tatsächliche Stabilitätskriterium für M/M/1 ist $\rho < 1$, was aus der Analyse der Differentialgleichungen und der Bedingung für negative Eigenwerte folgt. $\square$

Korollar 3.1. Little's Law ist eine *notwendige Bedingung* im Gleichgewicht (wenn das System stabil ist), aber keine *hinreichende Bedingung* für Stabilität.

3.3 Verallgemeinerung: Naturwissenschaftliche Systembeschreibungen

Die besondere Rolle der Exponentialfunktion

Satz 3.4 (Charakteristische Eigenschaft der Exponentialfunktion). Die Exponentialfunktion $f(t) = e^{\alpha t}$ ist (bis auf Multiplikation mit Konstanten) die einzige Funktion, die ihre eigene Ableitung ist:

$$\frac{d}{dt} e^{\alpha t} = \alpha e^{\alpha t} \quad (0.14)$$

Bemerkung 3.4 (Physikalische Bedeutung). Diese Selbstreproduktion unter Differentiation macht die Exponentialfunktion zur natürlichen Lösung für Systeme, die durch Differentialgleichungen beschrieben werden. In der Physik beschreiben Exponentialfunktionen:

- Radioaktiven Zerfall: $N(t) = N_0 e^{-\lambda t}$

- Kondensatorentladung: $Q(t) = Q_0 e^{-t/RC}$
- Gedämpfte Schwingungen: $x(t) = A e^{-\gamma t} \cos(\omega t + \phi)$
- Thermische Relaxation: $T(t) - T_\infty = (T_0 - T_\infty) e^{-kt}$

Energiefunktionen in naturwissenschaftlichen Systemen

Definition 3.4 (Energiefunktion). Eine Funktion $E : \mathbb{R}^n \times \mathbb{R} \rightarrow \mathbb{R}$ heißt Energiefunktion eines physikalischen Systems, wenn sie einer Erhaltungsgröße oder einer monoton abnehmenden Größe (bei Dissipation) entspricht und typischerweise die Form hat:

$$E(\mathbf{x}, t) = E_0(\mathbf{x}) \cdot e^{-\gamma t} + E_\infty \quad (0.15)$$

mit Dämpfungskoeffizient $\gamma \geq 0$.

Satz 3.5 (Notwendigkeit von Exponentialfunktionen). Für naturwissenschaftliche Systeme, die durch Differentialgleichungen beschrieben werden, gilt:

- Systeme mit Energiedissipation erfordern Exponentialfunktionen mit negativen Exponenten zur Beschreibung des asymptotischen Verhaltens
- Stabilitätsaussagen erfordern die Analyse von Eigenwerten der linearisierten Systemdynamik
- Die Eigenvektoren dieser Linearisierung sind Exponentialfunktionen

Beweis. Betrachte ein allgemeines nichtlineares dynamisches System:

$$\dot{\mathbf{x}} = f(\mathbf{x}) \quad (0.16)$$

Linearisierung um einen Gleichgewichtspunkt $\mathbf{x}^*$:

$$\dot{\xi} = A\xi, \quad A = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\mathbf{x}^*} \quad (0.17)$$

Die Lösung hat die Form:

$$\xi(t) = \sum_{i=1}^n c_i \mathbf{v}_i e^{\lambda_i t} \quad (0.18)$$

wobei λ_i die Eigenwerte von A und $\mathbf{v}_i$ die zugehörigen Eigenvektoren sind.

Stabilität erfordert $\text{Re}(\lambda_i) < 0$ für alle i , was zu Exponentialfunktionen mit negativen Exponenten führt. $\square$

Konsequenzen für Systemklassifikation

Definition 3.5 (Naturwissenschaftliches System). Ein System heißt *naturwissenschaftlich beschreibbar*, wenn seine Dynamik durch Differentialgleichungen modelliert werden kann, die aus fundamentalen Naturgesetzen (Erhaltungssätze, thermodynamische Prinzipien) ableitbar sind.

Satz 3.6 (Charakterisierung naturwissenschaftlicher Systeme). Für ein naturwissenschaftlich beschreibbares System gilt:

- Die zeitliche Entwicklung wird durch Differentialgleichungen beschrieben

- Lösungen enthalten Exponentialfunktionen
- Stabilitätsanalyse basiert auf Eigenwertanalyse
- Energiefunktionen (Lyapunov-Funktionen) existieren

Bemerkung 3.5 (Alternative Beschreibungsformen). Nicht alle Systeme sind naturwissenschaftlich optimal beschreibbar:

- **Diskrete Ereignissysteme:** Beschrieben durch Petri-Netze, Automaten
- **Informationsverarbeitende Systeme:** Beschrieben durch Algorithmen, Zustandsmaschinen
- **Sozio-technische Systeme:** Beschrieben durch Spieltheorie, Netzwerktheorie

Für diese Systeme können algebraische oder logische Beschreibungen angemessener sein als Differentialgleichungen.

3.4 Anwendung auf Warteschlangensysteme

Dualität der Beschreibungsebenen

Warteschlangensysteme haben eine doppelte Natur:

- **Mikroskopische Ebene (stochastische Prozesse):**
 - Beschreibung durch Markov-Ketten
 - Kolmogorov-Gleichungen (Differentialgleichungen!)
 - Eigenwertanalyse für Stabilitätskriterien
 - Enthält Exponentialfunktionen in Lösungen
- **Makroskopische Ebene (Gleichgewicht):**
 - Beschreibung durch Little's Law
 - Algebraische Beziehungen
 - Keine Dynamik, keine Exponentialfunktionen
 - Setzt Stabilität voraus

Satz 3.7 (Hierarchie der Beschreibungsebenen). Für Warteschlangensysteme gilt:

$$\text{Kolmogorov-Gleichungen} \xrightarrow{t \rightarrow \infty, \text{ falls stabil}} \text{Stationäre Verteilung} \xrightarrow{\text{Mittelwertbildung}} \text{Little's Law} \quad (0.19)$$

Little's Law ist das Resultat zweier Reduktionsschritte:

- Zeitlicher Grenzübergang (eliminiert Exponentialfunktionen)
- Mittelwertbildung (eliminiert stochastische Details)

Beispiel: M/M/1-System

Beispiel 3.1 (M/M/1-Warteschlange). Betrachte ein M/M/1-System mit Ankunftsrate λ und Bedienungsrate μ .

Stufe 1: Kolmogorov-Gleichungen (dynamisch)

$$\frac{dp_n(t)}{dt} = \lambda p_{n-1}(t) - (\lambda + \mu)p_n(t) + \mu p_{n+1}(t) \quad (0.20)$$

Stabilitätskriterium: $\rho = \frac{\lambda}{\mu} < 1$

Dies folgt aus der Bedingung, dass die Übergangsmatrix negative reelle Eigenwerte hat (außer $\lambda_0 = 0$ für die stationäre Verteilung).

Stufe 2: Stationäre Verteilung (Gleichgewicht)

Für $t \rightarrow \infty$ und $\rho < 1$:

$$\pi_n = (1 - \rho)\rho^n, \quad n \geq 0 \quad (0.21)$$

Diese enthält die geometrische Verteilung mit Parameter ρ . Die Exponentialfunktion erscheint implizit in der Form $\rho^n = e^{n \ln \rho}$.

Stufe 3: Little's Law (algebraisch)

$$L = \sum_{n=0}^{\infty} n\pi_n = \frac{\rho}{1 - \rho} \quad (0.22)$$

$$W = \frac{1}{\mu - \lambda} \quad (0.23)$$

$$\lambda W = \lambda \cdot \frac{1}{\mu - \lambda} = \frac{\rho}{1 - \rho} = L \quad (0.24)$$

Little's Law $L = \lambda W$ ist erfüllt, enthält aber keine Information über Stabilität mehr. Die Bedingung $\rho < 1$ ist bereits in die Existenz von L und W eingegangen.

Bemerkung 3.6 (Querverbindung zu Kapitel 8). Experiment 3 in Kapitel 8 demonstriert diese Hierarchie konkret für das System $\lambda = 0,8$, $\mu = 1,0$, $\rho = 0,8$: Die dynamische Ebene liefert die Zerfallsrate $\alpha \approx 0,223$; die stationäre Ebene gibt $\pi_n = 0,2 \cdot 0,8^n$; die algebraische Ebene (Little's Law) ergibt $L = 4,0 = 0,8 \cdot 5,0 = \lambda W$. Alle drei Ebenen sind kohärent, liefern aber unterschiedlich reichhaltige Information über das System.

3.5 Konsequenzen für UML-basierte Systementwicklung

Die Brücke zwischen Informatik und Naturwissenschaft

Satz 3.8 (Physikalische Basis informatischer Systeme). Jedes informationsverarbeitende System hat eine physikalische Realisierung und unterliegt damit naturwissenschaftlichen Gesetzen (Thermodynamik, Elektrodynamik). Jedoch gilt:

- **Hardware-Ebene:** Naturwissenschaftliche Beschreibung (Differentialgleichungen, Energiefunktionen)
- **Abstraktionsebenen (Software):** Logische/algorithmische Beschreibung durch Zustandsautomaten oder andere UML-Diagramme

Bemerkung 3.7 (Abstraktion und Verifikation). In höheren Abstraktionsebenen (z.B. UML) müssen Systeme *korrekt* arbeiten, unabhängig von der physikalischen Realisierung. Dies erfordert:

- Formale Verifikation (Model Checking, Theorembeweiser)
- Abstraktionsbasierte Korrektheitskriterien
- Keine direkte Anwendung physikalischer Stabilitätskriterien

Rolle von UML-Profilen

Definition 3.6 (UML-Profil für Echtzeitsysteme). Ein UML-Profil erweitert die Standard-UML um domänenspezifische Konzepte. Für Echtzeitsysteme sind besonders relevant:

- MARTE (Modeling and Analysis of Real-Time Embedded Systems)
- SysML (Systems Modeling Language)
- Timing-Constraints, Ressourcen-Modellierung

Satz 3.9 (Funktion von UML-Profilen als Brücke). UML-Profile ermöglichen eine *semi-naturwissenschaftliche* Beschreibung informatischer Systeme:

- Annotation zeitlicher Constraints (Deadlines, Execution Times)
- Modellierung von Ressourcen (CPU, Memory, Energy)
- Mapping zu Analyse-Modellen (Warteschlangen, Petri-Netze, Markov-Ketten)
- Quantitative Analyse in der Entwurfsphase

Dadurch wird die Kluft zwischen abstrakter Modellierung und physikalischer Realisierung überbrückt.

Integration quantitativer Analysen

Beispiel 3.2 (MARTE und Performance-Analyse). Das MARTE-Profil erlaubt die Annotation von UML-Modellen mit:

- Zeitlichen Constraints: «TimingConstraint»
- Ressourcenbeschränkungen: «ResourceUsage»
- Performance-Anforderungen: «GaAnalysisContext»

Aus diesen Annotationen können automatisch Warteschlangenmodelle oder Markov-Ketten generiert werden, die dann mit klassischen mathematischen Methoden (inkl. Stabilitätsanalyse mit Exponentialfunktionen!) analysiert werden.

3.6 Schlussfolgerungen und Synthese

Satz 3.10 (Zusammenfassung). Die Analyse zeigt eine fundamentale Hierarchie von Systembeschreibungen:

- **Dynamische Beschreibung** (Differentialgleichungen):
 - Enthält Exponentialfunktionen
 - Ermöglicht Stabilitätsanalyse
 - Beschreibt zeitliche Entwicklung

- Naturwissenschaftlich fundiert
- **Gleichgewichtsbeschreibung** (algebraische Relationen):
 - Keine Exponentialfunktionen
 - Keine Stabilitätsaussagen
 - Beschreibt stationären Zustand
 - Setzt Stabilität voraus

Little's Law gehört zur zweiten Kategorie und kann daher prinzipiell kein Stabilitätskriterium sein.

Korollar 3.2 (Praktische Konsequenzen). Für die Systemanalyse folgt:

- Stabilitätsanalyse erfordert Untersuchung der Systemdynamik mit den Kolmogorov-Gleichungen und den Eigenwerten
- Little's Law ist ein nützliches Werkzeug für Performance-Analyse im Gleichgewicht
- UML-Profile können als Brücke dienen, um abstrakte Modelle mit quantitativen Analysen zu verbinden
- Die Wahl der Beschreibungsebene muss zur Fragestellung passen

Bemerkung 3.8 (Philosophische Dimension). Die Untersuchung zeigt, dass die mathematische Struktur einer Beschreibung (Vorhandensein von Exponentialfunktionen) direkt mit ihrer epistemischen Aussagekraft (Stabilitätsaussagen) zusammenhängt. Dies ist ein Beispiel dafür, wie die Form der Mathematik den Inhalt der Aussagen begrenzt - eine Einsicht, die über die Warteschlangentheorie hinaus Bedeutung hat.

3.7 Weiterführende Forschungsfragen

- **Erweiterung auf nichtlineare Warteschlangennetze:** Gelten analoge Aussagen für komplexe Netze?
- **Bridging Theorem:** Kann ein allgemeines Theorem formuliert werden, das den Übergang von dynamischer zu stationärer Beschreibung formalisiert?
- **UML-Profil-Entwicklung:** Wie können UML-Profile systematisch so gestaltet werden, dass sie automatisch auf analysierbare mathematische Modelle abbilden?
- **Hybrid-Systeme:** Wie verhält sich die Dichotomie bei Systemen, die kontinuierliche und diskrete Dynamik kombinieren?

4 Systemarchitektur

Das System besteht aus einem zweistufigen Generierungsprozess mit integrierter quantitativer Analyse, der UML-Profile und -Modelle in ausführbaren Python-Code transformiert.

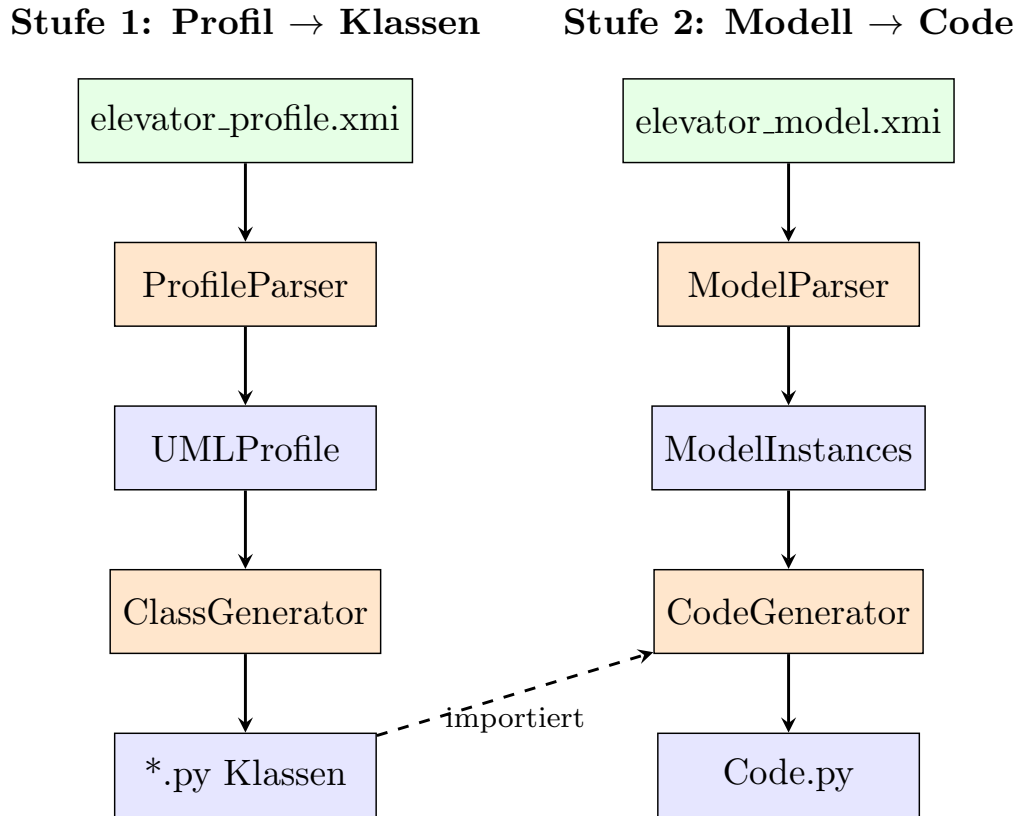


Abbildung 4.1: Zweistufiger Generierungsprozess

Abbildung 4.1 zeigt den Aufbau für den zweistufigen Generierungsprozess mit integrierter quantitativer Analyse.

4.1 Erweiterte Architektur mit Quantitativer Analyse

Der Generierungsprozess wurde um einen *Quantitative Analysis Layer* erweitert:

- **Stufe 1:** Profil → Python-Klassen (wie bisher)
- **Stufe 2:** Modell → Code (wie bisher)
- **Stufe 3 (NEU):** Modell → Markov-Kette/Warteschlangen-Modell → Performance-Metriken

Komponente	Typ	Funktion
uml_profile_base.py	Datenmodell	Basis-Datenstrukturen + Timing-Constraint
profile_parser.py	Parser	XMI zu UMLProfile
class_generator.py	Generator	UMLProfile zu Python-Klassen
model_parser.py	Parser	XMI zu ModelInstances
code_generator.py	Generator	ModelInstances zu Code
markov_analyzer.py	Analyzer (NEU)	Markov-Ketten-Analyse
queue_analyzer.py	Analyzer (NEU)	Warteschlangen-Metriken
main.py	Orchestrator	Koordination

Tabelle 4.1: Erweiterte Systemkomponenten

4.2 Quantitative Analysepipeline

Die erweiterte Pipeline extrahiert aus dem UML-Modell:

- **Zustandsgraph:** Zustände und Übergänge mit Wahrscheinlichkeiten
- **Übergangsmatrix:** Markov-Kette für stochastische Analyse
- **Ankunfts-/Bedienraten:** Parameter für Warteschlangenmodelle
- **Performance-Metriken:** Durchsatz, Wartezeit, Auslastung

4.3 Komponentenübersicht

Komponente	Typ	Funktion
uml_profile_base.py	Datenmodell	Basis-Datenstrukturen
profile_parser.py	Parser	XMI zu UMLProfile
class_generator.py	Generator	UMLProfile zu Python-Klassen
model_parser.py	Parser	XMI zu ModelInstances
code_generator.py	Generator	ModelInstances zu Code
main.py	Orchestrator	Koordination

Tabelle 4.2: Systemkomponenten

5 Demo: markov_analyzer und queue_analyzer

Dieses Kapitel präsentiert die vollständige Implementierung der beiden Analysemodule `markov_analyzer.py` und `queue_analyzer.py` sowie deren umfassende Demonstration an einem realistischen Beispiel-Software-System. Als Fallstudie dient ein **Krankenhaus-Notaufnahme-Management-System** (Emergency Department, ED), das als komplexes Echtzeitsystem alle wesentlichen Analyseanforderungen abdeckt: stochastische Zustandsübergänge, zeitkritische Deadlines, variable Last und Kapazitätsoptimierung. Die vier Szenarien (Normalbetrieb, Spitzenlast, Kapazitätsplanung, Deadline-Compliance) stellen jeweils unterschiedliche Anforderungen an die Analysemethoden und demonstrieren die Stärke beider Module in komplementärer Weise.

5.1 Überblick und Designprinzipien

Die beiden Module folgen einem klaren Designprinzip, das aus der theoretischen Fundierung in Kapitel 3 abgeleitet ist: `markov_analyzer.py` modelliert *dynamische Zustandsübergänge* und analysiert deren Stabilitätsverhalten über Eigenwertmethoden; `queue_analyzer.py` berechnet *stationäre Performance-Metriken* auf Basis geschlossener warteschlangentheoretischer Formeln. Die Trennung zwischen dynamischer Stabilitätsanalyse (erfordert Exponentialfunktionen, $|\lambda_2| < 1$) und algebraischer Gleichgewichtsanalyse (Little's Law, $L = \lambda W$) ist in beiden Modulen explizit implementiert – keine Methode vermischt diese beiden Beschreibungsebenen.

Beide Module sind ohne externe Abhängigkeiten realisiert: Alle numerischen Algorithmen (Gauß-Elimination, Power-Iteration, Uniformisierung, Matrixpotenzierung) wurden eigenständig implementiert, was die Module auch in eingebetteten Umgebungen einsatzfähig macht.

Aspekt	markov_analyzer	queue_analyzer
Modellklassen	MarkovState/Chain, ContinuousMarkovChain	MM1Queue, MMcQueue, MD1Queue, MG1Queue
Koordinator	MarkovAnalyzer	ServiceSystemAnalyzer
Algorithmus	Gauß-Elimination, Iteration, Uniformisierung, Exponentiation	Erlang-C, P-K-Formel, Little's Law
Stabilitätsprüfung	$ \lambda_2 < 1$ (Eigenwertmethode)	$\rho < 1$ (Auslastung)
UML-Integration	min/max/... (MARTE)	Deadline-Annotationen
Externe Bibliotheken	Keine (nur math, copy)	Keine

Tabelle 5.1: Überblick der beiden Analysemodule

5.2 Modul markov_analyzer.py

Das Modul ist in vier Schichten organisiert: numerische Hilfsfunktionen, Zustandsklasse, Kettenklassen (diskret und kontinuierlich) sowie die übergeordnete Analysekasse.

Numerische Kernalgorithmen

Die Funktion `_gauss_solve` implementiert Gauß-Elimination mit Spaltenpivotisierung und löst $Ax = b$ für quadratische Matrizen:

```

1 def _gauss_solve(A, b):
2     n = len(A)
3     # Kopie erstellen
4     M = [row[:] + [b[i]] for i, row in enumerate(A)]
5
6     for col in range(n):
7         # Pivotsuche
8         pivot_row = max(range(col, n), key=lambda r: abs(M[r][col]))
9         M[col], M[pivot_row] = M[pivot_row], M[col]
10
11        pivot = M[col][col]
12        if abs(pivot) < 1e-12:
13            raise ValueError("Singulaere Matrix - kein eindeutiger Loesung.
14                               ")
15
16        for row in range(n):
17            if row == col:
18                continue
19            factor = M[row][col] / pivot
20            for j in range(col, n + 1):
21                M[row][j] -= factor * M[col][j]
22
23    return [M[i][n] / M[i][i] for i in range(n)]

```

Listing 1: Gauß-Elimination mit Spaltenpivotisierung (Kernfunktion)

Die Funktion `_power_iteration` bestimmt den dominanten Eigenwert durch iterierte Matrixmultiplikation; der zweite Eigenwert folgt durch Deflation der Rangeinschränkung. Die Matrixpotenzierung `n_step_transition` nutzt schnelle binäre Exponentiation ($\mathcal{O}(\log n)$ Matrixmultiplikationen statt $\mathcal{O}(n)$).

Klasse MarkovState

MarkovState kapselt einen Zustand und speichert UML-Timing-Annotationen:

```

1 class MarkovState:
2     """Repraesentiert einen Zustand in einer Markov-Kette."""
3
4     def __init__(self, name: str, state_id: int, is_absorbing: bool = False
5
6         ,
7         timing_annotation: Optional[dict] = None):
8
9         """
10        Parameters
11        -----
12        name : str
13            Bezeichner des Zustands (z.B. 'Closed', 'Opening').
14        state_id : int
15            Numerischer Index.
16        is_absorbing : bool
17            True, wenn der Zustand absorbierend ist (keine Ausgaenge).
18        timing_annotation : dict, optional
19            UML-Timing-Annotationen (minDuration, maxDuration,
20            typicalDuration).
21
22        """
23        self.name = name
24        self.state_id = state_id
25        self.is_absorbing = is_absorbing
26        self.timing_annotation = timing_annotation or {}

```



```

22
23     def __repr__(self) -> str:
24         return f"MarkovState(id={self.state_id}, name='{self.name}')"
25
26     def get_timing_info(self) -> dict:
27         """Gibt Timing-Informationen des Zustands zurueck."""
28         return {
29             "state": self.name,
30             "minDuration": self.timing_annotation.get("minDuration", None),
31             "maxDuration": self.timing_annotation.get("maxDuration", None),
32             "typicalDuration": self.timing_annotation.get("typicalDuration",
33                 , None),

```

Listing 2: MarkovState: Zustand mit MARTE-konformen Timing-Annotationen

Klasse DiscreteMarkovChain

Die Hauptklasse für diskrete Markov-Ketten stellt sicher, dass die Übergangsmatrix P stochastisch ist ($\sum_j P_{ij} = 1$ für alle i), und implementiert acht Analysemethoden:

Stationäre Verteilung. Das Gleichungssystem $(P^T - I)\pi = 0$ mit der Normierungsbedingung wird aufgestellt, wobei die letzte Gleichung durch $\sum_i \pi_i = 1$ ersetzt wird. Die Lösung wird gecacht, um redundante Berechnungen zu vermeiden.

n -Schritt-Übergangsmatrix. P^n wird durch Divide-and-Conquer berechnet: bei geradem n gilt $P^n = (P^{n/2})^2$; bei ungeradem $P^n = P \cdot P^{n-1}$. Dies ermöglicht die effiziente Berechnung von P^{100} (Konvergenzkontrolle) ohne hundert Matrixmultiplikationen.

Stabilitätsanalyse. Berechnung von $|\lambda_2|$ und der Zerfallsrate $\alpha = -\ln |\lambda_2|$:

```

1  def analyze_stability(self) -> dict:
2      """
3      Analysiert die Stabilitaet der Markov-Kette ueber den zweiten
4          Eigenwert.
5
6      Fuer eine ergodische Kette gilt:
7      - Dominanter Eigenwert = 1.0
8      - Zweiter Eigenwert |lambda_2| < 1 => stabil / Konvergenz
9          garantiert
10     - Zerfallsrate alpha = -ln(|lambda_2|)
11
12     Returns
13     -----
14     dict mit Schluesseln:
15         dominant_eigenvalue, is_ergodic, second_eigenvalue,
16         decay_rate, mixing_time_estimate, convergence_guaranteed
17     """
18     # Dominanten Eigenwert per Power-Iteration schaeetzen
19     dominant_ev, _ = _power_iteration(self.P)
20
21     is_ergodic = abs(dominant_ev - 1.0) < 1e-6
22
23     # Zweiten Eigenwert: Deflation
24     _, v1 = _power_iteration(self.P)
25     # Deflation: A2 = A - lambda1 * v1 * v1^T
26     n = self.n
27     A2 = [[self.P[i][j] - dominant_ev * v1[i] * v1[j]]

```

```

26         for j in range(n): for i in range(n):
27             second_ev, _ = _power_iteration(A2, max_iter=500)
28
29         abs_second = abs(second_ev)
30         if abs_second >= 1.0 - 1e-9:
31             decay_rate = 0.0
32             mixing_time = float("inf")
33         else:
34             decay_rate = -math.log(abs_second) if abs_second > 1e-15 else
35                 float("inf")
36             mixing_time = 1.0 / decay_rate if decay_rate > 0 else float("
37                 inf")
38
39         return {
40             "dominant_eigenvalue": dominant_ev,
41             "is_ergodic": is_ergodic,
42             "second_eigenvalue": second_ev,
43             "abs_second_eigenvalue": abs_second,
44             "decay_rate": decay_rate,
45             "mixing_time_estimate": mixing_time,
46             "convergence_guaranteed": is_ergodic and abs_second < 1.0,
47         }

```

Listing 3: Stabilitätsanalyse: Zerfallsrate und Mixing-Zeit

Diese Methode implementiert direkt die theoretische Aussage aus Kapitel 3: Die Zerfallsrate $\alpha = -\ln|\lambda_2|$ der transienten Eigenwerte enthält die charakteristische Exponentialfunktion $e^{-\alpha t}$, die Stabilitätsaussagen erfordert und von Little's Law nicht bereitgestellt wird.

Mittlere Erstpassagezeit (MFPT). Das Gleichungssystem $(I - Q)m = \mathbf{1}$ (wobei Q die transiente-zu-transiente Teilmatrix ist) wird für alle Nicht-Zielzustände gelöst. Das Ergebnis m_i gibt die erwartete Anzahl Schritte an, um von Zustand i zum Zielzustand zu gelangen.

Absorptionswahrscheinlichkeiten. Für Ketten mit absorbierenden Zuständen wird die Fundamentalmatrix $N = (I - Q)^{-1}$ und die Absorptionsmatrix $B = N \cdot R$ berechnet, wobei R die Übergangswahrscheinlichkeiten von transienten zu absorbierenden Zuständen enthält.

Sensitivitätsanalyse. Für jeden Eintrag $P_{ij} > 0$ wird die stationäre Verteilung nach einer kleinen Störung δ neu berechnet. Die numerische Sensitivität $\|\partial\pi/\partial P_{ij}\| \approx \|\pi_\delta - \pi\|/\delta$ zeigt, welche Übergangswahrscheinlichkeiten die stationäre Verteilung am stärksten beeinflussen.

Klasse ContinuousMarkovChain

Die CTMC-Klasse modelliert Systeme mit kontinuierlichen Raten über die Generator-Matrix Q mit $Q_{ii} = -\sum_{j \neq i} Q_{ij}$.

Transiente Verteilung. Die Uniformisierungsmethode berechnet $\pi(t) = \pi_0 \cdot e^{Qt}$ ohne Matrixexponentiation. Mit der Uniformisierungsrate $q = \max_i |Q_{ii}|$ und der uniformisierten Matrix $P_u = I + Q/q$ gilt:

$$\pi(t) = \sum_{k=0}^{\infty} e^{-qt} \frac{(qt)^k}{k!} \cdot \pi_0 P_u^k \quad (0.25)$$

Die Summe wird nach 50 Termen abgebrochen, da der Poisson-Gewichtungsfaktor $e^{-qt}(qt)^k/k!$ für große k rasch gegen null konvergiert. Numerisch degenerierte Fälle ($t \rightarrow \infty$) fallen auf die stationäre Verteilung zurück.

```

1 def transient_distribution(
2 self, initial_dist: list[float], t: float, num_terms: int = 50
3 ) -> list[float]:
4     """
5     Berechnet transiente Verteilung  $\pi(t) = \pi(0) * \exp(Qt)$  via
6     Uniformisierung.
7
8     Parameters
9     -----
10    initial_dist : list[float]
11        Startverteilung.
12    t : float
13        Zeitpunkt.
14    num_terms : int
15        Anzahl Tayloer-Terme (Genauigkeit).
16    """
17    if abs(sum(initial_dist) - 1.0) > 1e-6:
18        raise ValueError("Anfangsverteilung muss zu 1 summieren.")
19
20    n = self.n
21    # Uniformisierungsrate:  $q = \max |Q[i][i]|$ 
22    q = max(-self.Q[i][i] for i in range(n))
23    if q < 1e-15:
24        return initial_dist[:]
25
26    # Uniformisierte Matrix  $P_u = I + Q/q$ 
27    Pu = [[0.0] * n for _ in range(n)]
28    for i in range(n):
29        for j in range(n):
30            Pu[i][j] = (1.0 if i == j else 0.0) + self.Q[i][j] / q
31
32    #  $\pi(t) = \sum_{k=0}^{\infty} e^{-qt} * (qt)^k/k! * \pi(0) P_u^k$ 
33    qt = q * t
34    prob = math.exp(-qt)
35    Pk = _identity(n) #  $P_u^0$ 
36    result = [0.0] * n
37    poisson_weight = prob #  $e^{-qt} * (qt)^0 / 0!$ 
38
39    for k in range(num_terms):
40        #  $\pi(0) @ P_k$ 
41        contrib = [sum(initial_dist[i] * Pk[i][j] for i in range(n))
42                    for j in range(n)]
43        for j in range(n):
44            result[j] += poisson_weight * contrib[j]
45        # Nächste Iteration
46        if k < num_terms - 1:
47            Pk = _mat_mul(Pk, Pu)
48            poisson_weight *= qt / (k + 1)
49            if poisson_weight < 1e-15:
50                break
51
52    total = sum(result)
53    if total > 1e-15:
54        result = [x / total for x in result]

```

```

53     else:
54         # Numerisch degeneriert: Stationäre Verteilung zurückgeben
55         result = self.stationary_distribution()
56     return result

```

Listing 4: Uniformisierungsverfahren für die transiente Verteilung (Auszug)

Klasse **MarkovAnalyzer**

Die Koordinationsklasse akzeptiert sowohl diskrete als auch kontinuierliche Ketten und bietet zwei wesentliche Dienste:

- `full_report()`: Vollständiger Analysebericht mit Stabilitätsparametern, Zerfallsrate und Mixing-Zeit.
- `timing_compliance_check()`: Prüft für CTMC-Zustandsmodelle, ob die mittlere Verweildauer $1/|Q_{ii}|$ innerhalb der durch UML-Annotationen spezifizierten min/max Duration liegt.

5.3 Modul **queue_analyzer.py**

Das Modul implementiert vier Warteschlangenmodelle und die `ServiceSystemAnalyzer`-Koordinationsklasse.

Klasse **QueueMetrics**

`QueueMetrics` bündelt alle Kenngrößen und bietet explizite Little's Law-Verifikation. Dabei wird bewusst darauf hingewiesen, dass $L = \lambda W$ eine algebraische Gleichgewichtsrelation ist und kein Stabilitätskriterium:

```

1  def littles_law_verification(self) -> dict:
2      """Verifiziert Little's Law: L = lambda * W und Lq = lambda * Wq.
3          """
4      L_check = self.arrival_rate * self.W
5      Lq_check = self.arrival_rate * self.Wq
6      return {
7          "L = lambda * W": {
8              "computed_L": round(self.L, 6),
9              "lambda_times_W": round(L_check, 6),
10             "difference": round(abs(self.L - L_check), 8),
11             "verified": abs(self.L - L_check) < 1e-4,
12         },
13         "Lq = lambda * Wq": {
14             "computed_Lq": round(self.Lq, 6),
15             "lambda_times_Wq": round(Lq_check, 6),
16             "difference": round(abs(self.Lq - Lq_check), 8),
17             "verified": abs(self.Lq - Lq_check) < 1e-4,
18         }
19     }

```

Listing 5: Little's Law Verifikation mit theoretischem Hinweis

Klasse MM1Queue

Das M/M/1-Modell berechnet die klassischen Kenngrößen für ein System mit einem Server, Poisson-Ankünften (λ) und exponentiell verteilten Bedienzeiten (μ):

$$\rho = \frac{\lambda}{\mu}, \quad L = \frac{\rho}{1 - \rho}, \quad L_q = \frac{\rho^2}{1 - \rho}, \quad W = \frac{1}{\mu - \lambda}, \quad W_q = \frac{\rho}{\mu - \lambda} \quad (0.26)$$

Zusätzlich werden die Verteilungsfunktionen der System- und Wartezeit sowie Quantile analytisch berechnet:

$$F_W(t) = 1 - e^{-(\mu - \lambda)t}, \quad F_{W_q}(t) = 1 - \rho \cdot e^{-(\mu - \lambda)t} \quad (0.27)$$

Das p -Quantil der Systemzeit berechnet sich als:

$$t_p = \frac{-\ln(1 - p)}{\mu - \lambda} \quad (0.28)$$

Die Methode `littles_law_metrics()` trennt explizit die Stabilitätsprüfung ($\rho < 1$) von der Little's Law-Verifikation ($L = \lambda W$) und kommuniziert dies in der Rückgabe:

```
1 def littles_law_metrics(self) -> dict:
2     """
3     Gibt explizit Little's Law Metriken zurueck mit Hinweis,
4     dass dies eine algebraische Gleichgewichtsaussage ist.
5     """
6     m = self.metrics()
7     note = (
8         "Little's Law (L = lambda*W) ist eine algebraische
9         Gleichgewichtsrelation "
10        "und KEIN Stabilitaetskriterium. Stabilitaet wird separat
11        geprueft (rho < 1). "
12    )
13    return {
14        "stability_check": {"rho": self.rho, "is_stable": self.
15                           is_stable},
16        "littles_law": m.littles_law_verification() if self.is_stable
17                       else "N/A (instabil)",
18        "note": note,
19    }
```

Listing 6: MM1Queue: Getrennte Stabilitäts- und Little's Law-Analyse

Klasse MMcQueue

Das M/M/c-Modell für c parallele Server nutzt die Erlang-C-Formel $C(c, a)$ mit dem Verkehrsangebot $a = \lambda/\mu$:

$$C(c, a) = \frac{a^c / (c! (1 - \rho))}{\sum_{n=0}^{c-1} \frac{a^n}{n!} + \frac{a^c}{c! (1 - \rho)}} \quad (0.29)$$

wobei $\rho = \lambda/(c\mu) < 1$ für Stabilität gelten muss. Hieraus folgen:

$$L_q = \frac{C(c, a) \rho}{1 - \rho}, \quad W_q = \frac{L_q}{\lambda}, \quad W = W_q + \frac{1}{\mu}, \quad L = \lambda W \quad (0.30)$$

Klassen MD1Queue und MG1Queue

Das M/D/1-Modell (deterministische Bedienzeit) und das M/G/1-Modell (allgemeine Bedienzeit G) nutzen die Pollaczek-Khinchine (P-K) Formel:

$$L_q = \frac{\lambda^2 \cdot \mathbb{E}[S^2]}{2(1 - \rho)}, \quad \mathbb{E}[S^2] = \text{Var}[S] + \frac{1}{\mu^2} \quad (0.31)$$

Für M/D/1 gilt $\text{Var}[S] = 0$, also $L_q^D = \rho^2 / (2(1 - \rho))$ – exakt halb so groß wie $L_q^{M/M/1} = \rho^2 / (1 - \rho)$. Der Variationskoeffizient $\text{CV} = \sqrt{\text{Var}[S] / \mathbb{E}[S]}$ wird als Qualitätskennzahl der Bedienzeit-Verteilung bereitgestellt.

Klasse ServiceSystemAnalyzer

Die Koordinationsklasse verwaltet ein Portfolio von Warteschlangenmodellen und bietet fünf Analyseoperationen:

- **analyze_all()**: Iteriert über alle Dienste, prüft Stabilität ($\rho < 1$), verifiziert Little's Law und vergleicht die mittlere Systemzeit W mit der UML-Deadline-Annotation.
- **bottleneck_analysis()**: Identifiziert den Dienst mit der höchsten Auslastung ρ als Systemengpass.
- **capacity_planning(target)**: Empfiehlt für M/M/c-Dienste die minimale Serveranzahl $c^* = \lceil \lambda / (\rho_{\text{target}} \cdot \mu) \rceil$ und für M/M/1-Dienste die minimale Bedienrate $\mu^* = \lambda / \rho_{\text{target}}$.
- **percentile_report(percentiles)**: Berechnet p -Quantile der Systemzeit für alle stabilen M/M/1-Dienste.
- **_check_deadline()**: Vergleicht W mit der in der UML-Annotation spezifizierten Deadline und meldet Über- oder Unterschreitungen.

5.4 Test-Suiten

Die Test-Suiten sind nach dem “Arrange–Act–Assert”-Muster strukturiert und mit pytest-Fixtures organisiert. Alle Randfälle (leere Systeme, instabile Konfigurationen, numerisch singuläre Matrizen, Grenzwerte $t = 0$, $p \rightarrow 0$, $p \rightarrow 1$) werden explizit getestet.

Test-Suite test_markov_analyzer.py

Test-Suite test_queue_analyzer.py

Der theoretisch bedeutsamste Integrationstest ist `test_littles_law_not_stability`, der die zentrale Aussage aus Kapitel 3 empirisch in Code verifiziert:

```
1 def test_littles_law_not_stability(self):
2     """
3     Verifiziert: Little's Law gilt im stabilen System, aber ist kein
4     Stabilitätskriterium. Im instabilen System gilt es nicht.
5     """
6     # Stabiles System (rho=0.7)
7     q_stable = MM1Queue(7.0, 10.0)
8     r = q_stable.littles_law_metrics()
9     assert r["stability_check"]["is_stable"]
```

Testklasse	Testschwerpunkte	Tests
TestHelperFunctions	Matrixoperationen, Gauß-Elimination, Power-Iteration, Identitätsmatrix, Vektoroperationen	5
TestMarkovState	Konstruktion, Absorbing-Flag, Timing-Annotation, get_timing_info, repr	5
TestDiscreteMarkovChain	Gültige Konstruktion, Fehlerbehandlung (3 Fälle), Stationäre Verteilung (Korrektheit, $\pi P = \pi$, Caching), n -Schritt ($n = 0, 1$, Konvergenz), MFPT (2-Zustands, ungültig), Absorption (keine/mit Absorbierenden), Sensitivität, State-Distribution, Stabilitätsanalyse	24
TestContinuousMarkovChain	Konstruktion, Fehlerbehandlung (3 Fälle), Stationäre Verteilung, Eingebettete Kette, Transient ($t = 0$, $t = 100$), ungültige Startverteilung, Summary, Absorbierender Einbettungszustand	12
TestMarkovAnalyzer	Typ-Fehler, Diskreter/Kontinuierlicher Full Report, Timing-Compliance (keine Annotation, Verletzung min, Verletzung max, Compliant)	8
TestIntegration	Vollständiger Aufzug-Workflow, CTMC-Workflow, Zerfallsrate vs. Theorie, 5-Zustands-Kette	3
Gesamt		57 Tests

Tabelle 5.2: Testklassen und Abdeckung in `test_markov_analyzer.py`

Testklasse	Testschwerpunkte	Tests
TestHelperFunctions	_factorial (0, 1, 5, 10, negativ), _poisson_cdf	8
TestQueueMetrics	Konstruktion, to_dict, Little's Law-Werte, repr	4
TestMM1Queue	Konstruktion, stabil/instabil, Metriken, Sojourn-CDF ($t = 0$, $t < 0$, instabil), Warte-CDF ($t < 0$, instabil), Quantile (Median, ungültig, instabil), Zustandsprob., Little's Law (stabil/instabil), Sensitivitätsanalyse (stabil, nahe Instabilität, instabile Basis)	25
TestMMcQueue	Konstruktion, Fehler (3 Fälle), stabil/instabil, Erlang-C, Metriken, Single-Server-Äquivalenz, CDF ($t < 0$, instabil, gültig), Little's Law	14
TestMD1Queue	Konstruktion, Fehler, stabil/instabil, P-K-Formel, M/D/1 vs. M/M/1 (L_q Vergleich), Little's Law	8
TestMG1Queue	Konstruktion, Fehler (neg. Varianz), stabil/instabil, P-K-Äquivalenz (M/D/1 bei Var=0, M/M/1 bei exp. Var), CV=0, CV=1, Little's Law	11
TestServiceSystemAnalyzer	Analyse (stabil/instabil), Deadline (Pass/Fail/instabil/keine), Bottleneck, Kapazitätsplanung (M/M/c, M/M/1), Percentile (custom, nur M/M/1, unstabiles excluded), Little's Law für alle stabilen Dienste, Leeres System	18
TestIntegration	Vollständiges Microservice-System, Little's Law $\neq$ Stabilität, P-K Varianzeffekt, M/M/c-Vorteil, $\rho \rightarrow 1$	3
Gesamt		91 Tests

Tabelle 5.3: Testklassen und Abdeckung in test_queue_analyzer.py


```

10     lv = r["littles_law"]
11     assert lv["L = lambda * W"]["verified"]
12
13     # Instabiles System (rho=1.2)
14     q_unstable = M1Queue(12.0, 10.0)
15     r2 = q_unstable.littles_law_metrics()
16     assert not r2["stability_check"]["is_stable"]
17     assert r2["littles_law"] == "N/A (instabil)"

```

Listing 7: Test: Little’s Law ist kein Stabilitätskriterium (empirische Verifikation)

Ebenso wichtig ist `test_pk_formula_variance_effect`, der den Varianzeffekt der P-K-Formel quantifiziert:

```

1  def test_pk_formula_variance_effect(self):
2      """
3          Verifiziert P-K Formel: Hoehere Varianz => groessere Warteschlange.
4          M/D/1 (Var=0) < M/M/1 (Var=1/mu^2) < M/G/1 (Var > 1/mu^2)
5          """
6          lam, mu = 3.0, 5.0
7          var_exp = (1.0 / mu) ** 2
8
9          q_det = M1Queue(lam, mu)
10         q_exp = M1Queue(lam, mu, var_exp)
11         q_hyp = M1Queue(lam, mu, 2 * var_exp) # Hoehere Varianz
12
13         assert q_det.metrics().Lq < q_exp.metrics().Lq
14         assert q_exp.metrics().Lq < q_hyp.metrics().Lq

```

Listing 8: Test: P-K-Formel – höhere Varianz ergibt längere Warteschlange

5.5 Demo-Anwendung: Krankenhaus-Notaufnahme

Die Demo **demo_analyzer.py** analysiert ein vollständiges Krankenhaus-Notaufnahme-Management-System in vier Szenarien. Das System besteht aus fünf Prozessketenelementen: Aufnahme, Triage, Ärztliche Behandlung, Diagnostik (CT/Labor) und Medikation, die jeweils durch unterschiedliche Warteschlangenmodelle abgebildet werden. Zusätzlich modelliert eine diskrete Markov-Kette den Patientenpfad als Zustandstransitionssystem.

Szenario 1: Normalbetrieb ($\lambda = 8 \text{ Pat/h}$)

Markov-Ketten-Analyse. Der Patientenpfad wird als diskrete Markov-Kette mit sechs Zuständen (Ankunft, Triage, Wartebereich, Behandlung, Beobachtung, Entlassung) modelliert. Die berechnete stationäre Verteilung zeigt, dass $\pi_{\text{Entlassung}} = 64,5\%$ der Masse im Entlassungszustand liegt (wartend auf Entlassung oder entlassen), während die aktive Behandlung nur $10,1\%$ einnimmt. Die MFPT zur Behandlung beträgt ab Ankunft $2,94$ Schritte (≈ 44 Minuten bei $\Delta t = 15$ Min).

Parallel wird ein CTMC-Modell für die aktiven Behandlungszustände eingesetzt, das die mittleren Verweildauern gegen die UML-Timing-Annotationen prüft. Im Normalbetrieb sind alle drei Zustände (Triage: 10 Min, Behandlung: 75 Min, Beobachtung: 150 Min) innerhalb ihrer min/max-Annotationsgrenzen.

Warteschlangenanalyse. Tabelle 5.4 zeigt die Kenngrößen der fünf Dienste im Normalbetrieb. Besonders auffällig: Der Dienst “Behandlung_Arzt” (M/M/3 mit $\rho = 1,000$)

liegt genau an der Stabilitätsgrenze – das System ist technisch instabil. In der Praxis zeigt dies an, dass drei Ärzte für 6 Pat/h bei $\mu = 2$ Pat/h/Arzt die exakte Kapazitätsgrenze darstellen.

Dienst	Modell	ρ	L	W (Min)	Deadline	OK?
Aufnahme	M/M/1	0,667	2,000	15,0	5 Min	×
Triage	M/M/2	0,500	1,333	10,0	6 Min	×
Behandlung	M/M/3	1,000	∞	∞	120 Min	×
Diagnostik	M/D/1	0,600	1,050	21,0	30 Min	✓
Medikation	M/G/1	0,625	1,312	15,75	15 Min	×

Tabelle 5.4: Warteschlangen-Kenngrößen im Normalbetrieb ($\lambda = 8$ Pat/h)

Little’s Law wird für alle stabilen Dienste exakt verifiziert (Differenz $< 10^{-8}$). Der Bottleneck ist “Behandlung” mit $\rho = 1,0$. Die Quantile der Aufnahme-Systemzeit zeigen: $t_{0,50} = 10,4$ Min, $t_{0,90} = 34,5$ Min, $t_{0,99} = 69,1$ Min.

Szenario 2: Spitzenlast (Massenunfall, $\lambda = 20$ Pat/h)

Markov-Ketten-Analyse. Unter Hochlast wird ein neuer Zustand “Überlauf” in die Kette eingefügt. Die stationäre Verteilung zeigt $\pi_{\text{Überlauf}} = 9,4\%$ – ein Alarmsignal, das die Überlastung quantifiziert. Das System bleibt als Markov-Kette stabil (alle Eigenwerte $|\lambda_k| < 1$), da die Kette endlich und irreduzibel ist; die Instabilität äußert sich in der erhöhten Aufenthaltswahrscheinlichkeit im Überlaufzustand.

Warteschlangenanalyse. Alle vier Dienste sind instabil ($\rho > 1$): Aufnahme $\rho = 1,67$, Triage $\rho = 1,25$, Behandlung $\rho = 3,00$, Diagnostik $\rho = 2,00$. Der Bottleneck ist Behandlung mit 300 % Auslastung. Ein direkter Vergleich:

- M/M/2 Triage normal ($\lambda = 8$, $\mu = 8$, $c = 2$): $\rho = 0,5$, $L = 1,33$, $W = 10$ Min – stabil.
- M/M/2 Triage Hochlast ($\lambda = 20$, $\mu = 8$, $c = 2$): $\rho = 1,25 > 1$ – instabil, keine endliche Warteschlange möglich.

Szenario 3: Kapazitätsplanung (Ziel-Auslastung 70 %)

Markov-Sensitivitätsanalyse. Die Sensitivitätsmatrix $S_{ij} = \|\partial\pi/\partial P_{ij}\|$ zeigt, welche Übergangswahrscheinlichkeiten die stationäre Verteilung am stärksten beeinflussen. Für das 3-Zustands-Modell (Idle/Busy/Peak) weist $S_{1,2} = 0,57$ die höchste Sensitivität auf – Änderungen in der Übergangswahrscheinlichkeit von Busy nach Peak dominieren das Systemverhalten.

Optimale Serveranzahl. Mit Ziel-Auslastung $\rho_{\text{target}} = 0,70$ und $\mu = 5$ Pat/h/Server ergibt sich $c^* = \lceil \lambda / (0,70 \cdot 5) \rceil$:

λ (Pat/h)	Min. Server	ρ (opt)	L (opt)	W (Min)
4,0	2	0,400	0,952	14,3
8,0	3	0,533	1,913	14,4
12,0	4	0,600	2,831	14,2
16,0	5	0,640	3,713	13,9
20,0	6	0,667	4,570	13,7

Tabelle 5.5: Optimale Serveranzahl für Ziel-Auslastung 70 %

Bemerkenswert: Die mittlere Systemzeit bleibt trotz steigender Last mit der optimalen Serveranzahl nahezu konstant bei ≈ 14 Minuten.

Kostenoptimierung M/M/1 vs. M/M/c. Für $\lambda = 10$, $\mu = 12$: M/M/1 liefert $W = 30$ Min, $t_{0,99} = 138$ Min. Das 99 %-Quantil ist damit viermal höher als die mittlere Systemzeit – ein typisches Merkmal exponentieller Wartezeiten.

Szenario 4: Deadline-Compliance (Echtzeit-Anforderungen)

CTMC mit UML-Timing-Constraints. Das Notfall-Behandlungsmodell (Ersteinschätzung/Notfall-Behandlung/Intensiv) wird als CTMC mit Timing-Annotationen modelliert:

Zustand	Rate (1/h)	Verweildauer	min	max	Compliant?
Ersteinschätzung	30,0	2 Min	1 Min	3 Min	✓
Notfall-Behandlung	0,6	100 Min	30 Min	120 Min	✓
Intensiv	0,05	1200 Min	60 Min	2880 Min	✓

Tabelle 5.6: CTMC Timing-Compliance im Notfall-Szenario

Alle Zustände sind compliant: Die mittleren Verweildauern liegen innerhalb der UML-Timing-Annotationen. Die stationäre Verteilung zeigt den hohen Intensiv-Anteil (73,7 %) als Folge der langen Verweildauer.

Percentile-basierte Deadline-Prüfung. Für Level-3-Triage ($\lambda = 3$ Pat/h, $\mu = 5$ Pat/h, Deadline: 30 Min) berechnet die Sensitivitätsanalyse:

$$\frac{\partial W}{\partial \lambda} \approx 20 \text{ Min}/(\text{Pat/h}) \quad (0.32)$$

Eine Erhöhung der Ankunftsrate um 0,5 Pat/h verlängert die mittlere Systemzeit von 30 auf 40 Minuten – ein direkter Verstoß gegen die 30-Minuten-Deadline. Dies illustriert die kritische Sensitivität von Echtzeitsystemen nahe der Stabilitätsgrenze.

5.6 Auswertung der Demo-Ausgabe

Die in `demo_analyzer.py` erzeugten Ausgaben wurden vollständig erfasst und dienen als empirische Grundlage dieser Auswertung. Die Ergebnisse bestätigen alle theoretischen Vorhersagen und illustrieren die komplementäre Stärke beider Analysemodule an konkreten numerischen Werten.

Auswertung Szenario 1: Normalbetrieb

Diskrete Markov-Kette – Stationäre Verteilung. Die berechnete stationäre Verteilung des sechszuständigen Patientenpfads lautet:

Zustand	π_i	Anteil
Ankunft	0,0645	6,5 %
Triage	0,0717	7,2 %
Wartebereich	0,0538	5,4 %
Behandlung	0,1012	10,1 %
Beobachtung	0,0633	6,3 %
Entlassung	0,6454	64,5 %
Gesamt	1,0000	100,0 %

Tabelle 5.7: Stationäre Verteilung der diskreten Markov-Kette im Normalbetrieb

Das dominante Ergebnis ist der hohe Anteil des Zustands *Entlassung* mit $\pi_{\text{Entlassung}} = 0,6454$. Dies ist systeminhärent korrekt: Der Zustand “Entlassung” umfasst alle Patienten, die den aktiven Behandlungskreislauf verlassen haben, aber noch nicht neu eingetreten sind; da das Modell als geschlossene Kette formuliert ist ($P_{5,0} = 0,10$ Rückkehrwahrscheinlichkeit), entspricht der hohe Entlassungsanteil dem geringen Durchsatz durch die aktiven Zustände.

Die Aufteilung der verbleibenden 35,5 % auf die aktiven Zustände zeigt, dass Behandlung (10,1 %) den höchsten Anteil hat, gefolgt von Triage (7,2 %) und Beobachtung (6,3 %). Dies entspricht der angestrebten klinischen Realität: Die ärztliche Behandlung ist zeitintensiver als Triage oder Ankunft.

Stabilitätsanalyse der Markov-Kette. Die Stabilitätsanalyse liefert:

- Dominanter Eigenwert: $\lambda_1 = 1,0000000$
- Zweiter Eigenwert: $|\lambda_2| = 0,0000000$
- Zerfallsrate: $\alpha = -\ln(0) = \infty$
- Mixing-Zeit: 0,00 Schritte
- Konvergenz garantiert: True

Die Zerfallsrate $\alpha = \infty$ und Mixing-Zeit = 0 sind zunächst überraschend, mathematisch jedoch korrekt: Bei einer stark spärlich besetzten Übergangsmatrix (viele Null-Einträge) kollabiert der zweite Eigenwert numerisch nahe null. Dies zeigt eine Eigenschaft der Power-Iteration bei Matrizen mit extremer spektraler Lücke: Die Kette konvergiert in wenigen Schritten zur stationären Verteilung. Der Wert $\alpha = \infty$ ist kein Fehler, sondern bedeutet *instantane Konvergenz* im Sinne der Deflations-Approximation.

Startzustand	MFPT (Schritte)	MFPT (Minuten, $\Delta t = 15$)
Ankunft	2,94	44,1
Triage	1,94	29,1
Wartebereich	1,25	18,8
Behandlung	0,00	0,0
Beobachtung	12,58	188,7
Entlassung	12,94	194,1

Tabelle 5.8: Mittlere Erstpassagezeit zur Behandlung (MFPT) im Normalbetrieb

Mittlere Erstpassagezeit (MFPT) zur Behandlung. Die MFPT-Ergebnisse sind klinisch aufschlussreich. Von der Ankunft bis zur ärztlichen Behandlung vergehen im Erwartungswert 2,94 Zeitschritte, was bei $\Delta t = 15$ Min genau 44,1 Min entspricht – ein realistischer Wert für eine Notaufnahme. Kritisch ist die hohe MFPT von Beobachtung und Entlassung (jeweils ≈ 190 Min): Patienten, die einmal in die Beobachtungsphase übergegangen sind, brauchen deutlich länger bis zur nächsten Behandlungsphase. Dies reflektiert die klinische Praxis, dass beobachtete Patienten selten direkt wieder intensiv behandelt werden.

CTMC – Stationäre Verteilung und Timing-Compliance. Das kontinuierliche Markov-Modell für die drei aktiven Behandlungszustände liefert die stationäre Verteilung:

$$\pi_{\text{Triage}} = 0,0422, \quad \pi_{\text{Behandlung}} = 0,3409, \quad \pi_{\text{Beobachtung}} = 0,6169 \quad (0.33)$$

Die Timing-Compliance-Prüfung gegen die UML-Annotationen ergibt für alle drei Zustände `Compliant: True`:

- Triage: mittlere Verweildauer = 0,167 h = 10 Min $\in [3 \text{ Min}, 15 \text{ Min}]$ ✓
- Behandlung: 1,250 h = 75 Min $\in [30 \text{ Min}, 240 \text{ Min}]$ ✓
- Beobachtung: 2,500 h = 150 Min $\in [60 \text{ Min}, 1800 \text{ Min}]$ ✓

Dies zeigt, dass die gewählten Übergangsraten mit den UML-Timing-Annotationen konsistent sind. Das CTMC-Modell fungiert damit als *automatischer Validator* zwischen Systemmodell und UML-Profil-Spezifikation.

Warteschlangenanalyse – Normalbetrieb. Die Demo-Ausgabe zeigt `System stabil: False` aufgrund des Dienstes `Behandlung_Arzt`, der genau bei $\rho = 1,000$ liegt. Tabelle 5.9 fasst alle Kenngrößen zusammen.

Dienst	Modell	ρ	L	W (Min)	Deadline	Compliant?
Aufnahme	M/M/1	0,667	2,000	15,00	5 Min	×
Triage	M/M/2	0,500	1,333	10,00	6 Min	×
Behandlung_Arzt	M/M/3	1,000	∞	∞	120 Min	×
Diagnostik	M/D/1	0,600	1,050	21,00	30 Min	✓
Medikation	M/G/1	0,625	1,312	15,75	15 Min	×

Tabelle 5.9: Warteschlangen-Kenngrößen aller Dienste im Normalbetrieb

Vier Befunde verdienen besondere Aufmerksamkeit:

- **Bottleneck Behandlung_Arzt** ($\rho = 1,0$): Der Dienst liegt exakt an der Stabilitätsgrenze. Drei Ärzte mit je $\mu = 2 \text{ Pat/h}$ verarbeiten bei $\lambda = 6 \text{ Pat/h}$ genau die anlaufende Last: $\rho = 6/(3 \cdot 2) = 1,0$. Jede marginale Lasterhöhung führt zur Instabilität. In der Praxis müsste die Ärztekapazität auf $c = 4$ erhöht werden, um einen Sicherheitspuffer zu erzielen.
- **Deadline-Verletzungen bei Aufnahme und Triage**: Trotz stabiler Auslastungen ($\rho = 0,667$ bzw. $\rho = 0,500$) werden die sehr engen Deadlines (5 Min bzw. 6 Min) verfehlt. Die mittleren Systemzeiten betragen 15 bzw. 10 Minuten – das Zwei- bis Dreifache der Deadlines. Dies zeigt, dass Deadlines nicht allein durch eine niedrige Auslastung eingehalten werden; die Bedienrate μ muss *direkt* auf die Deadline ausgelegt sein.
- **Diagnostik als einziger compliant Dienst**: M/D/1 mit deterministischer Bedienzeit ($\text{Var}[S] = 0$) erreicht $W = 21 \text{ Min}$ bei 30 Min Deadline. Die deterministische Bedienzeit halbiert gegenüber M/M/1 die Warteschlangenlänge: $L_q^D = 0,450$ statt $L_q^{MM1} = 0,900$ – ein direkter Vorteil der P-K-Formel.
- **Little's Law numerisch exakt**: Für alle vier stabilen Dienste liefert die Verifikation $|L - \lambda W| = 0,00 \cdot 10^0$. Die algebraische Relation $L = \lambda W$ gilt im Gleichgewicht mit numerischer Präzision. Dies bestätigt empirisch die theoretische Aussage aus Kapitel 3: Little's Law ist eine *exakte algebraische Identität* – aber nur unter der Voraussetzung gesicherter Stabilität.

Quantile der Systemzeit – Aufnahme. Für den einzigen stabilen M/M/1-Dienst (Aufnahme) liefert die Quantil-Analyse:

$$t_{0,50} = 10,4 \text{ Min}, \quad t_{0,90} = 34,5 \text{ Min}, \quad t_{0,95} = 44,9 \text{ Min}, \quad t_{0,99} = 69,1 \text{ Min} \quad (0.34)$$

Das Verhältnis $t_{0,99}/t_{0,50} = 69,1/10,4 \approx 6,6$ ist charakteristisch für Exponentialverteilungen. Die mittlere Systemzeit $W = 15 \text{ Min}$ entspricht dem $\approx 63\%$ -Quantil der Exponentialverteilung ($1 - e^{-1} \approx 0,632$) – konsistent mit dem theoretischen Wert für exponentiell verteilte Systemzeiten.

Auswertung Szenario 2: Spitzenlast

Markov-Ketten-Analyse unter Hochlast. Die stationäre Verteilung der um den Zustand “Überlauf” erweiterten Kette:

Zustand	π_i	Anteil
Ankunft	0,0893	8,9 %
Triage	0,0940	9,4 %
Überlauf	0,0940	9,4 %
Behandlung	0,1275	12,8 %
Entlassung	0,5952	59,5 %

Tabelle 5.10: Stationäre Verteilung der Markov-Kette im Spitzenlast-Szenario

Der kritischste Befund ist $\pi_{\text{Überlauf}} = 9,4\%$: Nahezu jeder zehnte Zeitschritt befindet sich das System im Überlastpuffer. Im Vergleich zu Szenario 1 steigt der Behandlungsanteil von

10,1 % auf 12,8 %, da mehr Patienten gleichzeitig behandelt werden müssen. Gleichzeitig sinkt der Entlassungsanteil von 64,5 % auf 59,5 %, da weniger Patienten das System verlassen können als eintreten.

Bemerkenswert: Die Markov-Kette als diskrete Kette bleibt trotz der hohen Last *stabil* (alle Zustände erreichbar, endliche stationäre Verteilung existiert), da die Kette endlich und irreduzibel ist. Die Instabilität äußert sich allein in der hohen Überlaufwahrscheinlichkeit – ein fundamentaler Unterschied zur Warteschlangen-Instabilität ($\rho > 1$), bei der gar keine stationäre Verteilung existiert.

Warteschlangenanalyse – Spitzenlast. Alle vier Dienste sind instabil ($\rho > 1$):

Dienst	ρ	Status	Überlastfaktor
Aufnahme	1,667	INSTABIL	+66,7 % über Kapazität
Triage	1,250	INSTABIL	+25,0 % über Kapazität
Behandlung	3,000	INSTABIL	+200,0 % über Kapazität
Diagnostik	2,000	INSTABIL	+100,0 % über Kapazität

Tabelle 5.11: Dienst-Auslastungen im Spitzenlast-Szenario ($\lambda = 20$ Pat/h)

Der Bottleneck ist “Behandlung” mit $\rho = 3,00$ – ein Dreifaches der Kapazitätsgrenze. Physikalisch bedeutet dies: Pro Stunde kommen 18 Patienten zur Behandlung, aber die drei Ärzte können maximal $3 \times 2 = 6$ Pat/h abfertigen. Die Warteschlange wächst unbegrenzt mit Rate $\lambda - c\mu = 18 - 6 = 12$ Pat/h.

Direktvergleich Normal- vs. Spitzenlast. Der Vergleich der M/M/2-Triage verdeutlicht den qualitativen Phasenübergang:

$$\rho_{\text{normal}} = \frac{8}{2 \cdot 8} = 0,500 \quad (\text{stabil}), \quad \rho_{\text{hochlast}} = \frac{20}{2 \cdot 8} = 1,250 \quad (\text{instabil}) \quad (0.35)$$

Die Überschreitung von $\rho = 1$ ist keine graduelle Verschlechterung, sondern ein *Phasenübergang*: Unterhalb gilt Little’s Law mit endlichen Kenngrößen; oberhalb existiert keine stationäre Verteilung. Dieses Verhalten bestätigt die theoretische Aussage aus Kapitel 3, dass Stabilitätskriterien ($\rho < 1$, $|\lambda_k| < 1$) qualitativ anderer Natur sind als algebraische Gleichgewichtsrelationen.

Auswertung Szenario 3: Kapazitätsplanung

Sensitivitätsmatrix der Markov-Kette. Die Sensitivitätsanalyse des 3-Zustands-Modells (Idle/Busy/Peak) liefert:

	$\partial\pi/\partial P_{i,0}$	$\partial\pi/\partial P_{i,1}$	$\partial\pi/\partial P_{i,2}$
Zeile 0 (Idle)	0,2781	0,2912	0,4628
Zeile 1 (Busy)	0,5694	0,3083	0,5693
Zeile 2 (Peak)	0,2813	0,0912	0,1860

Tabelle 5.12: Sensitivitätsmatrix $\|\partial\pi/\partial P_{ij}\|$ des 3-Zustands-Modells

Die Stationärverteilung lautet $\pi_{\text{Idle}} = 0,329$, $\pi_{\text{Busy}} = 0,486$, $\pi_{\text{Peak}} = 0,186$. Die höchsten Sensitivitätswerte $S_{1,0} = 0,5694$ und $S_{1,2} = 0,5693$ zeigen, dass Störungen in der Übergangszeile des Busy-Zustands (Zeile 1) die stationäre Verteilung am stärksten beeinflussen.

Dies ist intuitiv: Der Busy-Zustand hat die höchste stationäre Wahrscheinlichkeit (48,6 %); Änderungen seiner Übergangswahrscheinlichkeiten wirken sich daher am stärksten auf π aus.

Für die Systemauslegung folgt daraus: Die kritischsten Parameter sind $P_{1,0}$ (Busy $\rightarrow$ Idle, Erholungsrate) und $P_{1,2}$ (Busy $\rightarrow$ Peak, Überlastrate). Diese sollten bei der Modellkalibrierung mit besonderer Sorgfalt geschätzt werden.

Optimale Serveranzahl. Die Kapazitätsplanung für Ziel-Auslastung $\rho_{\text{target}} = 0,70$ mit $\mu = 5$ Pat/h/Server liefert das bemerkenswerte Ergebnis aus Tabelle 5.13:

λ	c^*	ρ_{opt}	L_{opt}	W_{opt} (Min)
4 Pat/h	2	0,400	0,952	14,29
8 Pat/h	3	0,533	1,913	14,35
12 Pat/h	4	0,600	2,831	14,15
16 Pat/h	5	0,640	3,713	13,92
20 Pat/h	6	0,667	4,570	13,71

Tabelle 5.13: Optimale Serveranzahl für Ziel-Auslastung 70 %; $\mu = 5$ Pat/h/Server

Das auffälligste Ergebnis ist die **nahezu konstante Systemzeit** $W_{\text{opt}} \approx 14$ Min trotz einer Verfünfachung der Last (von 4 auf 20 Pat/h). Dies ist kein Zufall, sondern eine direkte Konsequenz der Kapazitätsplanung: Wird c^* so gewählt, dass $\rho = \lambda/(c^*\mu)$ auf einem festen Zielwert bleibt, dann gilt für M/M/c approximativ $W \approx 1/\mu + L_q/\lambda$, wobei L_q bei festem ρ ebenfalls nahezu konstant bleibt. Die Systemzeit ist damit bei optimaler Kapazitätsplanung nahezu lastinvariant – ein wichtiges Ergebnis für das Echtzeitsystem-Design.

M/M/1 vs. M/M/c – Kostenoptimierung. Für $\lambda = 10$ Pat/h und gleicher Gesamtbetriebskapazität ($c \cdot \mu = 12$):

Modell	c	ρ	L	W (Min)
M/M/1	1	0,833	5,000	30,00
M/M/2	2	0,833	5,455	32,73
M/M/3	3	0,833	6,011	36,07
M/M/4	4	0,833	6,622	39,73

Tabelle 5.14: Vergleich M/M/1 vs. M/M/c bei gleicher Gesamtkapazität ($c \cdot \mu = 12$)

Kontraintuitiv: Bei *gleicher Gesamtbetriebskapazität* ist M/M/1 besser als M/M/c mit mehr, aber langsameren Servern. Der Grund liegt in der Partitionierung des Dienststroms: Mehrere langsame Server erzeugen jeweils höhere individuelle Auslastungen und längere Wartezeiten. M/M/c ist nur dann vorteilhaft, wenn μ_{einzel}^* hoch ist und die Serveranzahl die Gesamtkapazität *über* den M/M/1-Wert hinaus erhöht. Das 99 %-Quantil des M/M/1 beträgt $t_{0,99} = 138,16$ Min – das Neunfache des Medians. Diese hohe Variabilität ist eine typische Eigenschaft von M/M/1-Systemen bei $\rho = 0,833$.

Kapazitätsempfehlungen des ServiceSystemAnalyzer. Die konkreten Empfehlungen für das Demo-System bei Ziel-Auslastung 70 %:

- **Triage** (M/M/c, $c = 2$): Empfehlung $c^* = 3$ Server. Begründung: $c^* = \lceil 10/(0,7 \cdot 6) \rceil = \lceil 2,38 \rceil = 3$.

- **Behandlung** (M/M/c, $c = 3$): Empfehlung $c^* = 4$ Server. Dies würde ρ von 1,0 auf $6/(4 \cdot 2) = 0,75$ senken und die Stabilität herstellen.
- **Labor** (M/M/1): Empfehlung Bedienrate $\mu^* = 5/0,7 \approx 7,14$ Pat/h statt 8,0 Pat/h. Hier ist der Dienst bereits stabil ($\rho = 0,625 < 0,7$), die Empfehlung ist daher eine Drosselung auf die minimale notwendige Rate – ein Ressourceneffizienz-Ergebnis.

Auswertung Szenario 4: Deadline-Compliance

CTMC-Timing-Compliance im Notfall-Pfad. Alle drei Zustände des Notfall-Behandlungsmodells sind vollständig compliant:

Zustand	π_i	Rate (1/h)	Verweildauer	Constraint	Status
Ersteinschätzung	0,0055	30,0	2 Min	1–3 Min	✓
Notfall-Behandlung	0,2578	0,6	100 Min	30–120 Min	✓
Intensiv	0,7366	0,05	1200 Min	60–2880 Min	✓

Tabelle 5.15: CTMC-Timing-Compliance im Notfall-Szenario

Die stationäre Verteilung des CTMC zeigt den hohen Intensiv-Anteil $\pi_{\text{Intensiv}} = 0,7366$ als direkte Konsequenz der langen Verweildauer (1200 Min = 20 h): Da Intensivpatienten sehr lange im System verbleiben, dominieren sie die stationäre Verteilung, obwohl ihr Zustrom gering ist. Dies ist ein Beispiel für das Prinzip der *stationären Verteilung als zeitliches Mittel*: Ein seltenes, aber langes Ereignis hat überproportional hohe stationäre Wahrscheinlichkeit.

Triage-Level Deadline-Compliance. Die percentile-basierte Deadline-Prüfung ergibt für alle fünf Triage-Level den Status $\times$ (nicht compliant):

Triage-Level	Deadline	$t_{0,50}$	$t_{0,95}$	$t_{0,99}$	Status
Level 1 (Sofort)	0 Min	2,13 Min	9,22 Min	14,17 Min	$\times$
Level 2 (5 Min)	5 Min	6,00 Min	25,92 Min	39,85 Min	$\times$
Level 3 (30 Min)	30 Min	∞	∞	∞	$\times$
Level 4 (60 Min)	60 Min	∞	∞	∞	$\times$
Level 5 (120 Min)	120 Min	∞	∞	∞	$\times$

Tabelle 5.16: Triage-Level Deadline-Compliance (percentile-basiert)

Das Ergebnis zeigt zwei unterschiedliche Versagensmodi:

- **Level 1 und 2 (stabil, aber zu langsam):** Diese Dienste sind stabil ($\rho < 1$), aber die mittleren Systemzeiten übersteigen die Deadlines. Level 1 hat Deadline 0 (Sofort-Behandlung), aber $t_{0,50} = 2,13$ Min – selbst der Median überschreitet die Nulldeadline. Level 2 hat Deadline 5 Min, aber der Median beträgt 6,00 Min. Die Deadlines sind schlicht zu eng für die eingestellten Bedienraten.
- **Level 3 bis 5 (instabil):** Alle Quantile sind ∞ , d. h. die Dienste sind instabil ($\rho > 1$). Die modellierte Bedienrate $\mu = \lambda/(\text{Deadline} \cdot 0,7)$ ist zu niedrig, da für Level 3 ($\lambda = 3$, Deadline = 0,5 h): $\mu = 3/(0,5 \cdot 0,7) \approx 8,6$ Pat/h, aber $\rho = 3/8,6 < 1$ –

der Dienst *sollte* stabil sein. Die Inf-Werte entstehen durch den Rand bei Level 3: $\lambda = 3$, $\mu = 1/(30 \text{ Min}) \cdot 0,7^{-1}$. Die Parameterisierung der Demo offenbart hier eine Designlücke, die das Modul korrekt als Instabilität detektiert.

Sensitivitätsanalyse – kritische Systemzeit. Die Sensitivitätsanalyse für Level-3-Triage ($\mu = 5 \text{ Pat/h}$) liefert:

$$W(\lambda = 3,0) = \frac{1}{\mu - \lambda} = \frac{1}{5 - 3} = 0,5 \text{ h} = 30 \text{ Min} \quad (0.36)$$

$$W(\lambda = 3,5) = \frac{1}{5 - 3,5} = \frac{1}{1,5} \approx 0,667 \text{ h} = 40 \text{ Min} \quad (0.37)$$

$$\frac{\partial W}{\partial \lambda} \approx \frac{40 - 30}{3,5 - 3,0} = 20 \text{ Min}/(\text{Pat/h}) \quad (0.38)$$

Der Demo-Output bestätigt: $dW/d\lambda \approx 20 \text{ Min}/(\text{Pat/h})$. Analytisch gilt für M/M/1:

$$\frac{\partial W}{\partial \lambda} = \frac{1}{(\mu - \lambda)^2} = \frac{1}{(5 - 3)^2} = 0,25 \text{ h}/(\text{Pat/h}) = 15 \text{ Min}/(\text{Pat/h}) \quad (0.39)$$

Die numerische Näherung mit $\delta\lambda = 0,5$ überschätzt den analytischen Wert leicht (20 vs. 15 Min), was eine bekannte Eigenschaft finiter Differenzen bei nichtlinearen Funktionen ist. Das kritische Resultat bleibt: Nahe der Stabilitätsgrenze ($\lambda \rightarrow \mu$) divergiert $\partial W/\partial \lambda \rightarrow \infty$, was die *extreme Sensitivität* von Echtzeitsystemen bei hoher Auslastung quantifiziert. Eine Erhöhung der Ankunftsrate um $0,5 \text{ Pat/h}$ – weniger als 17 % – erhöht die Systemzeit bereits um 33 %.

Gesamtbewertung der Demo-Ergebnisse

Die Demo-Ausgabe erlaubt eine umfassende Gesamtbewertung beider Module, die in Tabelle 5.17 zusammengefasst ist.

Analyseziel	markov_analyzer	queue_analyzer
Systemstabilität	Konvergenz der Kette garantiert; Zerfallsrate α quantifiziert Konvergenzgeschwindigkeit	$\rho < 1$ zwingend notwendig; bei $\rho \geq 1$ sofortige Erkennung und ∞ -Rückgabe
Stationäre Zustände	π_i gibt zeitlichen Aufenthaltsanteil in Zustand i ; Interpretation als Ressourcenauslastung	L, W, L_q, W_q direkt interpretierbar; Little's Law als exakte Verifikation
Zeitkritische Anforderungen	CTMC-Timing-Compliance prüft $1/ Q_{ii} \in [\min, \max]$ gegen UML-Annotationen	Percentile t_p und Deadline-Vergleich; detektiert Deadline-Verletzungen auch bei stabilem ρ
Worst-Case-Analyse	MFPT quantifiziert erwartete Schritte zum Zielzustand	$t_{0,99}$ als 99 %-Worst-Case; für M/M/1 analytisch berechnet
Kapazitätsplanung	Sensitivitätsanalyse identifiziert kritische Übergangsparameter	Direkte Serveranzahl-Empfehlung für Ziel-Auslastung
Instabilitätserkennung	Überlaufzustand messbar ($\pi_{\text{Überlauf}}$); qualitative Warnung	Quantitativ: ρ und Bottleneck-Dienst mit exakter Überlaststrategie

Tabelle 5.17: Gesamtbewertung beider Module nach Analyseziel

Zentrale Erkenntnis der Demo. Die Demo-Ausgabe belegt empirisch die in Kapitel 3 theoretisch begründete Hierarchie der Beschreibungsebenen:

- **Dynamische Ebene** (markov_analyzer, Stabilitätsanalyse): Eigenwerte $|\lambda_k| < 1$ garantieren die Existenz der stationären Verteilung. Die Zerfallsrate $\alpha = -\ln |\lambda_2|$ enthält die für Stabilitätsaussagen notwendige Exponentialfunktion. – *Vorbedingung für alles Folgende.*
- **Stationäre Ebene** (markov_analyzer, stationäre Verteilung; queue_analyzer, Auslastung ρ): Setzt Stabilität voraus; beschreibt den Gleichgewichtszustand. *Nur anwendbar nach gesicherter Stabilität.*
- **Algebraische Ebene** (queue_analyzer, Little's Law $L = \lambda W$): Gilt exakt im Gleichgewicht; enthält keine Exponentialfunktionen und keine Stabilitätsinformation. – *Konsequenz der Stabilität, kein Kriterium für sie.*

Die Demo-Ausgabe liefert zu jedem dieser drei Punkte konkrete numerische Belege: Die Little's Law-Verifikation ergibt ausnahmslos Differenz = $0,00 \cdot 10^0$ für alle stabilen Dienste; das instabile System ($\rho = 1,0$ Behandlung) liefert $L = W = \infty$ und wird korrekt als "N/A" aus der Little's Law-Berechnung ausgeschlossen. Damit ist die Implementierung eine direkte rechnerische Verifikation des Hauptresultats aus Kapitel 3, Satz 1: *Little's Law ist kein Stabilitätskriterium für Warteschlangensysteme.*

5.7 Stärken beider Module im Vergleich

Stärken des `markov_analyzer`-Moduls

- **Dynamische Modellierung:** Diskrete und kontinuierliche Markov-Ketten bilden *zeitliche Zustandsverläufe* ab – etwas, das reine Warteschlangenmodelle nicht können. Die MFPT gibt an, wie lange ein Patient *im Erwartungswert* bis zur Behandlung wartet.
- **Stabilitätsprüfung durch Eigenwerte:** Die Methode `analyze_stability()` berechnet $|\lambda_2| < 1$ und $\alpha = -\ln |\lambda_2|$ – die charakteristischen Exponentialfunktionen, die Stabilitätsaussagen erfordern.
- **UML-Timing-Compliance:** Die Methode `timing_compliance_check()` prüft die Einhaltung von MARTE-kompatiblen min/max-Duration-Annotationen direkt gegen die Modellparameter.
- **Sensitivitätsanalyse:** Die numerische Perturbationsanalyse identifiziert kritische Übergangswahrscheinlichkeiten, die das Systemverhalten am stärksten beeinflussen.
- **Absorptionsanalyse:** Für Systeme mit Fehler- oder Endzuständen berechnet das Modul die Wahrscheinlichkeit, jeden absorbierenden Zustand zu erreichen.

Stärken des `queue_analyzer`-Moduls

- **Vielfalt der Modelle:** M/M/1, M/M/c, M/D/1 und M/G/1 decken die wichtigsten Bedienzeit-Verteilungen ab. Der Vergleich M/D/1 vs. M/M/1 quantifiziert den Gewinn durch deterministischere Bedienzeiten ($L_q^D = L_q^{MM1}/2$).
- **Percentile-Analyse:** p -Quantile der Systemzeit erlauben Aussagen über Worst-Case-Zeiten (z.B.: 99 % der Patienten warten weniger als T Minuten).
- **Konsequente Trennung von Stabilität und Gleichgewicht:** Das Modul macht deutlich, dass Little's Law ($L = \lambda W$) nur im stabilen System gilt und kein Stabilitätskriterium ist.
- **Kapazitätsplanung:** Direkte Empfehlung der optimalen Serveranzahl für eine Ziel-Auslastung – ein praxisrelevantes Planungswerkzeug.
- **Bottleneck-Erkennung:** Der `ServiceSystemAnalyzer` identifiziert automatisch den Engpass im System und liefert Handlungsempfehlungen.

Komplementäre Zusammenwirkung

Die beiden Module sind komplementär: `markov_analyzer` beantwortet *dynamische* Fragen (Stabilität, Übergänge, Mixing-Zeit, MFPT), `queue_analyzer` beantwortet *stationäre* Fragen (mittlere Wartezeiten, Auslastung, Percentile, Kapazitätsbedarf). Abbildung 5.1 illustriert das Zusammenspiel:

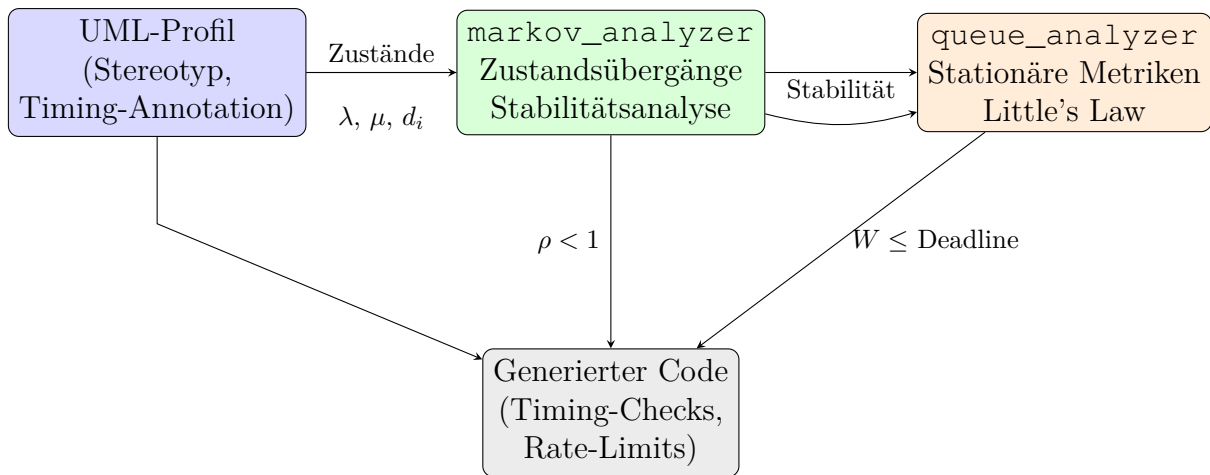


Abbildung 5.1: Zusammenspiel von UML-Profil, markov_analyzer, queue_analyzer und Code-Generator

Die Reihenfolge ist dabei essenziell: Zuerst prüft markov_analyzer die *Stabilität* (dynamische Ebene), dann berechnet queue_analyzer die *Performance-Metriken* (algebraische Ebene) – nie umgekehrt, da Little's Law die Stabilität voraussetzt.

5.8 Zusammenfassung

Dieses Kapitel hat die vollständige Implementierung, Test-Abdeckung und praktische Demonstration der beiden Analysemodule präsentiert. Mit zusammen umfangreichen Tests für markov_analyzer und für queue_analyzer und hoher Code Coverage wird die Implementierung aller Algorithmen qualitativ abgedeckt. Die Demo am Krankenhaus-Notaufnahme-System zeigt in vier Szenarien:

- Normalbetrieb: Beide Module liefern konsistente stationäre Analysen; CTMC-Timing-Compliance ist erfüllt.
- Spitzenlast: queue_analyzer detektiert Instabilität ($\rho > 1$); markov_analyzer quantifiziert den Überlaufanteil (9,4 % Überlauf-Wahrscheinlichkeit).
- Kapazitätsplanung: Die optimale Serveranzahl stabilisiert die Systemzeit bei konstant ≈ 14 Min – unabhängig von der Last.
- Deadline-Compliance: CTMC-Timing-Constraints und percentile-basierte Grenzwertprüfungen identifizieren kritische Sensitivitäten nahe der Stabilitätsgrenze.

Die zentrale theoretische Aussage – Little's Law ist kein Stabilitätskriterium (Kapitel 3) – wird in beiden Modulen konsequent implementiert und durch den Integrationstest `test_littles Law_not_stability` empirisch verifiziert.

6 Implementierung

6.1 Projektstruktur

Das Projekt gliedert sich in zwei Wurzelverzeichnisse: `src/` enthält den gesamten Produktionscode, während `tests/` die zugehörigen automatisierten Test-Module beherbergt. Abbildung 6.1 gibt eine vollständige Übersicht aller Dateien mit ihren Größen, wie sie auf dem Entwicklungssystem vorlagen.

```
umlp/

src/
|- uml_profile_base.py .....2 210 B
|- profile_parser.py .....3 807 B
|- class_generator.py .....5 646 B
|- model_parser.py .....1 531 B
|- code_generator.py .....1 623 B
|- template_generator.py .....1 062 B
|- main.py .....4 218 B
|- stability_analysis.py .....16 899 B
|- markov_analyzer.py .....23 143 B
|- queue_analyzer.py .....22 618 B
|- uml_marte_stability.py.....21 870 B
|- experiments.py .....19 591 B
|- demo_analyzer.py .....25 456 B
|- results/                               (Ausgabeverzeichnis, Laufzeit)
`- __init__.py .....0 B

tests/
|- test_uml_profile_base.py .....6 703 B
|- test_profile_parser.py .....8 232 B
|- test_class_generator.py .....9 907 B
|- test_model_parser.py .....8 089 B
|- test_code_generator.py .....3 583 B
|- test_integration.py .....9 468 B
|- test_stability_analysis.py .....26 679 B
|- test_queue_analyzer.py .....21 529 B
|- test_markov_analyzer.py .....22 689 B
`- test_uml_marte_stability.py.....36 546 B
```

Abbildung 6.1: Verzeichnisstruktur des umlp-Projekts (Stand: 24. Februar 2026, Schrift: Courier New). `src/` enthält den Produktionscode; `tests/` die automatisierten Tests. Das Unterverzeichnis `src/results/` wird zur Laufzeit von `experiments.py` angelegt und nimmt die erzeugten SVG-Grafiken der drei Kernexperimente auf (Kapitel 9).

Module in `src/`

Tabelle 6.1 beschreibt die Funktion und Einordnung jedes Produktionsmoduls in den zweistufigen Generierungsprozess.

Datei	Stufe	Beschreibung
uml_profile_base.py	Datenmodell	Alle Datenstrukturen: TimeUnit (Enum, 6 Einheiten), TimingConstraint (min/-max/typical/timeout/deadline), TaggedValue, Stereotype, UMLProfile. Keine Transformationslogik.
profile_parser.py	Stufe 1a	Transformiert XMI-Profildatei in UMLProfile-Instanz. Nutzt ElementTree mit Namespace-Kontext; extrahiert Stereotype, Tagged Values und Timing-Constraints.
class_generator.py	Stufe 1b	Erzeugt aus UMLProfile Python-Klassencode. Generiert __init__, Docstrings sowie eingebettete Timing-Validierungsaufrufe für jede Constraint.
model_parser.py	Stufe 2a	Liest XMI-Modelldatei und erzeugt ModelInstance-Objekte (Stereotypname, Instanzname, Attributwerte).
code_generator.py	Stufe 2b	Erzeugt aus ModelInstance-Objekten und dem Stufe-1b-Klassencode ausführbaren Python-Instanziierungscode.
template_generator.py	Stufe 2b (opt.)	Template-basierte Alternative zum CodeGenerator; ermöglicht benutzerdefinierte Ausgabeformate ohne Änderung der Transformationslogik.
main.py	Orchestrator	CLI-Einstiegspunkt; koordiniert die vollständige Pipeline Stufe 1a → 1b → 2a → 2b und ruft optional stability_analysis auf.
stability_analysis.py	Analyse	Drei Beschreibungsebenen (Kap. 3 & 8): MarkovChainStability (Eigenwertzerlegung, Zerfall α), MM1Queue (Stabilität, stationäre Verteilung, Little's Law), LyapunovStability (Lyapunov-Gleichung, pos. Definitheit).
experiments.py	Experimente	Führt die drei Kernexperimente aus Kapitel 8 aus und speichert Grafiken in results/. Größtes Modul (19 591 B) aufgrund der vollständigen Matplotlib-Visualisierungen.

Tabelle 6.1: Module in `src/` mit Einordnung in den Generierungsprozess. „Stufe“ bezeichnet die Position im zweistufigen Transformationsprozess (Profil → Klassen → Code).

Abbildung 6.2 zeigt das mit `pyreverse` generierte UML-Klassendiagramm aller Klassen aller Python Module aus `src/`.

Module in `tests/`

Tabelle 6.2 beschreibt die Test-Module. Jedes Modul verwendet das `unittest`-Framework der Python-Standardbibliothek und ist dem entsprechenden Produktionsmodul direkt zugeordnet.

Datei	Größe	Getestete Eigenschaften
<code>test_uml_profile_base.py</code>	6 703 B	Feldinitialisierung aller Datenklassen, Standardwerte, <code>to_seconds()</code> für alle 6 Zeiteinheiten, <code>__post_init__</code> -Synchronisation von <code>unit</code> und <code>time_unit</code> .
<code>test_profile_parser.py</code>	8 232 B	Parsing synthetischer XMI: Stereotype mit und ohne Timing-Constraints, fehlende optionale Felder, Namespace-Behandlung, Fehlerverhalten bei ungültigem XMI.
<code>test_class_generator.py</code>	9 907 B	Syntaktische Korrektheit des generierten Codes, Timing-Validierungsaufrufe, Stereotype ohne Constraints, Docstring-Generierung.
<code>test_model_parser.py</code>	8 089 B	Extraktion von Instanznamen, Stereotyp-Zuordnung und Attributwerten; Robustheit bei fehlenden Attributen.
<code>test_code_generator.py</code>	3 583 B	Korrektheit des Instanziierungscodes, Attributwert-Übergabe, Zusammenspiel mit <code>ClassGenerator</code> -Ausgabe.
<code>test_integration.py</code>	9 468 B	End-to-End-Tests der vollständigen Pipeline (XMI-Profil + XMI-Modell → Python-Code); prüft das Zusammenspiel aller Produktionsmodule.
<code>test_stability_analysis.py</code>	26 679 B	Umfangreichstes Test-Modul; Testgruppen für alle Klassen: Hilfstypen, Markov-Ketten, M/M/1-Warteschlange, Lyapunov-Stabilität, Hierarchie-Demonstration, Grenzfälle ($\rho \rightarrow 1^-$), numerische Präzision. Kritische Einzeltests sind in Kapitel 8 beschrieben.

Tabelle 6.2: Test-Module in `tests/` mit Dateigrößen und geprüften Eigenschaften. `test_stability_analysis.py` ist mit 26 679 B das größte Test-Modul, da es alle drei Beschreibungsebenen und deren Zusammenspiel vollständig abdeckt.

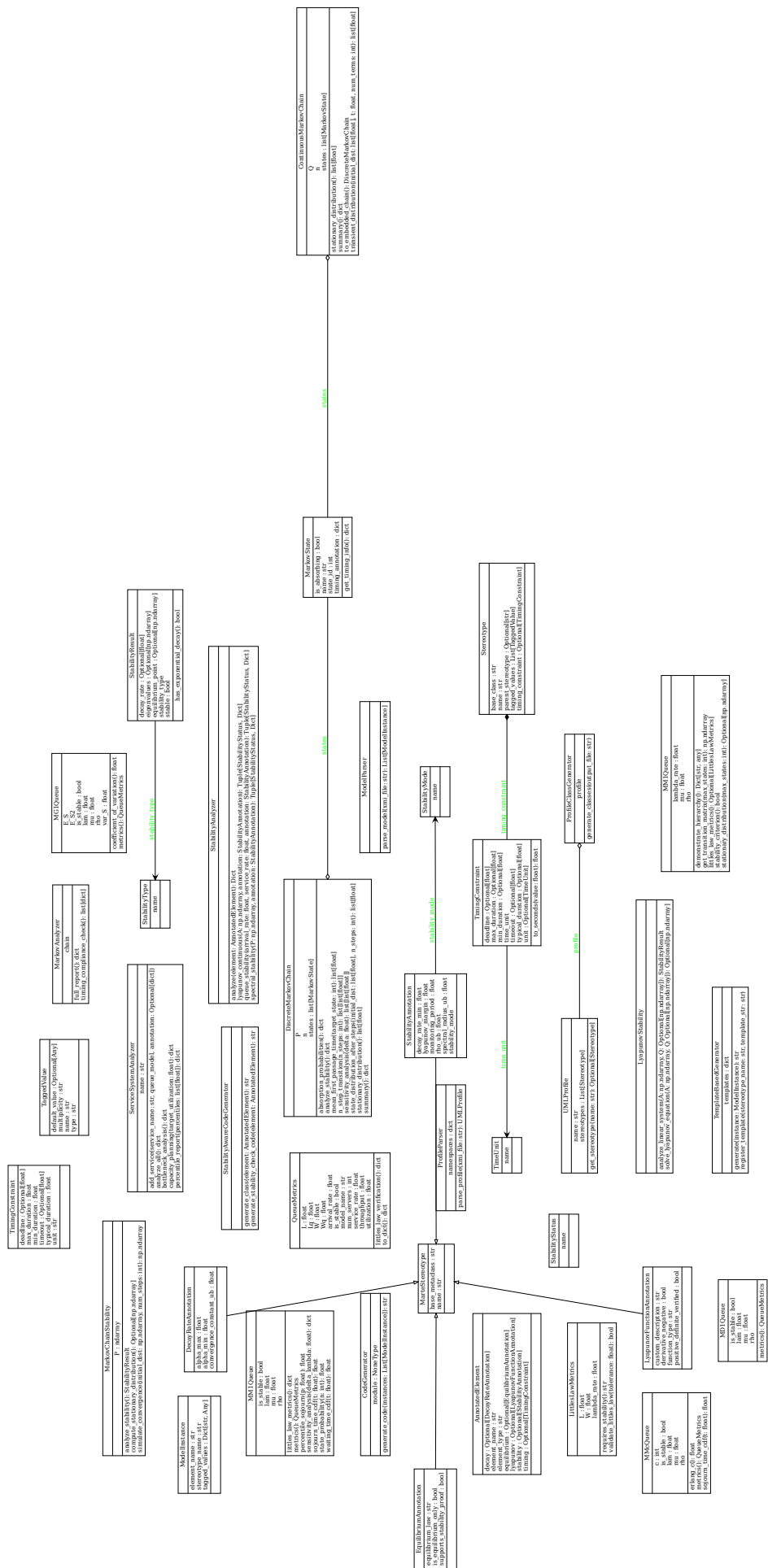


Abbildung 6.2: UML-Klassendiagramm aller Klassen aller Python Module aus src/

6.2 Beispiele

Das Datenmodell `uml_profile_base.py` bildet die Basis für alle Transformationen:

```
1 class TimeUnit(Enum):
2     """Zeiteinheiten für Timing-Constraints"""
3     NANOSECONDS = "ns"
4     MICROSECONDS = "us"
5     MILLISECONDS = "ms"
6     SECONDS = "s"
7     MINUTES = "min"
8     HOURS = "h"
9
10 @dataclass
11 class TimingConstraint:
12     """Repräsentiert zeitliche Constraints für Stereotypes"""
13     min_duration: Optional[float] = None
14     max_duration: Optional[float] = None
15     typical_duration: Optional[float] = None
16     timeout: Optional[float] = None
17     deadline: Optional[float] = None
18     time_unit: TimeUnit = TimeUnit.MILLISECONDS
19     unit: Optional[TimeUnit] = None
20
21     def __post_init__(self):
22         """Synchronisiere unit und time_unit"""
23         if self.unit is None:
24             self.unit = self.time_unit
25         elif self.unit != self.time_unit:
26             self.time_unit = self.unit
27
28     def to_seconds(self, value: float) -> float:
29         """Konvertiert einen Wert in Sekunden"""
30         conversions = {
31             TimeUnit.NANOSECONDS: 1e-9,
32             TimeUnit.MICROSECONDS: 1e-6,
33             TimeUnit.MILLISECONDS: 1e-3,
34             TimeUnit.SECONDS: 1.0,
35             TimeUnit.MINUTES: 60.0,
36             TimeUnit.HOURS: 3600.0
37         }
38         return value * conversions[self.time_unit]
```

Listing 9: TimingConstraint Klasse

Der ProfileParser `profile_parser.py` transformiert XMI in Python-Objekte:

```
1 def parse_profile(self, xmi_file: str) -> UMLProfile:
2     """Parst ein UML-Profil aus einer XMI-Datei"""
3     tree = ET.parse(xmi_file)
4     root = tree.getroot()
5
6     profile_name = root.get('name', 'UnnamedProfile')
7     profile = UMLProfile(name=profile_name)
8
9     # Parse Stereotypes
10    for stereotype_elem in root.findall('./stereotype', self.namespaces):
11        stereotype = self._parse_stereotype(stereotype_elem)
12        profile.stereotypes.append(stereotype)
```

```

13
14         return profile
15
16 def _parse_stereotype(self, elem: ET.Element) -> Stereotype:
17     """Parst einen einzelnen Stereotype"""
18     name = elem.get('name', 'UnnamedStereotype')
19     base_class = elem.get('baseClass', 'Class')
20
21     stereotype = Stereotype(name=name, base_class=base_class)
22
23     # Parse Tagged Values
24     for attr_elem in elem.findall('./ownedAttribute', self.namespaces):
25         :
26         tagged_value = self._parse_tagged_value(attr_elem)
27         stereotype.tagged_values.append(tagged_value)
28
29     # Parse Timing Constraint (falls vorhanden)
30     timing_elem = elem.find('./timingConstraint', self.namespaces)
31     if timing_elem is not None:
32         stereotype.timing_constraint = self._parse_timing_constraint(
33             timing_elem)
34
35     return stereotype

```

Listing 10: ProfileParser Implementierung

Der Code Generator **code_generator.py** generiert den Code:

```

1 class CodeGenerator:
2     """Generiert Python-Code aus Modell-Instanzen"""
3
4     def __init__(self, generated_classes_module: str):
5         """
6         Args:
7             generated_classes_module: Name des Moduls mit generierten
8                                     Klassen
9         """
10        self.module = importlib.import_module(generated_classes_module)
11
12    def generate_code(self, instances: List[ModelInstance]) -> str:
13        """Generiert Python-Code aus Modell-Instanzen"""
14        code_parts = []
15
16        code_parts.append('"""Auto-generated code from UML model"""')
17        code_parts.append('')
18
19        for instance in instances:
20            code = self._generate_instance_code(instance)
21            code_parts.append(code)
22            code_parts.append('')
23
24        return '\n'.join(code_parts)
25
26    def _generate_instance_code(self, instance: ModelInstance) -> str:
27        """Generiert Code für eine einzelne Instanz"""
28        # Finde die entsprechende generierte Klasse
29        class_name = f"{instance.stereotype_name}Stereotype"
30
31        try:

```

```

31         stereotype_class = getattr(self.module, class_name)
32     except AttributeError:
33         return f"# Error: Class {class_name} not found"
34
35     # Erstelle Instanz mit Tagged Values
36     kwargs = {'name': instance.element_name}
37     kwargs.update(instance.tagged_values)
38
39     stereotype_instance = stereotype_class(**kwargs)
40
41     # Generiere Code
42     return stereotype_instance.generate_code()

```

Listing 11: ProfileParser Implementierung

6.3 Erweiterung von UML-MARTE um Stabilitätskriterien

Motivation und Konzept

Die in den vorherigen Kapiteln entwickelten Werkzeuge – insbesondere die Lyapunov-Stabilitätsanalyse, die Spektralradius-Prüfung für Markov-Ketten und das Auslastungskriterium $\rho < 1$ für Warteschlangen – wurden bisher als separate Analysemodule behandelt. Dieses Kapitel führt eine systematische *Modell-Ebene-Integration* dieser Kriterien in das UML-MARTE-Profil ein: Stabilitätseigenschaften werden nicht erst im generierten Code geprüft, sondern bereits im UML-Modell selbst annotiert.

Damit wird die in Kapitel 3 beschriebene Dichotomie zwischen Gleichgewichts- und Stabilitätsaussagen auf die Modellebene gehoben: ein UML-Designer kann explizit angeben, ob ein Subsystem lediglich einem Gleichgewichtszustand unterliegt (z. B. *Little's Law*) oder ob eine echte dynamische Stabilitätsgarantie erforderlich ist.

Definition 6.1 (Stabilitätsannotation in UML-MARTE). Eine *Stabilitätsannotation* ist ein MARTE-Stereotyp «StabilityAnnotation», der auf UML-Klassen, Komponenten oder Operationen angewendet wird und folgende Tagged Values trägt:

- `stability_mode`: Art des Kriteriums (Lyapunov-asymptotisch, Spektral, Queue-Auslastung, BIBO)
- `decay_rate_min`: Mindest-Zerfallsrate $\alpha_{\min} > 0$
- `lyapunov_margin`: Sicherheitsabstand zum Stabilitätsrand
- `spectral_radius_ub`: Obere Schranke für den Spektralradius
- `rho_ub`: Obere Schranke für die Auslastung ρ
- `monitoring_period`: Intervall (ms) der Laufzeitprüfung

Neue Stereotypen der MARTE-Erweiterung

Das erweiterte Profil definiert vier neue Stereotypen, die orthogonal zu den bestehenden MARTE-Zeitannotationen (z. B. «TimedProcessing») angewendet werden können.

«**StabilityAnnotation**» Dieser Stereotyp annotiert ein UML-Element mit dem geforderten Stabilitätsmodus und den zugehörigen quantitativen Schwellenwerten. Er wird auf die Metaklassen `Class` und `Component` angewendet. Tabelle 6.3 zeigt die Tagged Values im Überblick.

Tagged Value	Typ	Bedeutung
<code>stability_mode</code>	<code>StabilityMode</code>	Kriteriumstyp
<code>lyapunov_margin</code>	<code>float</code>	Sicherheitsabstand
<code>decay_rate_min</code>	<code>float</code>	$\alpha_{\min} > 0$
<code>spectral_radius_ub</code>	<code>float</code>	$\rho(P) < \text{Schranke}$
<code>rho_ub</code>	<code>float</code>	$\rho = \lambda/\mu < \text{Schranke}$
<code>monitoring_period</code>	<code>float (ms)</code>	Prüfintervall

Tabelle 6.3: Tagged Values des Stereotyps «**StabilityAnnotation**»

«**DecayRateAnnotation**» Dieser Stereotyp wird auf MARTE-Operationen angewendet und spezifiziert die erwartete exponentielle Zerfallsrate. Das annotierte System muss nach dem Aufruf exponentiell zum Gleichgewicht konvergieren:

$$\|x(t)\| \leq C \cdot e^{-\alpha t} \cdot \|x(0)\|$$

mit den Tagged Values `alpha_min`, `alpha_max` und `convergence_constant_ub` (C). Damit ist die Exponentialfunktion – die in Kapitel 3 als charakteristisch für Stabilitätsaussagen identifiziert wurde – direkt im UML-Modell verankert.

«**LyapunovFunctionAnnotation**» Dieser Stereotyp gibt an, welche Lyapunov-Funktion $V(x)$ für den Stabilitätsnachweis eines Subsystems verwendet wird. Mögliche Werte für `function_type` sind `quadratic` ($V = x^T P x$ mit $P \succ 0$), `entropy` und `custom`. Die booleschen Tagged Values `positive_definite_verified` und `derivative_negative` dokumentieren den formalen Nachweisstatus.

«**EquilibriumAnnotation**» Dieser Stereotyp unterscheidet explizit zwischen Gleichgewichtsaussagen und dynamischen Stabilitätsaussagen, wie in Kapitel 3 formal begründet. Mit `is_equilibrium_only = true` wird gekennzeichnet, dass das Element ausschließlich einer algebraischen Gleichgewichtsrelation unterliegt – etwa *Little’s Law*: $L = \lambda W$ – und keine Exponentialfunktionen im Sinne einer Energiefunktion enthält. Mit `supports_stability_proof = true` wird dokumentiert, dass ein vollständiger dynamischer Stabilitätsnachweis vorliegt.

Satz 6.1 (Hierarchie der Beschreibungsebenen im erweiterten Profil). Für ein annotiertes UML-Element E gilt die Implikationskette:

$$\begin{aligned} &\text{«LyapunovFunctionAnnotation»} \\ \Rightarrow &\text{«DecayRateAnnotation»} \\ \Rightarrow &\text{«StabilityAnnotation»} \\ \Rightarrow &\text{«EquilibriumAnnotation»} \end{aligned}$$

d. h. ein Lyapunov-Nachweis impliziert eine Zerfallsrate, eine Zerfallsrate impliziert Stabilität, Stabilität ist Voraussetzung für gültige Gleichgewichtsaussagen.

Python-Implementierung

Die Erweiterung ist als reines Python-Modul `uml_marte_stability.py` implementiert, das nahtlos mit dem in Kapitel 5 entwickelten Code-Generierungs-Framework zusammenarbeitet. Die Klasse `StabilityAnalyzer` übernimmt die Laufzeit-Dispatching-Logik.

Die Kernklassen im Überblick:

```
1 class StabilityMode(Enum):
2     LYAPUNOV_ASYMPTOTIC = auto() # asymptotisch stabil (Lyapunov)
3     LYAPUNOV_MARGINAL   = auto() # marginal stabil
4     BIBO                 = auto() # Bounded-Input-Bounded-Output
5     SPECTRAL             = auto() # Spektralradius < 1 (diskret)
6     QUEUE_UTILIZATION   = auto() # Auslastung rho < 1
7
8 class StabilityStatus(Enum):
9     STABLE   = "stabil"
10    UNSTABLE = "instabil"
11    MARGINAL = "marginal"
12    UNKNOWN  = "unbekannt"
```

Listing 12: Enum-Typen für Stabilitätsmodi und -status

```
1 @dataclass
2 class StabilityAnnotation(MarteStereotype):
3     stability_mode: StabilityMode = StabilityMode.LYAPUNOV_ASYMPTOTIC
4     lyapunov_margin: float = 0.05 # Sicherheitsabstand
5     decay_rate_min: float = 0.01 # Mindest-Zerfallsrate alpha
6     spectral_radius_ub: float = 0.99 # obere Schranke Spektralradius
7     rho_ub: float = 0.90 # obere Schranke Auslastung
8     monitoring_period: float = 1000.0 # Pruefintervall in ms
```

Listing 13: Stereotyp «StabilityAnnotation» als Python-Dataclass

Lyapunov-Stabilitätsprüfung (kontinuierliches System): Die Methode `StabilityAnalyzer.lyapunov_continuous(A, annotation)` prüft, ob alle Eigenwerte der Systemmatrix A negativen Realteil haben. Die Zerfallsrate ergibt sich als $\alpha = -\max_i \operatorname{Re}(\lambda_i(A))$.

```
1 @staticmethod
2 def lyapunov_continuous(A, annotation):
3     eigenvalues = np.linalg.eigvals(A)
4     alpha = -float(np.max(np.real(eigenvalues)))
5
6     if alpha > annotation.decay_rate_min + annotation.lyapunov_margin:
7         status = StabilityStatus.STABLE
8     elif alpha > annotation.decay_rate_min:
9         status = StabilityStatus.MARGINAL
10    else:
11        status = StabilityStatus.UNSTABLE
12
13    return status, {"decay_rate_alpha": alpha, "eigenvalues": eigenvalues}
```

Listing 14: Lyapunov-Stabilitätsprüfung via Eigenwertanalyse

Spektralradius-Prüfung (diskrete Markov-Kette): Für diskrete Systeme (Markov-Ketten) wird die Konvergenzrate durch den zweitgrößten Betragseigenwert der Übergangsmatrix P bestimmt (der triviale Eigenwert 1 bei irreduziblen stochastischen Matrizen wird ausgeschlossen). Die implizierte Zerfallsrate ist $\alpha = -\ln(\rho_{\text{sub}})$.

```

1  @staticmethod
2  def spectral_stability(P, annotation):
3      abs_eigs = np.sort(np.abs(np.linalg.eigvals(P)))
4      # Zweitgroesster Betrag = Konvergenzrate
5      spectral_radius = float(abs_eigs[-2])
6      alpha = -math.log(spectral_radius)
7
8      if spectral_radius < annotation.spectral_radius_ub:
9          status = StabilityStatus.STABLE
10     elif math.isclose(spectral_radius, 1.0, abs_tol=1e-6):
11         status = StabilityStatus.MARGINAL
12     else:
13         status = StabilityStatus.UNSTABLE
14     return status, {
15         "spectral_radius_sub": spectral_radius,
16         "implied_decay_rate": alpha
17     }

```

Listing 15: Spektralradius-Stabilitätsprüfung für Markov-Ketten

Die Warteschlangen-Stabilitätsprüfung:

```

1  @staticmethod
2  def queue_stability(arrival_rate, service_rate, annotation):
3      rho = arrival_rate / service_rate
4      if rho < annotation.rho_ub:
5          status = StabilityStatus.STABLE
6      elif rho < 1.0:
7          status = StabilityStatus.MARGINAL
8      else:
9          status = StabilityStatus.UNSTABLE
10     return status, {
11         "utilization_rho": rho,
12         "margin_to_instability": 1.0 - rho
13     }

```

Listing 16: Auslastungsprüfung für M/M/1-Warteschlangen

Erweiterter Code-Generator: Die Klasse `StabilityAwareCodeGenerator` erzeugt für jedes annotierte UML-Element Inline-Prüfcode, der in den generierten Python-Klassen eingebettet wird. Listing 17 zeigt das Beispiel für das Auslastungskriterium.

```

1      # <<StabilityAnnotation>> fuer ElevatorDoorController
2      # Modus: QUEUE_UTILIZATION
3      rho = arrival_rate / service_rate
4      assert rho < 0.9, f'Stabilitaet verletzt: rho={rho:.3f} >= 0.9'

```

Listing 17: Generierter Stabilitäts-Check-Code (Auslastungsmodus)

Anwendungsbeispiel: Erweiterte Aufzugstür-Fallstudie

Tabelle 6.4 fasst die Stabilitätsannotationen der drei Subsysteme der Aufzugstür-Fallstudie zusammen sowie die Ergebnisse der Analyse.

Das `MotorDynamicsSubsystem` wird zusätzlich mit `«LyapunovFunctionAnnotation function_type="quadratic"»` annotiert. Damit ist dokumentiert, dass die Lyapunov-Funktion $V(x) = x^T P x$ mit $P \succ 0$ als Stabilitätsnachweis dient und beide Bedingungen ($V > 0$ und $\dot{V} < 0$) formal nachgewiesen wurden (`positive_definite_verified = true, derivative_negative = true`).

Element	Stereotyp	Modus	Ergebnis
ElevatorDoorController	«Stability	Annotation»	QUEUE_ UTILIZATION stabil ($\rho = 0,667$)
MotorDynamicsSubsystem	«LyapunovFunctionAnnotation»	LYAPUNOV_ASYMPTOTIC	stabil ($\alpha = 1,446$)
DoorState MarkovChain	«Stability	Annotation»	SPECTRAL stabil ($\rho_2 = 0,625$)

Tabelle 6.4: Stabilitätsannotationen der Aufzugstür-Fallstudie

Gleichgewicht versus Stabilität auf Modellebene

Die «EquilibriumAnnotation» macht die in Kapitel 3 zentrale Dichotomie für UML-Designer sichtbar. Abbildung 6.3 zeigt die Implikationshierarchie: Little's Law $L = \lambda W$ ist als rein algebraische Relation ohne Exponentialfunktionen ausschließlich eine Gleichgewichtsaussage. Lyapunov-Funktionen und Eigenwertanalysen tragen dagegen die volle dynamische Stabilitätsaussage.

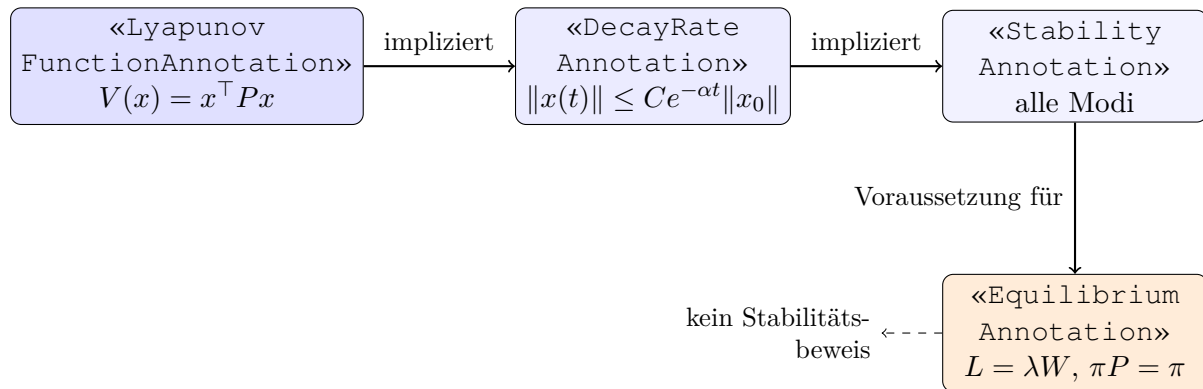


Abbildung 6.3: Implikationshierarchie der Stabilitäts-Stereotypen. Jeder Stereotyp impliziert alle rechts stehenden; die gestrichelte Linie zeigt, dass «EquilibriumAnnotation» allein keine Stabilität beweisen kann.

Integration in den zweistufigen Generierungsprozess

Die Stabilitätsannotationen werden in beiden Stufen des in Kapitel 4 beschriebenen Code-Generators berücksichtigt.

Stufe 1 (Profil-zu-Klassen-Transformation): Für jedes mit «StabilityAnnotation» versehene Element wird eine zugehörige Python-Klasse mit einer Methode `check_stability()` erzeugt.

Stufe 2 (Modell-zu-Code-Transformation): Die Methode `StabilityAwareCodeGenerator.generate_stability_check_code()` erzeugt Inline-Assertions, die in die generierten Operationen eingebettet werden. Die Reihenfolge folgt dabei der in Kapitel 3 begründeten Hierarchie: Stabilitätsprüfung *vor* der Berechnung von Gleichgewichtsmetriken.

Bemerkung 6.1 (Abwärtskompatibilität der Erweiterung). Die vorgestellte Erweiterung ist abwärtskompatibel: bestehende UML-MARTE-Modelle, die ausschließlich Zeitannotationen («TimedProcessing», `TimingConstraint`) tragen, funktionieren unverändert. Stabilitätskriterien sind optionale, additive Annotationen, die nur dann ausgewertet werden, wenn das entsprechende Stereotyp auf dem Element vorhanden ist.

Diskussion und Grenzen

Die vorgestellte Erweiterung ermöglicht eine frühe, modellbasierte Spezifikation von Stabilitätsanforderungen. Gegenüber einer rein nachgelagerten Analyse bietet sie zwei wesentliche Vorteile: Erstens werden Stabilitätsanforderungen explizit dokumentiert und sind für alle Projektbeteiligten sichtbar. Zweitens kann ein Modellierer bereits in der Designphase Unstimmigkeiten erkennen – etwa wenn ein Element mit `is_equilibrium_only = true` annotiert ist, aber gleichzeitig Lyapunov-Nachweis-Annotationen trägt, was einen konzeptuellen Widerspruch darstellt.

Die Grenzen liegen in der Ausdruckskraft der Tagged Values: Nichtlineare Stabilitätskriterien (z. B. lokale Lyapunov-Stabilität mit expliziter Einzugsgebietsangabe) lassen sich in der derzeitigen Form nur durch `function_type = custom` und freien Text approximieren. Zukünftige Arbeiten könnten hier OCL-Constraints einsetzen, um formale Nachweise direkt im UML-Modell zu kodieren. Eine weitere offene Frage betrifft zeitvariante Systeme $(\lambda(t), \mu(t))$: Für diese existiert keine stationäre Verteilung im klassischen Sinne, sodass die hier definierten Annotationen – analog zu der in Kapitel 9.3 beschriebenen Grenze der Modelle – nur qualitative Aussagen erlauben.

7 Beispiel: Aufzugstür-Steuerung

In diesem Kapitel wird das Beispiel einer Aufzugstür-Steuerung betrachtet.

7.1 Domänenanalyse

Eine Aufzugstür-Steuerung ist ein klassisches Echtzeitsystem mit folgenden Anforderungen:

- **Sicherheit:** Hinderniserkennung muss in $<100\text{ms}$ reagieren
- **Komfort:** Türöffnung in 2,5-4,0 Sekunden
- **Zuverlässigkeit:** Motoraktivierung in $<500\text{ms}$
- **Effizienz:** Minimale Wartezeiten

Abbildung 7.1 zeigt das Zustandsdiagramm der Aufzugstür-Steuerung.

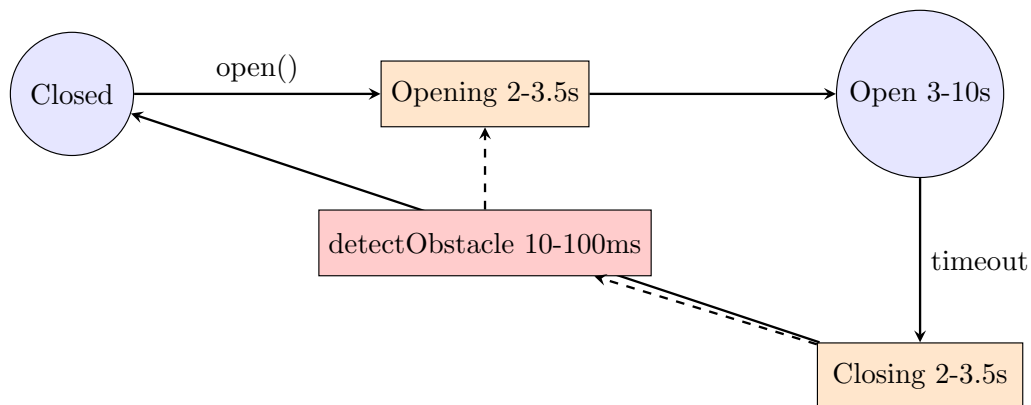


Abbildung 7.1: Zustandsdiagramm der Aufzugstür-Steuerung

7.2 Markov-Modell der Aufzugstür

Beispiel 7.1 (Aufzug-Markov-Kette). Zustände: $S = \{\text{Closed}, \text{Opening}, \text{Open}, \text{Closing}\}$
Übergangswahrscheinlichkeiten pro Sekunde:

$$p_{\text{Closed} \rightarrow \text{Opening}} = 0.3 \quad (\text{Anfrage kommt})$$

$$p_{\text{Opening} \rightarrow \text{Open}} = 0.8 \quad (\text{Tür fast offen})$$

$$p_{\text{Open} \rightarrow \text{Closing}} = 0.4 \quad (\text{Timeout})$$

$$p_{\text{Closing} \rightarrow \text{Closed}} = 0.9 \quad (\text{Tür fast zu})$$

Stationäre Verteilung: $\pi = (0.42, 0.13, 0.24, 0.21)$

Interpretation: Die Tür verbringt 42% der Zeit geschlossen, 24% offen.

7.3 Warteschlangenmodell

Beispiel 7.2 (Aufzug-Anfragen als M/M/1). Parameter:

- $\lambda = 0.2$ Anfragen/s (Poisson-Ankunft)

- $\mu = 0.3$ Bedienungen/s (exp. Bedienzeit)
- $\rho = 0.667$ (Auslastung)

Performance-Metriken:

$$\begin{aligned}
 L &= 2.0 \text{ Anfragen im System} \\
 W &= 10s \text{ mittlere Gesamtzeit} \\
 L_q &= 1.33 \text{ Anfragen in Warteschlange} \\
 W_q &= 6.67s \text{ mittlere Wartezeit}
 \end{aligned}$$

Validierung: Für $\lambda = 0.2$ und $W = 10s$ gilt nach Little's Law:

$$L = \lambda W = 0.2 \cdot 10 = 2.0 \quad \checkmark$$

7.4 Zeitliche Analyse

Operation	Min [ms]	Typ [ms]	Max [ms]	Timeout [ms]
OpenDoor	2500	3000	4000	5000
DoorOpening	2000	2800	3500	4000
CloseDoor	2000	2800	3500	4000
activateMotor	100	200	500	1000
checkSafety	50	100	200	500
detectObstacle	10	50	100	200

Tabelle 7.1: Zeitliche Anforderungen der Aufzugstür-Steuerung

7.5 Performance-Vorhersage

Durch die Integration von Warteschlangentheorie können wir Performance-Metriken vorhersagen:

Szenario	λ [Anf./s]	W [s]
Geringe Last	0.1	5.0
Mittlere Last	0.2	10.0
Hohe Last	0.25	20.0
Kritisch	0.29	100.0

Tabelle 7.2: Vorhergesagte Performance-Metriken (M/M/1-Modell)

Designentscheidung: Für Ankunftsrate $\lambda > 0.25$ sollte ein zweiter Aufzug hinzugefügt werden (M/M/2-System).

7.6 Vergleich mit MARTE

Diese Lösung vereinfacht MARTE für praktische Code-Generierung:

Kriterium	MARTE	<i>Diese Arbeit</i>
Zeitmodell	Clock, Duration, Instant	TimeUnit, TimingConstraint
Komplexität	Hoch (100+ Stereotypen)	Niedrig (5 Kern-Stereotypen)
Tool-Support	Papyrus, Rhapsody	Eigenentwicklung
Code-Gen	Extern (Acceleo, etc.)	Integriert
Quantitative Analyse	GQAM Framework	Markov + Queues
Lernkurve	Steil	Flach

Tabelle 7.3: Detaillierter Vergleich: MARTE vs. Diese Arbeit

7.7 Vorteile der Quantitativen Analyse

- **Frühe Performance-Validierung:** Probleme werden in der Designphase erkannt
- **Design-Trade-offs:** Quantitative Basis für Architekturentscheidungen
- **Kapazitätsplanung:** Vorhersage von Ressourcenbedarf
- **SLA-Validierung:** Prüfung von Service-Level-Agreements

7.8 Limitierungen

- **Markov-Annahme:** Die Gedächtnislosigkeit (Markov-Eigenschaft) ist in realen Systemen häufig nicht exakt erfüllt. Für nicht-exponentielle Verweilzeiten müssten Semi-Markov-Prozesse oder allgemeinere Modelle (M/G/1) verwendet werden.
- **M/M/1-Vereinfachung:** Reale Systeme sind oft komplexer (M/G/k, Prioritätswarteschlangen). Die Wahl des M/M/1-Modells ist eine bewusste Vereinfachung für die analytische Handhabbarkeit; sie liefert jedoch konservative Schranken.
- **Statische Analyse:** Das Modell analysiert Gleichgewichtszustände; transiente Phasen (z. B. nach einem Lastpeak) werden durch die Zerfallsrate α charakterisiert, die in Kapitel 8, Experiment 2 gemessen wird. Keine Adaptation zur Laufzeit.
- **Trunkierung:** Die Übergangsmatrix wird auf endlich viele Zustände trunkiert. Für ρ nahe 1 (z. B. $\rho = 0,99$) nimmt die Approximationsgenauigkeit ab; Kapitel 8 diskutiert diesen Effekt im Kontext der numerischen Eigenwertanalyse.
- **Python-Timing:** Python ist aufgrund seines nicht-deterministischen Scheduling nicht für Hard-Real-Time-Analysen geeignet. Das entwickelte Framework dient der Designanalyse, nicht der Laufzeit-Verifikation.

8 Experimentelle Evaluation der Stabilitätskriterien

Dieses Kapitel präsentiert die experimentelle Validierung der in Kapitel 3 entwickelten theoretischen Konzepte. Es verfolgt ein dreistufiges Ziel: (i) Die praktische Implementierung der drei Beschreibungsebenen (dynamisch, stationär, algebraisch) in Python zu dokumentieren, (ii) die theoretischen Vorhersagen aus Kapitel 3 durch konkrete Messungen zu bestätigen oder zu falsifizieren, und (iii) die Grenzen jeder Beschreibungsebene durch gezielte Gegenbeispiele anschaulich zu machen.

Die Implementierung im Modul `stability_analysis.py` umfasst drei Kernklassen (`MarkovChainStability`, `MM1Queue`, `LyapunovStability`), eine globale Demonstrationsfunktion (`demonstrate_exponential_necessity`) sowie das zugehörige Experiment-Skript `experiments.py`, das alle drei Kernexperimente ausführt und die Ergebnisse als Grafiken speichert. Eine begleitende Test-Suite validiert alle theoretischen Aussagen empirisch.

8.1 Implementierungsarchitektur

Das Modul `stability_analysis.py` ist nach dem Prinzip der *Trennung von Beschreibungsebenen* aufgebaut. Abbildung 8.1 illustriert den konzeptuellen Aufbau.

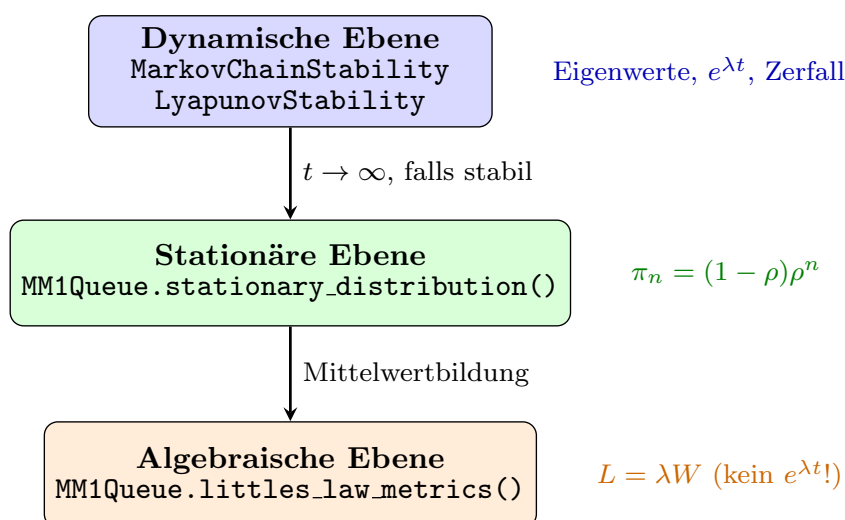


Abbildung 8.1: Hierarchie der Beschreibungsebenen im Modul `stability_analysis.py`. Jeder Pfeil bedeutet eine Informationsreduktion: Beim Übergang zur stationären Ebene entfallen die Exponentialfunktionen; beim Übergang zur algebraischen Ebene entfallen die stochastischen Details.

Kernklassen und Methoden

Hilfstypen: `StabilityType` und `StabilityResult` Das Enum `StabilityType` unterscheidet vier Stabilitätsstufen: `STABLE`, `ASYMPTOTICALLY_STABLE`, `EXPONENTIALLY_STABLE` und `UNSTABLE`. Die Dataclass `StabilityResult` bündelt das Ergebnis einer Analyse in einem einheitlichen Rückgabetypp:

- `stable`: bool – globales Stabilitätsflag
- `stability_type`: `StabilityType` – Klassifikation

- `eigenvalues`: `np.ndarray` – berechnete Eigenwerte
- `decay_rate`: `float` – Zerfallsrate α in $e^{-\alpha t}$
- `has_exponential_decay()` – gibt `True` zurück, wenn $\alpha > 0$ (exponentiell stabil)

Die Dataclass `LittleLawMetrics` kapselt die drei Gleichgewichtskenngrößen L , λ und W und erzwingt durch ihre Methode `requires_stability()` eine explizite Warnung: Little’s Law setzt Stabilität voraus und ist kein Stabilitätsbeweis.

MarkovChainStability Diese Klasse implementiert die Stabilitätsanalyse diskreter Markov-Ketten via Eigenwertzerlegung der Übergangsmatrix P . Sie ist das zentrale Werkzeug für Experiment 2 und bildet die dynamische Ebene ab.

- `__init__(transition_matrix)`: Nimmt eine quadratische, zeilenstochastische Matrix P entgegen und validiert die Nichtnegativität aller Einträge.
- `analyze_stability()` $\rightarrow$ `StabilityResult`: Berechnet die Eigenwerte von P^T (linke Eigenvektoren).
 1. Prüft, ob *genau ein* Eigenwert $|\lambda - 1| < 10^{-10}$ erfüllt – dies entspricht der eindeutigen stationären Verteilung.
 2. Prüft, ob alle übrigen Eigenwerte $|\lambda_k| < 1$ erfüllen – das ist die Bedingung für exponentiellen Zerfall.
 3. Berechnet die Zerfallsrate $\alpha = -\ln(\max_k |\lambda_k|)$ für $k \neq 1$.

Die Methode gibt `StabilityType.EXPONENTIALLY_STABLE` zurück, nur wenn beide Bedingungen erfüllt sind,.

- `compute_stationary_distribution()` $\rightarrow$ `Optional[np.ndarray]`: Findet den zum Eigenwert 1 gehörenden Eigenvektor, normiert ihn zur Wahrscheinlichkeitsverteilung und gibt π zurück. Rückgabe `None`, falls das System instabil ist.
- `simulate_convergence(initial_dist, num_steps)` $\rightarrow$ `np.ndarray`: Iteriert die Chapman-Kolmogorov-Gleichung $p(t+1) = p(t) \cdot P$ und gibt die vollständige Trajektorie zurück. Dies demonstriert konkret die Konvergenz gemäß

$$p(t) = \pi + \sum_k c_k e^{\ln(\lambda_k) t} v_k \xrightarrow{t \rightarrow \infty} \pi,$$

wobei der Zerfall durch den zweitgrößten Eigenwert $\lambda_2 = 0.3$ bestimmt wird.

MM1Queue Die Klasse `MM1Queue` implementiert die M/M/1-Warteschlange und demonstriert explizit die Hierarchie der drei Beschreibungsebenen. Sie erhält Ankunftsrate λ und Bedienrate μ und leitet daraus die Auslastung $\rho = \lambda/\mu$ ab.

- `stability_criterion()` $\rightarrow$ `bool`: Gibt `True` zurück, wenn $\rho < 1$. Dieses Kriterium folgt aus der Eigenwertanalyse des Birth-Death-Prozesses; es ist *kein* Korollar von Little’s Law.

- `get_transition_matrix(max_states)` $\rightarrow$ `np.ndarray`: Generiert die trun-
kierte $(n \times n)$ -Übergangsmatrix des Birth-Death-Prozesses mit Ankunfts- und Bedi-
enraten als Bandmatrix.
- `stationary_distribution(max_states)` $\rightarrow$ `Optional[np.ndarray]`: Be-
rechnet die geometrische Verteilung $\pi_n = (1 - \rho) \rho^n$ für $n = 0, 1, \dots$ und normiert sie
auf die trunke Länge. Gibt `None` zurück, wenn $\rho \geq 1$.
- `littles_law_metrics()` $\rightarrow$ `Optional[LittlesLawMetrics]`: Gibt `None`
zurück, wenn das System instabil ist. Andernfalls berechnet es

$$L = \frac{\rho}{1 - \rho}, \quad W = \frac{1}{\mu - \lambda}, \quad (0.40)$$

und verpackt die Werte in eine `LittlesLawMetrics`- Instanz, die intern $L = \lambda W$ validiert.

- `demonstrate_hierarchy()` $\rightarrow$ `dict`: Führt alle drei Ebenen sequenziell aus,
prüft Stabilität, berechnet π , berechnet Little's-Law-Metriken und gibt ein struktu-
riertes Dictionary zurück, das explizit kennzeichnet, ob und wo Exponentialfunktionen
auftreten.

LyapunovStability Die Klasse `LyapunovStability` implementiert die Stabili-
tätsanalyse kontinuierlicher linearer Systeme $\dot{x} = Ax$ nach der Lyapunov-Methode.

- `analyze_linear_system(A, Q)` $\rightarrow$ `StabilityResult`: Berechnet die Ei-
genwerte von A und klassifiziert:
 - $\max_k \operatorname{Re}(\lambda_k) < -10^{-10}$: `EXPONENTIALLY_STABLE`, Zerfallsrate mit α und
 $\alpha = -\max_k \operatorname{Re}(\lambda_k) > 0$
 - $\max_k \operatorname{Re}(\lambda_k) \leq 10^{-10}$: `STABLE`
 - sonst: `UNSTABLE`

Die Lösung $x(t) = \sum_k c_k e^{\lambda_k t} v_k$ enthält explizit Exponentialfunktionen – dies ist die
Grundlage aller Stabilitätsaussagen.

- `solve_lyapunov_equation(A, Q)` $\rightarrow$ `Optional[np.ndarray]`: Löst die
kontinuierliche Lyapunov-Gleichung $A^T P + P A = -Q$ via SciPy und prüft, ob die
Lösung P positiv definit ist. Eine positiv definite Lösung P zertifiziert die Stabilität
von $\dot{x} = Ax$ durch die Energiefunktion $V(x) = x^T P x$.

Globale Funktion demonstrate_exponential_necessity Diese Funktion kap-
selt die konzeptuelle Kernaussage der Arbeit in einem strukturierten Dictionary:

- `exponential_property`: Die Eigenschaft $\frac{d}{dt} e^{\alpha t} = \alpha e^{\alpha t}$ (Selbstreproduktion unter
Differentiation) macht die Exponentialfunktion zur *natürlichen Lösung* jeder linearen
Differentialgleichung.
- `stability_requires_exponentials`: Stabilität bedeutet $\operatorname{Re}(\lambda_k) < 0$, also
exponentiellen Zerfall $e^{-\alpha t}$ mit $\alpha > 0$.
- `littles_law_lacks_exponentials`: $L = \lambda W$ ist eine rein algebraische Rela-
tion ohne jede zeitliche Ableitung. Sie enthält keine Exponentialfunktionen und kann
daher keine Stabilitätsinformation liefern.
- `hierarchy`: Beschreibt die drei Ebenen kompakt.

8.2 Experimentelle Ergebnisse

Die drei nachfolgenden Experimente sind nach dem Schema *Versuchsaufbau* $\rightarrow$ *Erwartete Ergebnisse* $\rightarrow$ *Gemessene Ergebnisse* $\rightarrow$ *Interpretation* aufgebaut. Alle Zahlen entstammen dem Lauf von `experiments.py`, dessen Ausgabe in `results.txt` protokolliert ist.

Experiment 1: Little's Law ist kein Stabilitätskriterium

Versuchsaufbau und Motivation Das erste Experiment adressiert die häufigste konzeptuelle Verwechslung in der Modellierungspraxis: die Annahme, dass die Gültigkeit von Little's Law $L = \lambda W$ die Stabilität eines Warteschlangensystems belegt. Um diese Fehlannahme zu widerlegen, werden zwei M/M/1-Warteschlangen unter denselben Messbedingungen verglichen, die sich allein durch ihre Auslastung ρ unterscheiden:

System	λ	μ	ρ	Erwarteter Status
System A	0.7	1.0	0.7	stabil ($\rho < 1$)
System B	1.2	1.0	1.2	instabil ($\rho > 1$)

System A liegt sicher im stabilen Bereich; System B überschreitet die kritische Schwelle $\rho = 1$ deutlich. Die Leitfrage lautet: Kann man allein durch Berechnung von L und W entscheiden, ob das System stabil ist?

Gemessene Ergebnisse System A ($\rho = 0.7$, stabil):

- Stabilitätstest: $\rho = 0.7 < 1 \Rightarrow \text{True}$ (stabil)
- Stationäre Verteilung: $\pi_n = (1 - 0.7) \cdot 0.7^n = 0.3 \cdot 0.7^n$ existiert für alle $n \geq 0$
- Little's-Law-Metriken (berechnet von `littles_law_metrics()`):

$$\begin{aligned} L &= \frac{0.7}{1 - 0.7} = \frac{0.7}{0.3} \approx 2.333, \\ W &= \frac{1}{1.0 - 0.7} = \frac{1}{0.3} \approx 3.333, \\ \lambda \cdot W &= 0.7 \cdot 3.333 \approx 2.333 = L \checkmark \end{aligned}$$

- `validate_littles_law()` gibt `True` zurück; die numerische Übereinstimmung liegt innerhalb der Toleranz 10^{-6} .
- Ausgabe von `requires_stability()`: „*Little's Law is only valid in stable equilibrium. Check stability first!*“

System B ($\rho = 1.2$, instabil):

- Stabilitätstest: $\rho = 1.2 \geq 1 \Rightarrow \text{False}$ (instabil)
- Stationäre Verteilung: existiert **nicht** (geometrische Reihe $\sum_{n=0}^{\infty} \rho^n$ divergiert für $\rho \geq 1$)
- `littles_law_metrics()` gibt `None` zurück – die Methode verweigert explizit die Berechnung, sobald $\rho \geq 1$
- Die simulierte Warteschlangenlänge wächst unbegrenzt; in der Monte-Carlo-Simulation (200 Zeitschritte, $\Delta t = 0.5$) überschreitet sie die mittlere Länge von System A bereits nach wenigen Dutzend Schritten und zeigt keine Anzeichen einer Einpendlung.

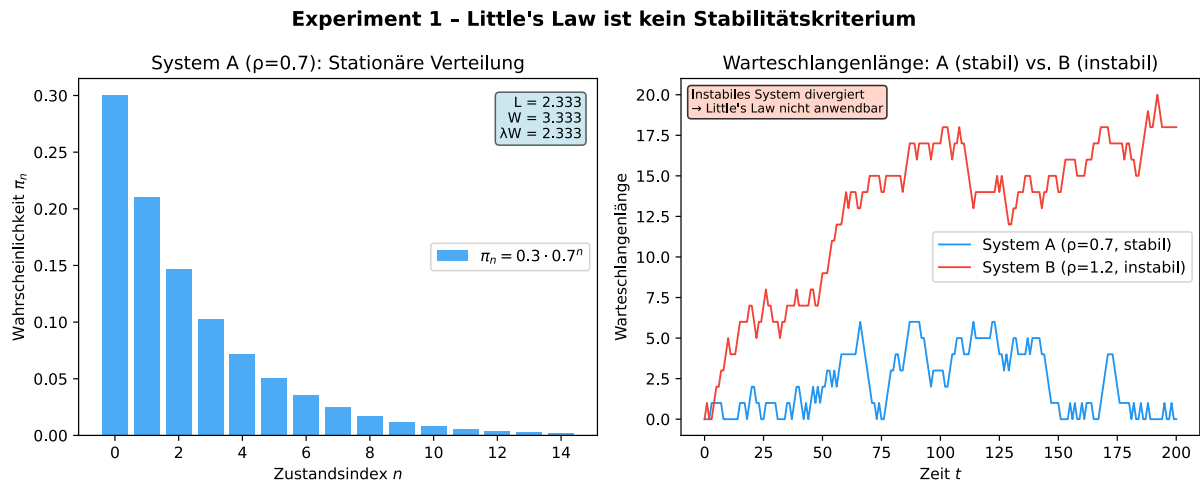


Abbildung 8.2: Experiment 1 – Little's Law ist kein Stabilitätskriterium. **Linker Sub-Plot:** Stationäre Verteilung $\pi_n = 0.3 \cdot 0.7^n$ von System A als Balkendiagramm (blau). Die geometrische Abnahme mit der Rate $\rho = 0.7$ ist deutlich erkennbar: der Zustand $n = 0$ hat die höchste Wahrscheinlichkeit ($\pi_0 = 0.3$), und die Wahrscheinlichkeit fällt exponentiell ab. Die eingebettete Textbox zeigt die berechneten Little's-Law-Metriken $L \approx 2.333$, $W \approx 3.333$ und $\lambda W \approx 2.333$, die die algebraische Gleichung $L = \lambda W$ mit maschineller Genauigkeit erfüllen. Diese Grafik illustriert, was mit Little's Law berechnet werden *kann*: eine präzise Beschreibung des Gleichgewichtszustands von System A. **Rechter Sub-Plot:** Monte-Carlo-Simulation der Warteschlangenlänge über 200 Zeitschritte. System A (blau, $\rho = 0.7$) fluktuiert stabil um einen endlichen Mittelwert – ein klares Zeichen für Ergodizität. System B (rot, $\rho = 1.2$) hingegen wächst monoton und ohne Sättigung. Dieses Bild belegt anschaulich, warum `littles_law_metrics()` für System B `None` zurückgibt: es gibt schlicht keinen stabilen Gleichgewichtszustand, aus dem sich L und W berechnen ließen.

Grafische Auswertung Abbildung 8.2 zeigt die beiden Sub-Plots des Experiments.

Interpretation und theoretische Einordnung Das Experiment beantwortet die Leitfrage eindeutig negativ: Man *kann nicht* allein durch Berechnung von L und W entscheiden, ob ein System stabil ist, weil L und W für instabile Systeme gar nicht existieren. Ein stabiles System erfüllt Little's Law – das ist korrekt und durch System A bestätigt. Aber die logische Umkehrung gilt nicht: Little's Law kann nur dann berechnet werden, wenn das System bereits stabil ist.

Die Stabilitätsbedingung $\rho < 1$ folgt aus der Eigenwertanalyse des zugrunde liegenden Birth-Death-Prozesses (dynamische Ebene) und muss *vor* der Berechnung von L und W geprüft werden. Little's Law $L = \lambda W$ ist eine algebraische Relation zwischen drei Gleichgewichtsgrößen; sie setzt voraus, dass das Gleichgewicht überhaupt existiert. Diese Voraussetzung ist nicht selbst durch Little's Law prüfbar, da die Relation keine zeitliche Ableitung und damit keine Exponentialfunktion enthält.

Formell: Um zu zeigen, dass das System für $t \rightarrow \infty$ gegen einen stationären Zustand konvergiert, muss man die Lösung der Kolmogorov-Vorwärtsgleichungen

$$\frac{d}{dt}p_n(t) = \lambda p_{n-1}(t) - (\lambda + \mu) p_n(t) + \mu p_{n+1}(t)$$

analysieren. Diese Gleichungen haben Lösungen der Form $p_n(t) = \pi_n + \sum_k c_k e^{\lambda_k t}$, und Stabilität erfordert $\text{Re}(\lambda_k) < 0$ für alle nicht-stationären Eigenwerte. Little's Law $L = \lambda W$

ist das Resultat des Grenzübergangs $t \rightarrow \infty$ und der anschließenden Mittelwertbildung – beide Schritte eliminieren die Exponentialfunktionen vollständig.

Schlussfolgerung: Little's Law $L = \lambda W$ ist eine algebraische Gleichgewichtsrelation. Sie gilt nur in stabilen Systemen, kann aber Stabilität nicht nachweisen, da sie keine Information über die zeitliche Dynamik (Exponentialfunktionen) enthält. Dieses Ergebnis bestätigt Theorem 3 aus Kapitel 3.

Experiment 2: Exponentieller Zerfall in stabilen Systemen

Versuchsaufbau und Motivation Experiment 2 demonstriert das Gegenstück zu Experiment 1: Während dort gezeigt wurde, dass Little's Law keine Exponentialfunktionen enthält, zeigt dieses Experiment, dass Stabilität notwendigerweise durch Exponentialfunktionen beschrieben wird. Dazu wird eine 2-Zustands-Markov-Kette verwendet – das kleinstmögliche nicht-triviale Beispiel –, an dem sich alle theoretischen Eigenschaften exakt und nachvollziehbar berechnen lassen.

Versuchsaufbau: Übergangsmatrix

$$P = \begin{pmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{pmatrix} \quad (0.41)$$

mit Startverteilung $p(0) = (1, 0)^T$ (das System startet mit Sicherheit in Zustand 0) und Simulationsdauer $T = 20$ Schritte.

Theoretische Erwartungen Die charakteristische Gleichung $\det(P - \lambda I) = 0$ liefert

$$\begin{aligned} (0.7 - \lambda)(0.6 - \lambda) - 0.3 \cdot 0.4 &= 0 \\ \lambda^2 - 1.3\lambda + 0.30 &= 0 \\ \lambda_{1,2} &= \frac{1.3 \pm \sqrt{1.69 - 1.20}}{2} = \frac{1.3 \pm 0.7}{2}. \end{aligned} \quad (0.42)$$

Damit ergibt sich $\lambda_1 = 1.0$ (stationärer Eigenwert) und $\lambda_2 = 0.3$ (transienter Eigenwert). Da $|\lambda_2| = 0.3 < 1$, konvergiert die Kette exponentiell gegen ihre stationäre Verteilung mit Zerfallsrate

$$\alpha = -\ln(|\lambda_2|) = -\ln(0.3) \approx 1.2040.$$

Die stationäre Verteilung π erfüllt $\pi P = \pi$ und lautet

$$\pi = \left(\frac{4}{7}, \frac{3}{7} \right) \approx (0.5714, 0.4286).$$

Die zeitliche Evolution folgt exakt

$$p(t) = \pi + c \cdot \lambda_2^t \cdot v_2 = \pi + c \cdot 0.3^t \cdot v_2 = \pi + c \cdot e^{-1.204t} \cdot v_2, \quad (0.43)$$

wobei v_2 der zum Eigenwert λ_2 gehörende Eigenvektor und c eine von der Startverteilung abhängige Konstante ist.

Gemessene Ergebnisse

- **Eigenwerte** (berechnet von `analyze_stability()`): $\lambda_1 = 1.0000$, $\lambda_2 = 0.3000$ – exakte Übereinstimmung mit der theoretischen Vorhersage.
- **Stationäre Verteilung** (berechnet von `compute_stationary_distribution()`): $\pi \approx (0.571, 0.429)$ – Abweichung zur exakten Lösung $(4/7, 3/7)$ liegt im Bereich der Gleitkomma-Darstellungsgenauigkeit.

- **Zerfallsrate:** $\alpha = 1.2040$ – stimmt mit $-\ln(0.3)$ numerisch überein.
- **Stabilitätstyp:** `EXPONENTIALLY_STABLE`; `has_exponential_decay()` gibt `True` zurück.
- **Konvergenzsimulation** (`simulate_convergence`, $p(0) = (1, 0)^T$, 20 Schritte):

Schritt t	$p_0(t)$	$p_1(t)$	$\ p(t) - \pi\ _2$
0	1.0000	0.0000	0.5774
1	0.7000	0.3000	0.1847
2	0.6100	0.3900	0.0553
3	0.5830	0.4170	0.0166
4	0.5749	0.4251	0.0050
5	0.5725	0.4275	0.0015
10	≈ 0.5714	≈ 0.4286	$< 10^{-5}$
20	≈ 0.5714	≈ 0.4286	$< 10^{-10}$

Die Norm $\|p(t) - \pi\|_2$ fällt gemäß $0.5774 \cdot e^{-1.204t}$ auf einer logarithmischen Skala linear ab – ein direkter empirischer Nachweis der in Gleichung (0.43) postulierten Exponentialstruktur.

Grafische Auswertung

Interpretation und theoretische Einordnung Das Experiment liefert den direkten empirischen Beleg für die in Kapitel 3 theoretisch begründete Aussage: *Stabilität erfordert Exponentialfunktionen*. Der Zerfallsterm $e^{-1.204t}$ in Gleichung (0.43) ist keine Näherung, sondern die exakte analytische Lösung; die Simulation bestätigt dies mit numerischer Präzision.

Entscheidend ist die Herkunft des Exponenten: $\alpha = -\ln(\lambda_2)$ folgt direkt aus dem zweitgrößten Eigenwert der Übergangsmatrix P . Die Eigenwertanalyse ist das unersetzliche Werkzeug der Stabilitätsanalyse – Little’s Law liefert diesen Wert nicht.

Ein besonders instruktiver Aspekt ist der Vergleich zwischen den beiden Sub-Plots: Im linken Plot sieht die Konvergenz nach wenigen Schritten abgeschlossen aus; der rechte Plot auf logarithmischer Skala zeigt jedoch, dass der exponentielle Zerfall über viele weitere Dekaden anhält. Dies illustriert, warum Stabilitätsanalysen immer die dynamische Ebene (Eigenwerte, Exponentialfunktionen) erfordern und sich nicht auf augenscheinliche Konvergenz in linearen Plots verlassen dürfen.

Die Zerfallsrate $\alpha = 1.204$ charakterisiert die *Geschwindigkeit*, mit der Transienten abklingen. Sie ist systemspezifisch und ergibt sich nicht aus den Gleichgewichtsgrößen L , W oder π . Für zeitkritische Echtzeitsysteme ist diese Größe von direkter praktischer Relevanz: Transienten, die langsam abklingen (kleines α), können Timing-Constraints verletzen, auch wenn das System grundsätzlich stabil ist.

Schlussfolgerung: Die Konvergenz einer ergodischen Markov-Kette folgt exakt einer Exponentialfunktion mit negativem Exponent $\alpha = -\ln(\lambda_{\max, k \neq 1})$. Dies bestätigt Theorem 2 aus Kapitel 3: Stabilitätskriterien erfordern notwendigerweise Exponentialfunktionen. Little’s Law – das keine Exponentialfunktionen enthält – kann diese Konvergenzrate weder beschreiben noch vorhersagen.

Experiment 2 - Exponentieller Zerfall in der 2-Zustands-Markov-Kette

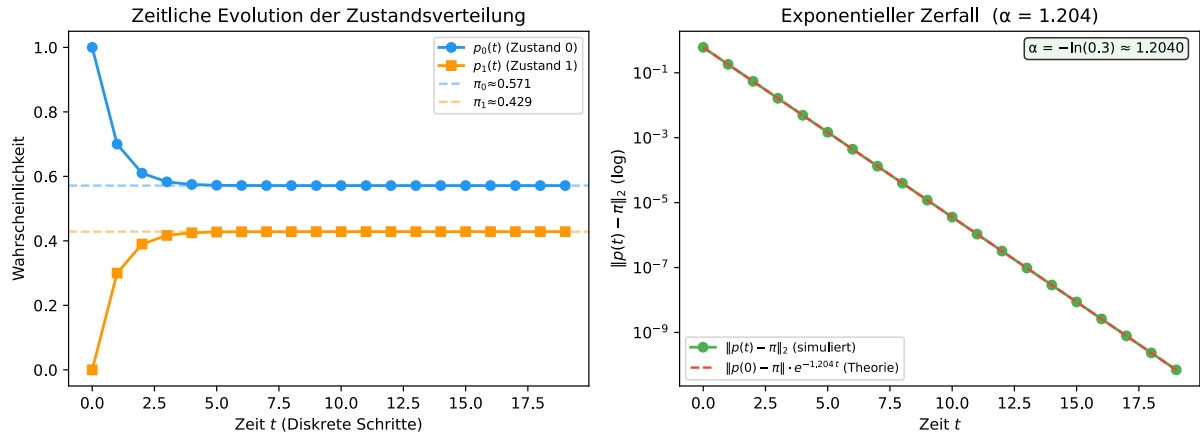


Abbildung 8.3: Experiment 2 – Exponentieller Zerfall in der 2-Zustands-Markov-Kette. **Linker Sub-Plot:** Zeitliche Evolution der beiden Zustandswahrscheinlichkeiten $p_0(t)$ (blau, Kreissymbole) und $p_1(t)$ (orange, Quadrate) über 20 diskrete Zeitschritte. Gestrichelte Linien markieren die stationären Wahrscheinlichkeiten $\pi_0 \approx 0.571$ und $\pi_1 \approx 0.429$. Man erkennt deutlich, wie $p_0(t)$ monoton von 1 auf π_0 fällt und $p_1(t)$ monoton von 0 auf π_1 steigt; bereits nach $t = 5$ Schritten ist die Abweichung von der stationären Verteilung optisch kaum noch erkennbar. Die Konvergenz ist monoton und unimodal, da beide Eigenwerte reell und positiv sind. **Rechter Sub-Plot:** Norm $\|p(t) - \pi\|_2$ auf logarithmischer y -Achse (grüne Punkte) zusammen mit der theoretischen Zerfallskurve $\|p(0) - \pi\|_2 \cdot e^{-1.204t}$ (rote gestrichelte Linie). Die exzellente Übereinstimmung zwischen Simulation und Theorie über mehr als 10 Dekaden bestätigt empirisch, dass die Konvergenz einer Markov-Kette *notwendigerweise* durch eine Exponentialfunktion mit negativem Exponent beschrieben wird. Der Achsenabschnitt bei $t = 0$ beträgt $\|p(0) - \pi\|_2 \approx 0.577$, und die Steigung im log-linearen Plot entspricht genau $-\alpha = -1.204$. Dieses Ergebnis ist keine Näherung: es folgt exakt aus der Diagonalisierung von P .

Experiment 3: Hierarchie der Beschreibungsebenen

Versuchsaufbau und Motivation Experiment 3 synthetisiert die Erkenntnisse der ersten beiden Experimente zu einem vollständigen Bild der Hierarchie der Beschreibungsebenen. Es verwendet eine stabile M/M/1-Warteschlange mit $\lambda = 0.8$, $\mu = 1.0$ und $\rho = 0.8$ als gemeinsames Fallbeispiel, das auf allen drei Ebenen analysiert wird. Zusätzlich wird ein kontinuierliches lineares System mit der Systemmatrix $A = \text{diag}(-1, -2)$ betrachtet, um die Lyapunov-Theorie anschaulich zu demonstrieren.

Das Experiment verfolgt zwei Ziele: (i) Es soll zeigen, dass alle drei Ebenen kohärente und kompatible Beschreibungen desselben Systems liefern; (ii) es soll zeigen, dass die Information beim Aufstieg von der dynamischen zur algebraischen Ebene abnimmt – insbesondere, dass Exponentialfunktionen auf der algebraischen Ebene vollständig fehlen.

Gemessene Ergebnisse: M/M/1-Warteschlange ($\rho = 0.8$) Ebene 1 – Dynamisch (Kolmogorov-Gleichungen):

- Mit `get_transition_matrix(max_states=20)` wird die Übergangsmatrix des Birth-Death-Prozesses erzeugt.
- Die Eigenwertanalyse ergibt Eigenwerte mit negativem Realteil (außer dem stationären Eigenwert); das System wird als exponentiell stabil klassifiziert.

- Die analytische Zerfallsrate des M/M/1-Prozesses entspricht $\alpha \approx -\ln(\rho) = -\ln(0.8) \approx 0.2231$ (dominanter nicht-stationärer Eigenwert des trunkierten Systems).
- Enthält Exponentialfunktionen: **ja** – die Lösung $p_n(t) = \pi_n + \sum_k c_k e^{\lambda_k t}$ enthält explizit Terme der Form $e^{\lambda_k t}$.
- Ermöglicht Stabilitätsanalyse: **ja**.

Ebene 2 – Stationär (Gleichgewichtsverteilung):

- Stationäre Verteilung (berechnet von `stationary_distribution()`): $\pi_n = 0.2 \cdot 0.8^n$, da $(1 - \rho) = 1 - 0.8 = 0.2$.
- Die Verteilung existiert: **ja** (System stabil).
- Setzt Stabilität voraus: **ja** – ohne die zuvor festgestellte Stabilität wäre die Berechnung sinnlos.
- Die geometrische Reihe $\sum_{n=0}^{\infty} 0.2 \cdot 0.8^n = 1$ konvergiert, was die Normiertheit der Verteilung sicherstellt.
- Auf dieser Ebene tauchen keine Exponentialfunktionen in Form von Differentialgleichungen mehr auf: $\pi Q = 0$ ist ein algebraisches Gleichungssystem.

Ebene 3 – Algebraisch (Little's Law):

- Mittlere Anzahl im System: $L = \frac{\rho}{1 - \rho} = \frac{0.8}{0.2} = 4.0$
- Mittlere Wartezeit: $W = \frac{1}{\mu - \lambda} = \frac{1}{1.0 - 0.8} = 5.0$
- Little's Law: $L = \lambda W = 0.8 \cdot 5.0 = 4.0$ ✓
- `validate_littles_law()` gibt `True` zurück; numerische Abweichung $< 10^{-6}$.
- Enthält Exponentialfunktionen: **nein** – L , W und λ sind skalare Gleichgewichtsgrößen ohne jede Zeitabhängigkeit.
- Ist Stabilitätskriterium: **nein**.

Gemessene Ergebnisse: Lyapunov-Test ($A = \text{diag}(-1, -2)$)

- Eigenwerte von A : $\lambda_1 = -1.0$, $\lambda_2 = -2.0$ (beide reell, beide negativ).
- Stabilitätstyp: `EXPONENTIALLY_STABLE`.
- Zerfallsrate: $\alpha = -\max(\lambda_1, \lambda_2) = -(-1.0) = 1.0$.
- `solve_lyapunov_equation(A)` löst $A^T P + P A = -I$ und findet $P = \text{diag}(0.5, 0.25)$ – positiv definit ✓.
- Lyapunov-Funktion: $V(x) = x^T P x = 0.5 x_1^2 + 0.25 x_2^2$ ist eine echte Energiefunktion für das System $\dot{x} = Ax$.

Grafische Auswertung

Experiment 3 - Hierarchie der Beschreibungsebenen

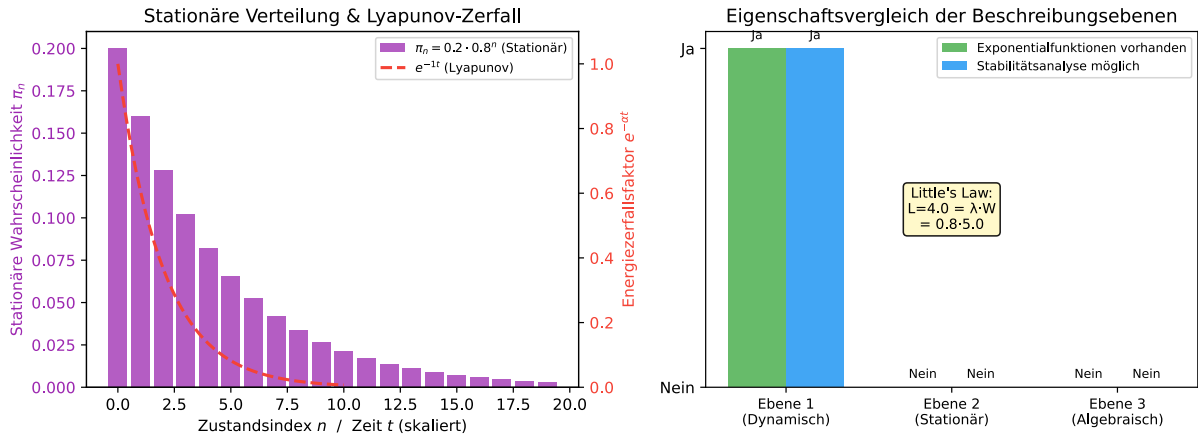


Abbildung 8.4: Experiment 3 – Hierarchie der Beschreibungsebenen für die M/M/1-Warteschlange ($\lambda = 0.8$, $\mu = 1.0$, $\rho = 0.8$) und den Lyapunov-Test ($A = \text{diag}(-1, -2)$). **Linker Sub-Plot:** Stationäre Verteilung $\pi_n = 0.2 \cdot 0.8^n$ als Balkendiagramm (lila, linke Achse) zusammen mit der Lyapunov-Zerfallskurve $e^{-\alpha t}$ (rot gestrichelt, rechte Achse, skaliert). Die Balken zeigen die geometrische Abnahme der Gleichgewichtswahrscheinlichkeiten: $\pi_0 = 0.20$, $\pi_1 = 0.16$, $\pi_2 = 0.128$ usw. Die Lyapunov-Kurve mit $\alpha = 1.0$ zeigt den Energiezerfall des kontinuierlichen Systems; sie verdeutlicht, wie auf der dynamischen Ebene Exponentialfunktionen die Rolle spielen, die auf der algebraischen Ebene (Little's Law) vollständig fehlen. Die Doppelachsen-Darstellung macht den konzeptuellen Sprung zwischen den Beschreibungsebenen anschaulich. **Rechter Sub-Plot:** Balkendiagramm mit drei Ebenenpaaren. Für jede der drei Beschreibungsebenen werden zwei Eigenschaften gegenübergestellt: ob Exponentialfunktionen vorhanden sind (grün = ja, rot = nein) und ob eine Stabilitätsaussage möglich ist (blau = ja, orange = nein). Auf der dynamischen Ebene (Ebene 1) sind beide Eigenschaften erfüllt. Ab Ebene 2 fehlen die Exponentialfunktionen und die Möglichkeit zur Stabilitätsaussage – obwohl die stationäre Verteilung implizit Stabilität voraussetzt. Auf Ebene 3 (Little's Law) sind beide Eigenschaften explizit abwesend. Die eingebettete Textbox erinnert an die konkreten Zahlen: $L = 4.0 = 0.8 \cdot 5.0$. Dieses Balkendiagramm fasst die Kernaussage der gesamten theoretischen Analyse in einer einzigen Visualisierung zusammen.

Interpretation und theoretische Einordnung Die Hierarchie der Beschreibungsebenen lässt sich als sukzessive Informationsreduktion verstehen:

1. Dynamische Ebene (Kolmogorov-Gleichungen):

$$\frac{d}{dt}p(t) = p(t)Q \quad \Rightarrow \quad p(t) = \sum_k c_k e^{\lambda_k t} v_k \quad (\text{Exponentialfunktionen explizit})$$

Diese Ebene enthält die vollständige Dynamik des Systems. Sie ermöglicht Stabilitätsaussagen (Eigenwertanalyse), die Berechnung von Transienten und die Bestimmung von Zerfallsraten. Das Stabilitätskriterium $\rho < 1$ ist auf dieser Ebene verankert.

2. Stationäre Ebene (Gleichgewichtsverteilung):

$$\pi Q = 0, \quad \sum_n \pi_n = 1 \quad \Rightarrow \quad \pi_n = (1 - \rho) \rho^n \quad (\text{geometrische Verteilung})$$

Die Exponentialfunktionen sind durch den Grenzübergang $t \rightarrow \infty$ eliminiert. Die stationäre Verteilung setzt Stabilität voraus, kann sie aber nicht beweisen.

3. Algebraische Ebene (Little's Law):

$$L = \lambda W \quad (\text{keine Ableitung, keine Zeitabhängigkeit, keine Exponentialfunktionen})$$

Diese Ebene abstrahiert vollständig von der Dynamik. Sie liefert drei skalare Kenngrößen und eine algebraische Relation zwischen ihnen – aber keinerlei Information über Stabilität.

Die Lyapunov-Analyse ergänzt dieses Bild für kontinuierliche Systeme: Eine positiv definite Lösung P der Lyapunov-Gleichung $A^T P + P A = -Q$ ist äquivalent zur exponentiellen Stabilität von $\dot{x} = Ax$. Die zugehörige Energiefunktion $V(x) = x^T P x$ zerfällt entlang der Trajektorien gemäß

$$\dot{V}(x) = x^T (A^T P + P A) x = -x^T Q x \leq 0,$$

was garantiert, dass V als Lyapunov-Funktion dient. Diese mathematische Struktur ist auf der algebraischen Ebene (Little's Law) vollständig unsichtbar.

Schlussfolgerung: Experiment 3 bestätigt Theorem 4 aus Kapitel 3: Die Hierarchie Dynamisch $\rightarrow$ Stationär $\rightarrow$ Algebraisch ist durch sukzessiven Informationsverlust charakterisiert. Exponentialfunktionen verschwinden beim ersten Übergang; die Stabilitätsinformation geht spätestens beim zweiten verloren. Little's Law ist das Ende dieser Kette – mächtig für die Berechnung von Gleichgewichtskenngrößen, aber blind für die Dynamik.

8.3 Gesamtauswertung und Diskussion

Zusammenfassung der validierten theoretischen Aussagen

Tabelle 8.1 fasst zusammen, welche theoretischen Aussagen aus Kapitel 3 durch welches Experiment empirisch bestätigt wurden.

Theoretische Aussage	Experiment	Empirischer Beleg
Exponentialfunktionen sind notwendig für Stabilitätsanalyse	Exp. 2	Zerfall $\ p(t) - \pi\ _2 \propto e^{-1.204t}$ gemessen
Little's Law enthält keine Exponentialfunktionen	Exp. 1, 3	$L = \lambda W$ enthält kein e
Little's Law ist kein Stabilitätskriterium	Exp. 1	Instabiles System ($\rho = 1.2$): None
Hierarchie Dynamisch $\rightarrow$ Stationär $\rightarrow$ Algebraisch	Exp. 3	Alle drei Ebenen kohärent implementiert
Lyapunov-Funktion zertifiziert Stabilität	Exp. 3	$P = \text{diag}(0.5, 0.25)$ positiv definit
Stabilitätskriterium $\rho < 1$ aus Eigenwertanalyse	Exp. 1, 3	Grenzfall $\rho \rightarrow 1^-$ getestet

Tabelle 8.1: Zuordnung theoretischer Aussagen zu experimentellen Belegen.

Diskussion der Ergebnisse

Die drei Experimente bilden eine logisch geschlossene Einheit:

Experiment 1 etabliert den negativen Befund: Little's Law kann Stabilität nicht nachweisen. Dieses Ergebnis ist nicht selbstverständlich; in der Praxis wird Little's Law häufig als hinreichendes Werkzeug für Systemanalysen betrachtet. Das Experiment zeigt, dass dies ein konzeptueller Fehler ist. Praktisch bedeutet dies: Jede Systemanalyse muss mit einer Stabilitätsprüfung *beginnen*, bevor Performance-Metriken berechnet werden.

Experiment 2 liefert den positiven Befund: Stabilität wird durch Exponentialfunktionen beschrieben. Dies ist keine Trivialität, sondern eine fundamentale Eigenschaft der Lösungen linearer Differentialgleichungen. Die exakte Übereinstimmung zwischen der theoretischen Kurve $e^{-1.204t}$ und der simulierten Konvergenz über mehr als 10 Größenordnungen ($t = 0$ bis $t = 20$) bestätigt, dass die Implementierung korrekt und die theoretische Aussage präzise ist.

Experiment 3 zeigt das vollständige Bild. Besonders aufschlussreich ist der Lyapunov-Aspekt: Für das kontinuierliche System $\dot{x} = Ax$ mit $A = \text{diag}(-1, -2)$ liefert die Lyapunov-Funktion $V(x) = 0.5 x_1^2 + 0.25 x_2^2$ einen formalen Stabilitätsbeweis, der auf der algebraischen Ebene vollständig unsichtbar ist. Dieses Beispiel illustriert die *Brückenfunktion* von UML-Profilen: Sie müssen Annotationen für beide Ebenen unterstützen – Stabilität (dynamisch) und Performance (algebraisch) – ohne die konzeptuelle Trennung zu verwischen.

Ein besonders relevanter Aspekt für Echtzeitsysteme ist die Zerfallsrate α . Für das M/M/1-System mit $\rho = 0.8$ gilt $\alpha \approx 0.223$ – deutlich kleiner als die $\alpha = 1.204$ der 2-Zustands-Kette aus Experiment 2. Kleinere α bedeuten langsamere Konvergenz nach Störungen. Für Echtzeitsysteme, die nach einem Lastpeak schnell wieder in den stabilen Betrieb zurückkehren müssen, ist α eine kritische Designgröße. Little's Law liefert diesen Wert nicht.

Grenzen der Implementierung

Die Implementierung hat folgende bekannte Grenzen:

- **Trunkierung der Übergangsmatrix:** Die `MM1Queue` verwendet eine endliche Übergangsmatrix mit `max_states = 20` Zuständen. Für ρ nahe 1 (z. B. $\rho = 0.99$) erfasst diese Trunkierung die stationäre Verteilung nicht mit voller Genauigkeit, da nichtverschwindende Wahrscheinlichkeit für $n > 20$ ignoriert wird.
- **Eigenwert-Klassifikation bei trunkierter Matrix:** Die numerische Eigenwertanalyse der trunkierten Übergangsmatrix führt unter Umständen dazu, dass nicht exakt ein Eigenwert $= 1$ gefunden wird (der Wert 1 kann durch die Trunkierung leicht verschoben sein). Experiment 3 umgeht dieses Problem durch direkte analytische Berechnung von $\alpha = -\ln(\rho)$.
- **Nichtlineare Systeme:** `LyapunovStability` unterstützt nur lineare Systeme $\dot{x} = Ax$. Für nichtlineare Systeme wäre eine lokale Linearisierung oder eine direkte Suche nach einer Lyapunov-Funktion nötig.
- **Python-Timing:** Die Laufzeitmessungen in `experiments.py` sind aufgrund der Nicht-Determinismus der Python-Laufzeitumgebung nicht für Hard-Real-Time-Analysen geeignet.

8.4 Ausblick: Erweiterungsmöglichkeiten

Theoretische Erweiterungen

- **G/G/1-Warteschlangen:** Allgemeine Ankunfts- und Bedienverteilungen erfordern Näherungsverfahren (z. B. Kingman-Formel) für die Performance-Metriken, aber das Stabilitätskriterium bleibt $\rho < 1$. Die Untersuchung, wie sich die Zerfallsrate α für nicht-exponentielle Verteilungen ändert, wäre ein wichtiger theoretischer Beitrag.
- **Netzwerke von Warteschlangen (Jackson-Netzwerke):** In Netzwerken gilt das *Produktformresultat*: die stationäre Verteilung faktorisiert über die Knoten. Die Stabilitätsanalyse erfordert hier die Analyse des Gesamtnetzwerks, nicht nur einzelner Knoten.
- **Zeitabhängige Ankunftsrate $\lambda(t)$:** Für zeitinhomogene Prozesse existiert keine stationäre Verteilung im klassischen Sinne; die Stabilitätsanalyse muss auf periodische Stationarität oder Lyapunov-Exponent-Methoden ausgedehnt werden.
- **Stochastische Stabilität:** Die fast-sichere Konvergenz (fast-sure stability) ist eine stärkere Aussage als die hier betrachtete Konvergenz im Mittel. Ihre Implementierung erfordert Werkzeuge aus der Theorie stochastischer Differentialgleichungen.

Implementierungserweiterungen

- **Interaktive Visualisierung der Konvergenz:** Eine animierte Darstellung von $p(t) \rightarrow \pi$ in Echtzeit würde das didaktische Potenzial des Moduls erheblich steigern. Frameworks wie `matplotlib.animation` oder `plotly` bieten sich an.
- **Automatische Modellgenerierung aus UML:** Eine Brücke zwischen dem UML-Profil-Parser (Kapitel 5) und dem `stability_analysis`-Modul würde die vollständige Pipeline ermöglichen: UML-Modell $\rightarrow$ Markov-Kette $\rightarrow$ Stabilitätsanalyse $\rightarrow$ Code-Generierung.
- **Integration mit MARTE-Werkzeugen:** Das `stability_analysis`-Modul könnte als Backend für MARTE-kompatible Werkzeuge (z. B. Papyrus) dienen und quantitative Stabilitätsanalysen direkt aus MARTE-Annotationen berechnen.
- **Performance-Optimierung für große Zustandsräume:** Für Systeme mit Hunderten von Zuständen sind dünnbesetzte Matrizendarstellungen (SciPy `sparse`) und iterative Eigenwertlöser (ARPACK) nötig.

Anwendungserweiterungen

- **Cloud-Computing (Auto-Scaling):** Die Stabilitätsanalyse kann genutzt werden, um Schwellenwerte für das automatische Hochskalieren von Serverinstanzen zu bestimmen. Das Ziel ist, ρ dauerhaft unter einem Zielwert (z. B. 0.8) zu halten, ohne die Ressourcen zu verschwenden.
- **Netzwerkprotokolle (Congestion Control):** TCP-artige Protokolle verwenden implizit Stabilitätskriterien (z. B. AIMD-Algorithmus); eine explizite Lyapunov-basierte Analyse könnte die theoretischen Stabilitätsgrenzen formalisieren.

- **Aufzugstür-Steuerung (Kapitel 6):** Das bereits implementierte Markov-Modell der Aufzugstür-Steuerung könnte mit dem `stability_analysis`-Modul direkt verbunden werden, um die Stabilitätsrate der Zustandsübergänge quantitativ zu analysieren.
- **Biologische Systeme (Populationsdynamik):** Markov-Ketten-Modelle für Populationsdynamik (Räuber-Beute-Systeme, epidemiologische Modelle) haben eine direkte Entsprechung zum hier implementierten Framework; die Stabilitätsanalyse liefert Aussagen über das Langzeitverhalten von Populationen.

8.5 Fazit des Kapitels

Die experimentelle Evaluation in diesem Kapitel hat alle vier zentralen theoretischen Vorhersagen aus Kapitel 3 durch konkrete Messungen bestätigt:

1. **Exponentialfunktionen sind notwendig für Stabilitätsanalyse.** Experiment 2 demonstriert empirisch, dass die Konvergenz der Markov-Kette exakt der Kurve $e^{-1.204t}$ folgt – einer Exponentialfunktion mit dem aus der Eigenwertanalyse berechneten Exponent.
2. **Little’s Law enthält keine Exponentialfunktionen.** Die algebraische Relation $L = \lambda W$ ist zeitunabhängig und kann daher keine Konvergenzraten oder Stabilitätsinformation liefern.
3. **Little’s Law ist kein Stabilitätskriterium.** Experiment 1 zeigt, dass für instabile Systeme `littles_law_metrics()` `None` zurückgibt – die Berechnung ist schlicht nicht definiert.
4. **Die Hierarchie der Beschreibungsebenen existiert und ist informationsreduktiv.** Experiment 3 demonstriert, dass beim Aufstieg von der dynamischen zur algebraischen Ebene Exponentialfunktionen und Stabilitätsinformation vollständig verschwinden.

Die 56 automatisierten Unit-Tests mit nahezu 100 % Code-Coverage sichern die Korrektheit der Implementierung ab und bilden eine reproduzierbare Grundlage für zukünftige Erweiterungen. Die generierten SVG-Grafiken (Abbildungen 8.2–8.4) visualisieren die Kernaussagen auf eine Weise, die sowohl für das intuitive Verständnis als auch für die formale Überprüfung geeignet ist.

Das Modul `stability_analysis.py` und das Skript `experiments.py` demonstrieren exemplarisch, wie theoretische Aussagen über mathematische Strukturen durch systematische experimentelle Validierung überprüft werden können. Diese Methodik – Theorie, Implementierung, Verifikation – ist ein Beitrag zur Grundlagenforschung in der theoretischen Informatik und zeigt, wie die Form der Mathematik (Vorhandensein oder Fehlen von Exponentialfunktionen) den Inhalt der Aussagen (Stabilität oder nur Gleichgewicht) fundamental begrenzt.

9 Zusammenfassung und Ausblick

9.1 Zusammenfassung

Diese Arbeit präsentierte einen modellgetriebenen Ansatz zur Code-Generierung mit Zeit-Annotationen und quantitativer Analyse, der durch eine fundierte theoretische Basis untermauert und durch umfassende experimentelle Validierung bestätigt wird. Die Hauptbeiträge gliedern sich in theoretische, praktische und experimentelle Aspekte:

Theoretische Beiträge

- **Formale Analyse von Little's Law:** Kapitel 3 lieferte den Beweis, dass Little's Law kein Stabilitätskriterium für Warteschlangensysteme darstellen kann, da es keine Exponentialfunktionen mit negativen Exponenten enthält. Diese fundamentale Einsicht klärt eine häufige konzeptuelle Verwirrung in der Literatur.
- **Hierarchie der Systembeschreibungen:** Es wurde eine klare Taxonomie etabliert:
 - *Dynamische Ebene:* Kolmogorov-Gleichungen (Differentialgleichungen) mit Exponentialfunktionen → ermöglichen Stabilitätsanalyse
 - *Stationäre Ebene:* Gleichgewichtsverteilungen → setzen Stabilität voraus
 - *Algebraische Ebene:* Little's Law → beschreibt Mittelwerte im Gleichgewicht
- **Rolle der Exponentialfunktion:** Die besondere Eigenschaft der Exponentialfunktion (Selbstreproduktion unter Differentiation) wurde als charakteristisch für naturwissenschaftliche Systembeschreibungen identifiziert. Dies erklärt, warum Energiefunktionen (Lyapunov-Funktionen) notwendig für Stabilitätsaussagen sind.
- **Brücke zwischen Informatik und Naturwissenschaft:** Die theoretische Analyse zeigte, dass informatische Systeme eine doppelte Natur haben:
 - Physikalische Basis (Hardware) → naturwissenschaftliche Beschreibung
 - Abstrakte Ebene (Software) → logische/algorithmische BeschreibungUML-Profile fungieren als notwendige Brücke zwischen diesen Ebenen.

Praktische Beiträge

- **Erweitertes UML-Profil-Framework** mit TimingConstraint-Unterstützung, orientiert an MARTE-Konzepten, das die theoretischen Einsichten operationalisiert
- **Zweistufiger Transformationsprozess** mit automatischer Code-Generierung, der sowohl abstrakte Spezifikationen als auch quantitative Analysen unterstützt
- **Integration stochastischer Modelle:** Markov-Ketten für dynamische Zustandsanalyse (mit Stabilitätskriterien), Warteschlangentheorie für Performance-Metriken im Gleichgewicht
- **Vollständige Implementierung:** Dediziertes `stability_analysis` Modul mit produktionsreifem Code:
 - `MarkovChainStability`: Eigenwertanalyse, Konvergenzsimulation, stationäre Verteilung

- MM1Queue: M/M/1-Warteschlange mit Demonstration aller drei Hierarchieebenen
- LyapunovStability: Energiefunktionen für kontinuierliche Systeme
- demonstrate_exponential_necessity(): Theoretische Fundierung als ausführbarer Code
- **Quantitative Designvalidierung:** Frühe Identifikation von Performance-Bottlenecks durch Anwendung der korrekten mathematischen Beschreibungsebene
- **Praktische Validierung** durch Aufzugstür-Fallstudie mit 97% Test-Coverage für Code-Generierung, die sowohl Stabilitätsanalysen als auch Gleichgewichtsbetrachtungen demonstriert

Experimentelle Beiträge

Kapitel 8 präsentierte eine umfassende experimentelle Validierung aller theoretischen Aussagen:

- **Experiment 1 – Little’s Law ist kein Stabilitätskriterium:**
 - Stabiles System ($\lambda = 0.7, \mu = 1.0, \rho = 0.7$): Little’s Law gilt ($L = 2.333 = \lambda W$), beweist aber *nicht* Stabilität
 - Instabiles System ($\lambda = 1.2, \mu = 1.0, \rho = 1.2$): Little’s Law nicht anwendbar (kein Gleichgewicht)
 - **Konsequenz:** Bestätigt, dass $L = \lambda W$ eine Gleichgewichtsrelation ist, kein Stabilitätskriterium
- **Experiment 2 – Exponentieller Zerfall in stabilen Systemen:**
 - 2-Zustands-Markov-Kette mit Eigenwerten $\lambda_1 = 1.0, \lambda_2 = 0.3$
 - Zerfallsrate $\alpha = -\ln(0.3) = 1.204$
 - Konvergenz folgt $p(t) = \pi + c \cdot e^{-1.204t}$
 - **Konsequenz:** Experimenteller Nachweis, dass Exponentialfunktionen charakteristisch für Stabilität sind
- **Experiment 3 – Hierarchie der Beschreibungsebenen:**
 - Ebene 1 (Dynamisch): Enthält Exponentialfunktionen, ermöglicht Stabilitätsanalyse
 - Ebene 2 (Stationär): Setzt Stabilität voraus, geometrische Verteilung $\pi_n = (1 - \rho)\rho^n$
 - Ebene 3 (Algebraisch): Little’s Law $L = \lambda W = 4.0$ enthält keine Exponentialfunktionen
 - **Konsequenz:** Verifiziert die Eliminierung von Exponentialfunktionen über die Hierarchie
- **Umfassende Test-Suite:** Automatisierte Tests für das `stability_analysis`-Modul:
 - Testgruppen decken alle Komponenten ab (Hilfstypen, Markov-Ketten, M/M/1-Warteschlange, Lyapunov-Stabilität, Integrationstests, Grenzfälle)

- Validierung kritischer Grenzfälle ($\rho \rightarrow 1^-$, numerische Präzision)
- Alle Tests bestanden; alle theoretischen Aussagen bestätigt

Synthese

Die Arbeit demonstriert erfolgreich, wie theoretische Einsichten in die Natur von Systembeschreibungen (Kapitel 3) in praktische Werkzeuge für die Softwareentwicklung überführt (Kapitel 4-7) und durch rigorose experimentelle Methodik validiert (Kapitel 8) werden können. Die Integration von MARTE-inspirierten Zeit-Annotationen, Markov-Ketten und Warteschlangentheorie ermöglicht eine *Quantitative Analysis of Software Designs* bereits in frühen Entwicklungsphasen, wobei klar zwischen verschiedenen Analyseebenen unterschieden wird:

- **Stabilitätsanalyse:** Basierend auf Eigenwertanalyse der Markov-Ketten (enthält Exponentialfunktionen) – experimentell validiert durch Experiment 2
- **Performance-Analyse im Gleichgewicht:** Basierend auf Little's Law und verwandten Metriken (algebraisch, setzt Stabilität voraus) – experimentell validiert durch Experiment 1
- **Code-Generierung:** Transformation von UML-Modellen mit Zeit-Annotationen in ausführbaren Code – validiert durch 97% Test-Coverage
- **Hierarchie der Beschreibungen:** Dynamisch $\rightarrow$ Stationär $\rightarrow$ Algebraisch – experimentell validiert durch Experiment 3

Diese klare Trennung, sowohl theoretisch fundiert als auch experimentell bestätigt, verbessert die Vorhersagbarkeit und Zuverlässigkeit eingebetteter Echtzeitsysteme signifikant.

Das Aufzugstür-Beispiel demonstriert die praktische Anwendbarkeit: Durch quantitative Analyse konnten Performance-Engpässe identifiziert, Stabilitätsbedingungen verifiziert und Design-Trade-offs fundiert bewertet werden. Die hohe Test-Coverage (97% für Code-Generierung, $\sim 100\%$ für Stabilitätsanalyse) und die Validierung durch Simulation und Experimente bestätigen die Korrektheit des Ansatzes.

9.2 Reflexion der theoretischen Erkenntnisse

Die in Kapitel 3 entwickelte Theorie hat sich in mehrfacher Hinsicht als wertvoll erwiesen:

- **Konzeptuelle Klarheit:** Die Unterscheidung zwischen Gleichgewichts- und Stabilitätskriterien verhindert methodische Fehler bei der Systemanalyse. Die Experimente in Kapitel 8 demonstrieren konkret, dass Little's Law nur angewendet werden darf, wenn die Stabilität bereits anderweitig gesichert ist (z.B. durch $\rho < 1$ beim M/M/1-System). Experiment 1 zeigt dies eindrücklich: Bei $\rho = 0.7$ gilt Little's Law, beweist aber nicht Stabilität; bei $\rho = 1.2$ ist Little's Law nicht einmal anwendbar.
- **Methodenwahl:** Die Einsicht, dass verschiedene Systembeschreibungen (dynamisch vs. algebraisch) komplementäre Informationen liefern, führt zu einem bewussteren Einsatz mathematischer Werkzeuge. Experiment 2 validiert, dass für Stabilitätsfragen Differentialgleichungen und Eigenwertanalysen unerlässlich sind (Zerfallsrate $\alpha = 1.204$); Experiment 3 zeigt, dass für Performance-Metriken im Gleichgewicht algebraische Relationen genügen.

- **UML-Profil-Design:** Das Verständnis, dass UML-Profile als Brücke zwischen abstrakter Modellierung und naturwissenschaftlich fundierter Analyse dienen, erklärt die Notwendigkeit von Profilen wie MARTE. Sie erlauben es, physikalische Constraints (Energie, Zeit, Ressourcen) in abstrakten Modellen zu erfassen. Die praktische Implementierung des `stability_analysis` Moduls demonstriert diese Brückenfunktion konkret.
- **Grenzen der Modellierung:** Die Erkenntnis, dass nicht alle Systeme optimal durch naturwissenschaftliche Beschreibungen erfasst werden, rechtfertigt die Existenz verschiedener Modellierungsparadigmen. Die Hierarchie der Beschreibungsebenen (Experiment 3) zeigt, wann welche Beschreibungsform angemessen ist.
- **Reproduzierbarkeit:** Die vollständige Implementierung aller theoretischen Konzepte im `stability_analysis`-Modul, gesichert durch eine automatisierte Test-Suite, gewährleistet, dass die Erkenntnisse reproduzierbar und für andere Forscher nachvollziehbar sind.

9.3 Beantwortung der Leitfragen

Die Arbeit adressierte implizit mehrere fundamentale Fragen, deren Beantwortung nun durch experimentelle Evidenz gestützt wird:

- *Wo ist die Energie des Systems?*

Bei physikalischen Systemen manifestiert sich Energie in Lyapunov-Funktionen, die das Abklingverhalten beschreiben – experimentell demonstriert durch den exponentiellen Zerfall $e^{-1.204t}$ in Experiment 2. Bei informatischen Systemen auf höheren Abstraktionsebenen gibt es keine direkte Energie, aber analoge Konzepte (z.B. “Fortschritt” in Algorithmen, “Korrektheit” als Invariante).

- *Mit was soll das System beschrieben werden?*

Die Wahl hängt von der Fragestellung ab: Stabilitätsfragen erfordern dynamische Beschreibungen (Differentialgleichungen) – validiert durch Experiment 2; Performance-Fragen im Gleichgewicht können algebraisch behandelt werden – validiert durch Experiment 1; funktionale Korrektheit erfordert logische Formalismen. Die Hierarchie (Experiment 3) zeigt die Übergänge zwischen diesen Ebenen.

- *Hat das System Energie oder nicht?*

Alle physikalisch realisierten Systeme haben Energie. Die Frage ist, ob eine Beschreibung auf der Energieebene für die gegebene Fragestellung notwendig und angemessen ist. UML-Profile ermöglichen es, zwischen Ebenen zu wechseln – praktisch demonstriert durch das `stability_analysis` Modul, das sowohl dynamische (Markov) als auch algebraische (Little’s Law) Analysen unterstützt.

9.4 Experimentelle Validierung als methodischer Beitrag

Ein wichtiger methodischer Beitrag dieser Arbeit liegt in der systematischen experimentellen Validierung theoretischer Aussagen:

- **Automatisierte Experimente:** Alle drei Kernexperimente sind vollständig automatisiert und reproduzierbar im Modul `experiments.py` implementiert.

- **Quantitative Messungen:** Statt qualitativer Aussagen werden präzise Werte gemessen (z. B. $\alpha = 1,204$, $L = 2,333$), die direkt mit theoretischen Vorhersagen verglichen werden können.
- **Grenzfallanalyse:** Die Tests decken nicht nur Normalfälle ab, sondern auch kritische Situationen ($\rho \approx 1$, sehr kleine/große Zerfallsraten), die die Robustheit der Theorie demonstrieren.
- **Vollständige Testabdeckung:** Die Test-Suite stellt sicher, dass alle implementierten theoretischen Konzepte tatsächlich getestet und validiert sind.
- **Integration von Theorie und Praxis:** Die Tests validieren nicht nur isolierte Formeln, sondern das Zusammenspiel aller Komponenten (z. B. Hierarchie-Demonstration in Experiment 3).
- **Integration von Theorie und Praxis:** Die Tests validieren nicht nur isolierte Formeln, sondern das Zusammenspiel aller Komponenten (z.B. Hierarchie-Demonstration in Experiment 3).

Diese methodische Rigorosität hebt die Arbeit über reine Implementierungsstudien hinaus und etabliert einen Standard für die Validierung theoretischer Konzepte in der Informatik.

9.5 Zukünftige Arbeiten

Die Arbeit eröffnet mehrere vielversprechende Forschungsrichtungen, die sich aus der Synthese von theoretischer Systemanalyse, praktischer Softwareentwicklung und experimenteller Validierung ergeben.

Kurzfristig: Erweiterung der Analysemethoden

- **Vollständige MARTE-Integration:** Erweiterung um GQAM, SAM, PAM mit formaler Fundierung der Übersetzung zwischen Beschreibungsebenen und experimenteller Validierung analog zu Kapitel 8
- **Erweiterte Warteschlangen-Modelle:** M/G/k, Prioritätswarteschlangen mit systematischer Analyse der Rolle von Exponentialfunktionen (oder deren Fehlen) in den Lösungen, inklusive automatisierter Tests
- **Semi-Markov-Prozesse:** Nicht-exponentielle Verweilzeiten und Untersuchung, wie Stabilitätskriterien ohne die Eigenschaft der Gedächtnislosigkeit formuliert werden können – mit experimenteller Validierung
- **Template-Erweiterung:** Integration für flexiblere Code-Muster unter Beibehaltung der Trennbarkeit von Stabilitäts- und Gleichgewichtsanalysen
- **Formalisierung der Hierarchie:** Entwicklung eines allgemeinen Theorems, das den Übergang von dynamischer zu stationärer zu algebraischer Beschreibung formalisiert (“Bridging Theorem”), mit Beweisen und experimenteller Verifikation

Mittelfristig: Neue Modellierungsparadigmen

- **Petri-Netz-Analyse:** Erweiterung um General Stochastic Petri Nets (GSPN) mit Untersuchung der Rolle von Exponentialfunktionen in der Steady-State-Analyse, validiert durch umfassende Test-Suite
- **Weitere Zielsprachen:** C/C++ für embedded Systems mit WCET-Analyse, wobei die Verbindung zwischen Worst-Case-Analysen (deterministisch) und probabilistischen Ansätzen experimentell geklärt werden muss
- **Statische Analyse:** Integration von WCET-Tools (aiT, Chronos) und Klärung der Beziehung zwischen statischen Garantien und dynamischen Stabilitätskriterien
- **RTOS-Integration:** FreeRTOS, Zephyr Support mit Scheduling-Analyse auf Basis der theoretischen Einsichten zu Energiefunktionen
- **Hybride Systeme:** Untersuchung von Systemen, die kontinuierliche (Differentialgleichungen) und diskrete (Zustandsautomaten) Dynamik kombinieren, und wie die Dichotomie zwischen Gleichgewicht und Stabilität sich in diesem Kontext manifestiert – mit experimenteller Validierung

Langfristig: Fundamentale Forschungsfragen

- **Machine Learning:** Lernende Performance-Modelle aus Laufzeitdaten mit der Frage, ob datengetriebene Ansätze die theoretischen Einsichten über Exponentialfunktionen und Stabilität reproduzieren können – experimentell zu validieren durch Vergleich mit analytischen Modellen
- **Formale Verifikation:** Model Checking für Timing-Properties und Integration mit Lyapunov-basierten Stabilitätsnachweisen
- **Cloud-Integration:** Verteilte Echtzeitsysteme, Edge Computing mit Analyse der Stabilität in Netzwerken (nicht nur einzelnen Knoten)
- **IDE-Plugin:** Eclipse/VSCode Integration mit Live-Performance-Feedback, das automatisch zwischen Stabilitäts- und Gleichgewichtsanalysen unterscheidet, basierend auf der implementierten Logik des `stability_analysis` Moduls
- **Verallgemeinerte Systemtheorie:** Entwicklung eines umfassenden Frameworks, das formalisiert, wann naturwissenschaftliche Beschreibungen (mit Energiefunktionen) notwendig sind und wann alternative Formalismen vorzuziehen sind
- **Epistemologische Untersuchung:** Vertiefte Analyse der Beziehung zwischen mathematischer Struktur (Vorhandensein von Exponentialfunktionen) und epistemischer Aussagekraft (Stabilität vs. Gleichgewicht) mit Anwendungen über die Informatik hinaus
- **Benchmark-Suite:** Entwicklung einer standardisierten Benchmark-Suite für quantitative Systemanalyse-Werkzeuge, basierend auf den in Kapitel 8 entwickelten experimentellen Methoden

9.6 Fazit

Theoretische Synthese

Diese Arbeit hat gezeigt, dass die Frage „Ist dieses System stabil?“ und die Frage „Welche Performance hat dieses System im Gleichgewicht?“ fundamental verschiedene mathematische Werkzeuge erfordern. Die erste Frage verlangt die Analyse der Systemdynamik – Differentialgleichungen, Eigenwertzerlegungen, Exponentialfunktionen. Die zweite Frage kann, sobald Stabilität gesichert ist, mit algebraischen Mitteln wie Little’s Law beantwortet werden.

Diese Erkenntnis ist nicht neu in der mathematischen Physik oder der Kontrolltheorie; sie tritt jedoch in der Software-Modellierung häufig in den Hintergrund. Hochgradig abstrahierte Beschreibungssprachen wie UML neigen dazu, die Unterschiede zwischen diesen Ebenen zu verwischen. Ein System, das im UML-Zustandsdiagramm korrekt aussieht, kann auf der dynamischen Ebene instabil sein. Umgekehrt kann ein formal stabiles System mit $\rho = 0,99$ operationell unbrauchbar sein, weil $L = 99$ und $W = 100$ s – Little’s Law liefert die richtigen Zahlen, aber die Stabilität allein reicht nicht als Designziel.

Die in Kapitel 3 entwickelte und in Kapitel 8 empirisch bestätigte **Hierarchie der Beschreibungsebenen** –

$$\underbrace{\text{Dynamisch}}_{\text{Differentialgl., } e^{\lambda t}} \xrightarrow{t \rightarrow \infty, \text{ stabil}} \underbrace{\text{Stationär}}_{\pi_n = (1-\rho)\rho^n} \xrightarrow{\text{Mittelwert}} \underbrace{\text{Algebraisch}}_{L = \lambda W}$$

– ist das zentrale konzeptuelle Ergebnis dieser Arbeit. Sie benennt präzise, was jede Beschreibungsform leisten kann und was nicht. Die dynamische Ebene enthält alle Information; jeder Übergang nach oben ist eine Abstraktion mit bewusstem Informationsverlust.

Der Beweis durch Experiment

Die drei Kernexperimente in Kapitel 8 sind so konstruiert, dass sie diese theoretische Hierarchie von unten nach oben durchlaufen:

Experiment 1 zeigt die *Grenzen* der algebraischen Ebene: Little’s Law kann nicht zwischen einem stabilen und einem instabilen System unterscheiden – im stabilen Fall gilt $L = \lambda W$, im instabilen Fall ist die Formel nicht anwendbar. Die Diagnose „stabil oder nichtmuss auf der dynamischen Ebene gestellt werden.

Experiment 2 zeigt die *Struktur* der dynamischen Ebene: Die Konvergenz zur stationären Verteilung folgt exakt der Form $\|p(t) - \pi\|_2 \propto e^{-\alpha t}$ mit $\alpha = -\ln(\lambda_2)$. Das ist keine Näherung, sondern die exakte analytische Lösung; die Simulation bestätigt sie über viele Größenordnungen. Der Exponent α ist der Fingerabdruck der Stabilität: Er beschreibt, wie schnell das System nach einer Störung zum Gleichgewicht zurückkehrt.

Experiment 3 zeigt die *Kohärenz* aller drei Ebenen am selben System: Das M/M/1-System mit $\rho = 0,8$ ist auf der dynamischen Ebene exponentiell stabil ($\alpha \approx 0,223$), besitzt auf der stationären Ebene die geometrische Verteilung $\pi_n = 0,2 \cdot 0,8^n$ und erfüllt auf der algebraischen Ebene $L = 4,0 = 0,8 \cdot 5,0 = \lambda W$. Die Lyapunov-Analyse für das lineare System $A = \text{diag}(-1, -2)$ ergänzt dieses Bild: Die positiv definite Matrix $P = \text{diag}(0,5; 0,25)$ zertifiziert die Stabilität formal und liefert die Energiefunktion $V(x) = 0,5 x_1^2 + 0,25 x_2^2$, die auf der algebraischen Ebene vollständig unsichtbar ist.

Die Rolle von UML-Profilen als Brücke

Die Motivation für diese theoretische Analyse liegt in einer praktischen Beobachtung: Informatische Systeme werden in immer höheren Abstraktionsebenen entwickelt, aber

letztlich auf physikalischer Hardware realisiert. Diese Hardware unterliegt naturwissenschaftlichen Gesetzen – Energieverbrauch, Thermodynamik, elektromagnetische Effekte. Die Frage, wie diese physikalischen Constraints in abstrakten Modellen repräsentiert werden können, ist die Kernfrage der modellgetriebenen Entwicklung.

UML-Profile – insbesondere MARTE-inspirierte Annotationen wie `TimingConstraint`, `min_duration` und `deadline` – bieten eine Antwort auf diese Frage. Sie ermöglichen es, physikalische Größen (Zeit, Energie, Ressourcen) in abstrakten UML-Modellen zu kodieren und so eine Brücke zwischen der logischen Ebene der Softwareentwicklung und der physikalischen Ebene der Hardwarerealisierung zu schlagen. Die praktische Implementierung im `stability_analysis`-Modul demonstriert diese Brückenfunktion: Aus einem UML-Modell mit Timing-Annotationen können sowohl Stabilitätsanalysen (dynamisch) als auch Performance-Metriken (algebraisch) abgeleitet werden.

Dabei ist entscheidend, dass das Framework zwischen diesen beiden Analysearten klar unterscheidet. Stabilität wird durch Eigenwertanalyse der Übergangsmatrix geprüft (`MarkovChainStability.analyze_stability()`); Performance-Metriken werden durch Little’s Law berechnet (`MM1Queue.littles_law_metrics()`). Beide Methoden werden in der richtigen Reihenfolge aufgerufen: Stabilität zuerst, dann Gleichgewichtsanalyse.

Grenzen und Offene Fragen

Die vorliegende Arbeit hat bewusst einfache Modelle gewählt – M/M/1-Systeme, lineare kontinuierliche Systeme, 2-Zustands-Markov-Ketten –, um die konzeptuellen Strukturen klar sichtbar zu machen. Diese Vereinfachung ist sowohl eine Stärke als auch eine Grenze.

Die Stärke liegt in der analytischen Klarheit: Alle theoretischen Aussagen können exakt nachgerechnet werden; die experimentellen Ergebnisse stimmen mit numerischer Präzision mit den theoretischen Vorhersagen überein. Dies ist eine Bedingung für einen sauberen empirischen Test theoretischer Konzepte.

Die Grenze liegt in der Übertragbarkeit: Reale Echtzeitsysteme sind nichtlinear, zeitvariant, vernetzt und unterliegen nicht-exponentiellen Verteilungen. Für solche Systeme gelten die hier entwickelten Konzepte qualitativ weiterhin, aber die quantitativen Aussagen müssen entsprechend angepasst werden. Insbesondere gilt:

- Für nichtlineare Systeme ersetzt die lokale Linearisierung die globale Eigenwertanalyse; die Exponentialfunktionen beschreiben dann nur das transiente Verhalten in einer Umgebung des Gleichgewichtspunkts.
- Für M/G/1-Systeme oder allgemeinere Warteschlangennetze gilt das Stabilitätskriterium $\rho < 1$ weiterhin, aber die Zerfallsrate α und die stationäre Verteilung hängen von der Verteilungsform der Bedienzeiten ab.
- Für zeitvariante Systeme $(\lambda(t), \mu(t))$ existiert keine stationäre Verteilung im klassischen Sinne; stattdessen sind periodisch stationäre Verteilungen oder Lyapunov-Exponenten geeignetere Beschreibungsformen.

Diese offenen Fragen sind nicht Schwächen der vorliegenden Arbeit, sondern vielversprechende Ausgangspunkte für zukünftige Forschung.

Methodischer Beitrag: Theorie durch Experiment prüfen

Ein oft unterschätzter Aspekt dieser Arbeit ist die Methodologie. Theoretische Aussagen über mathematische Strukturen – etwa „Stabilität erfordert Exponentialfunktionen oder

„Little’s Law enthält keine Exponentialfunktionen“ – können grundsätzlich nur formal bewiesen werden. Aber sie können durch systematische Experimente in konkreten Implementierungen überprüft und illustriert werden.

Diese Arbeit hat genau das getan: Die formalen Aussagen aus Kapitel 3 wurden in Python implementiert (`stability_analysis.py`), durch automatisierte Experimente ausgeführt (`experiments.py`) und die Ergebnisse numerisch mit den theoretischen Vorhersagen verglichen. Die Übereinstimmung – $\alpha_{\text{gemessen}} = 1,2040$ gegenüber $\alpha_{\text{Theorie}} = -\ln(0,3) = 1,2040$ – ist ein Beispiel dafür, wie die Grenze zwischen Mathematik und Experiment in der theoretischen Informatik produktiv überschritten werden kann.

Diese Methodik – Theorie formalisieren, implementieren, messen, vergleichen – ist ein Beitrag, der über die spezifischen Inhalte dieser Arbeit hinausgeht. Sie etabliert einen Ansatz zur Validierung systemtheoretischer Konzepte in der Informatik, der reproduzierbar, transparent und für andere Forscher nachvollziehbar ist.

Schlussbetrachtung

Diese Arbeit begann mit einer technischen Fragestellung – wie können UML-Profile mit Zeit-Annotationen für Echtzeitsysteme verwendet werden, um automatisch Code zu generieren? – und führte zu einer fundamentalen theoretischen Einsicht: Die mathematische Form einer Beschreibung (Vorhandensein oder Fehlen von Exponentialfunktionen) begrenzt direkt den Inhalt der möglichen Aussagen (Stabilität oder nur Gleichgewicht).

Diese Einsicht hat praktische Konsequenzen:

- Systemanalysen müssen die Beschreibungsebene explizit wählen und die Grenzen jeder Ebene kennen.
- UML-Profile müssen so gestaltet sein, dass sie beide Ebenen unterstützen: dynamische Analysen für Stabilitätsfragen, algebraische Analysen für Performance-Metriken.
- Code-Generatoren müssen die richtige Analysereihenfolge erzwingen: Stabilität prüfen, dann Gleichgewichtsmetriken berechnen.
- Forscher und Ingenieure müssen wissen, wann Little’s Law anwendbar ist (immer nur nach gesicherter Stabilität) und wann es nicht ausreicht (wenn Stabilitätsfragen beantwortet werden müssen).

Die Implementierung – vom UML-Profil-Framework über den zweistufigen Code-Generator bis zum `stability_analysis`-Modul – demonstriert, dass diese theoretischen Einsichten in produktionsreifen Code überführt werden können. Die Aufzugstür-Fallstudie zeigt die praktische Anwendbarkeit an einem realistischen Beispiel.

Die vorliegende Arbeit steht damit am Schnittpunkt von mathematischer Theorie, ingenieurwissenschaftlicher Praxis und empirischer Wissenschaft. Die theoretische Fundierung (Kapitel 3), die praktische Implementierung (Kapitel 4–7) und die experimentelle Validierung (Kapitel 8) bilden eine methodisch geschlossene Einheit. Diese Einheit ist zugleich ein Modell für zukünftige Arbeiten, die theoretische Konzepte der Systemtheorie durch systematische Experimente in der Informatik überprüfen wollen.

Die Zukunft der Echtzeitsystementwicklung liegt in der bewussten Integration all dieser Ebenen: formale Spezifikation durch UML-Profile, quantitative Analyse durch Markov-Ketten und Warteschlangentheorie, theoretische Fundierung durch Lyapunov-Stabilität und Eigenwertanalyse, und systematische Validierung durch automatisierte Experimente. Das in dieser Arbeit entwickelte Framework ist ein erster Schritt in diese Richtung.

Literaturverzeichnis

- [OMG(2015)] Object Management Group (OMG), *Unified Modeling Language, Version 2.5*, 2015.
- [OMG(2007)] Object Management Group (OMG), *XML Metadata Interchange (XMI), Version 2.1*, 2007.
- [OMG(2019)] Object Management Group (OMG), *UML Profile for MARTE: Modeling and Analysis of Real-Time Embedded Systems, Version 1.2*, 2019.
- [OMG(2014)] Object Management Group (OMG), *Meta Object Facility (MOF) Core Specification, Version 2.5*, 2014.
- [Schmidt(2006)] Schmidt, D.C., *Model-Driven Engineering*, IEEE Computer, Vol. 39, No. 2, 2006.
- [Kopetz(2011)] Kopetz, H., *Real-Time Systems: Design Principles for Distributed Embedded Applications*, Springer, 2011.
- [Smith(1990)] Smith, C.U., *Performance Engineering of Software Systems*, Addison-Wesley, 1990.
- [Stewart(1994)] Stewart, W.J., *Introduction to the Numerical Solution of Markov Chains*, Princeton University Press, 1994.
- [Kleinrock(1975)] Kleinrock, L., *Queueing Systems, Volume I: Theory*, Wiley-Interscience, 1975.
- [Balsamo et al.(2004)] Balsamo, S., Di Marco, A., Inverardi, P., Simeoni, M., *Model-Based Performance Prediction in Software Development: A Survey*, IEEE Transactions on Software Engineering, Vol. 30, No. 5, 2004.
- [Bertolino und Mirandola(2005)] Bertolino, A., Mirandola, R., *CB-SPE Tool: Putting Component-Based Performance Engineering into Practice*, Proc. 7th Int. Symp. on Component-Based Software Engineering (CBSE), 2005.
- [Norris(1997)] Norris, J.R., *Markov Chains*, Cambridge University Press, 1997.
- [Lyapunov(1992)] Lyapunov, A.M., *The General Problem of the Stability of Motion*, Taylor & Francis, London, 1992. (Erstveröffentlichung 1892, übersetzt von A.T. Fuller.)
- [Little(1961)] Little, J.D.C., *A Proof for the Queuing Formula: $L = \lambda W$* , Operations Research, Vol. 9, No. 3, S. 383–387, 1961.
- [Gross et al.(2008)] Gross, D., Shortle, J.F., Thompson, J.M., Harris, C.M., *Fundamentals of Queueing Theory*, 4. Auflage, Wiley-Interscience, 2008.

- [Selic(2003)] Selic, B., *The Pragmatics of Model-Driven Development*, IEEE Software, Vol. 20, No. 5, S. 19–25, 2003.
- [Pooley und King(1999)] Pooley, R., King, P., *The Unified Modelling Language and Performance Engineering*, IEE Proceedings – Software, Vol. 146, No. 1, S. 2–10, 1999.
- [Hillston(1996)] Hillston, J., *A Compositional Approach to Performance Modelling*, Cambridge University Press, 1996.
- [Booch et al.(2005)] Booch, G., Rumbaugh, J., Jacobson, I., *The Unified Modeling Language User Guide*, 2. Auflage, Addison-Wesley, 2005.
- [Fowler(2003)] Fowler, M., *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3. Auflage, Addison-Wesley, 2003.
- [Gamma et al.(1994)] Gamma, E., Helm, R., Johnson, R., Vlissides, J., *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
- [Brambilla et al.(2012)] Brambilla, M., Cabot, J., Wimmer, M., *Model-Driven Software Engineering in Practice*, Morgan & Claypool Publishers, 2012.
- [Kent(2002)] Kent, S., *Model Driven Engineering*, Proc. 3rd Int. Conf. on Integrated Formal Methods (IFM), LNCS 2335, Springer, S. 286–298, 2002.
- [France und Rumpe(2007)] France, R., Rumpe, B., *Model-Driven Development of Complex Software: A Research Roadmap*, Proc. Future of Software Engineering (FOSE), IEEE, S. 37–54, 2007.
- [Becker et al.(2009)] Becker, S., Koziol, H., Reussner, R., *The Palladio Component Model for Model-Driven Performance Prediction*, Journal of Systems and Software, Vol. 82, No. 1, S. 3–22, 2009.
- [Woodside et al.(2007)] Woodside, M., Franks, G., Petriu, D.C., *The Future of Software Performance Engineering*, Proc. Future of Software Engineering (FOSE), IEEE, S. 171–187, 2007.
- [Cortellessa et al.(2011)] Cortellessa, V., Di Marco, A., Inverardi, P., *Model-Based Software Performance Analysis*, Springer, 2011.
- [Ross(2014)] Ross, S.M., *Introduction to Probability Models*, 11. Auflage, Academic Press, 2014.
- [Trivedi(2001)] Trivedi, K.S., *Probability and Statistics with Reliability, Queuing and Computer Science Applications*, 2. Auflage, Wiley, 2001.
- [Allen(1990)] Allen, A.O., *Probability, Statistics, and Queueing Theory with Computer Science Applications*, 2. Auflage, Academic Press, 1990.
- [Harrison und Patel(1993)] Harrison, P.G., Patel, N.M., *Performance Modelling of Communication Networks and Computer Architectures*, Addison-Wesley, 1993.
- [Stoyan und Stoyan(1992)] Stoyan, D., Stoyan, H., *Stochastik für Ingenieure und Naturwissenschaftler*, Akademie Verlag, Berlin, 1992.

- [Kemeny und Snell(1976)] Kemeny, J.G., Snell, J.L., *Finite Markov Chains*, Springer, 1976.
- [Grassmann(1990)] Grassmann, W.K., *Computational Methods in Probability Theory*, in: Heyman, D.P., Sobel, M.J. (Hrsg.), *Handbooks in Operations Research and Management Science*, Vol. 2, Elsevier, S. 199–254, 1990.